



Table of Contents

1. Introduction to Computer Science	26
1.1. What is Computer Science?	26
1.2. Computer Science Applications	27
1.3. Local Job Market.....	27
1.4. International Job Market	28
1.5. Are you not a student of Computer Science?	28
2. Breadth First Learning	29
2.1. Search Engine Usage Techniques	29
2.2. History of Computing.....	29
2.3. Data Storage	29
2.4. Data Manipulation.....	29
2.5. Operating System	29
2.6. Networking and the Internet.....	29
2.7. Algorithms.....	29
2.8. Programming Languages	29
2.9. Software Engineering.....	29
2.10. Data Abstraction.....	29
2.11. Database Systems.....	30
2.12. Artificial Intelligence	30
2.13. CS impact on society.....	30
2.14. Content Filtering, Spam, International laws	30
2.15. Word Processing	30
2.16. Presentations Development.....	30
2.17. Spreadsheet	30
2.18. Database MS Access.....	30
2.19. Web Page Development.....	30
Module 3	31
3. Search Engines	31
3.1. Query.....	31
3.2. How Google Works	31
3.3. Try Searching on Google	32

3.4. Use of Microphone	32
3.5. Flip a Coin	32
3.6. Query Formulation	32
3.7. Capitalization	32
Module 4	33
4. Searching Tricks	33
4.1. Weather Searching	33
4.2. Performing Calculations on Google	33
4.3. Currency Conversion.....	34
Module 5	35
5. Search Operators (1).....	35
5.1. Search on Social Media	35
5.2. Search for a price	35
5.3. Searching Hash tags	36
5.4. Exclude words from Query	36
5.5. Exact Match	36
5.6. Wild Card based Searching.....	36
Module 6	37
6. Search Operators (2).....	37
6.1. Searching within the range	37
6.2. Boolean Operators.....	37
6.3. Search with a Specific Site.....	37
6.4. Searching Related Websites	37
6.5. Searching a cached version	37
Module 7	38
7. Search Operators (3).....	38
7.1. Stocks Operator	38
7.2. Map Operator	38
7.3. Movie Operator	38
7.4. Compare Food	38
7.5. Define Operator.....	38
7.6. Image Search	38
7.7. Tilt	38

Module 8	39
8. Advanced Search Operators	39
8.1. Intitle	39
8.2. Allintitle	39
8.3. inurl and allinurl	39
8.4. Intext and allintext.....	39
8.5. Proximity Search	39
8.6. Let's solve a complex Query.....	39
Module 9	40
9. What we should not search on Internet	40
9.1. Avoiding Ads	40
9.2. Dangerous to Search.....	40
9.3. Avoid Cyber-security attacks	40
9.4. Unpleasant Results	41
Module 10	42
10. Roots of Computing	42
10.1. Abacus.....	42
10.2. Technology of Gears	42
10.3. Punch Cards.....	43
10.4. ENIAC.....	44
10.5. Factsheet of ENIAC	44
10.6. Rapid Advancement	44
Module 11	46
11. Bits.....	46
11.1. Basics	46
11.2. What Bits can represent.....	46
11.3. Bits units.....	46
11.4. Patterns Using Bits	46
Module 12	48
12. Boolean Operations.....	48
12.1. AND Boolean Operation	48
12.2. OR Boolean Operation	48
12.3. XOR Boolean Operation	48

12.4. Not Operation.....	48
12.5. Boolean Operation Example	49
Module 13	50
13. Hexadecimal Notation.....	50
13.1. Why we need Hexadecimal Notation.....	50
13.2. Hexadecimal Representation.....	50
13.3. Hexadecimal Example.....	50
Module 14	51
14. Storing a Bit.....	51
14.1. Main Memory.....	51
14.2. Main Memory Organization	51
14.3. Byte Size Organization.....	51
14.4. Memory Address	51
14.5. RAM.....	52
14.6. DRAM.....	52
14.7. SDRAM.....	52
Module 15	53
15. Magnetic Systems.....	53
15.1. Mass Storage	53
15.2. How it works	53
15.3. Zoned-bit recording.....	53
15.4. Seek Time.....	53
15.5. Rotation Delay.....	53
15.6. Access Time	53
15.7. Transfer rate	54
Module 16	55
16. Optical System.....	55
16.1. Compact Disk.....	55
16.2. DVD (Digital Versatile Disks).....	55
16.3. BDs (Blue Ray Disks)	55
Module 17	55
17. Flash Drives	55
17.1. Issues in Magnetic and Optical Systems	55

17.2. Flash Drive Technology	56
17.3. SSDs (Solid State Disks).....	56
17.4. SDs (Secure Digital Memory Cards).....	56
Module 18	57
18. Representing Text	57
18.1. Representation as Code	57
18.2. ASCII Codes	57
18.3. ASCII code example	57
18.4. Limitation of ASCII codes	57
18.5. Limitations of ASCII-extensions.....	57
18.6. Unicode	57
18.7. UTF-8	57
Module 19	58
19. Representing Numeric Values	58
19.1. Issues in storing numeric as Unicode.....	58
19.2. Binary Notation	58
19.3. Binary Notation Variations	58
Module 20	59
20. Representing Images.....	59
20.1. Pixel.....	59
20.2. Encoding Method: Pixel to Bitmap	59
20.3. Encoding Method: Handling shades	59
20.4. Encoding Method: Colorful Images	59
20.5. Brightness Chrominance	59
20.6. Image Scaling	59
20.7. Geometric Structures for Image Scaling	59
20.8. Scalable Fonts	59
Module 21	60
21. Representing Sound.....	60
21.1. Sound Amplitude.....	60
21.2. How to encode the Sound.....	60
21.3. How Sound data communication takes place.....	60
21.4. Sample Intervals	60

21.5. Alternative Method: MIDI	60
Module 22	61
22. Binary Notation.....	61
22.1. Power Method	61
22.2. Algorithm for Finding Binary Representation of a Positive Decimal Number	61
Module 23	63
23. Binary Addition.....	63
Module 24	65
24. Fraction in Binary	65
24.1. Radix Point.....	65
24.2. Example of Radix Point	65
24.3. Addition in fraction	65
Module 25	66
25. 2's Complement Notation to Store Numbers.....	66
25.1. Integer Representation in 2's Complement.....	66
25.2. Conversion between positive and negative representations	66
25.3. Addition in 2's complement notation.....	66
25.4. Problem of Overflow	67
Module 26	68
26. Excess Notation	68
26.1. Integer Representations in Excess Notation.....	68
26.2. Excess Four Notation	68
26.3. Excess Four Notation Comparisons	68
Module 27	70
27. Floating Point Notation	70
27.1. Storing Radix.....	70
27.2. Storing Fractions	70
Module 28	71
28. Truncation Errors in Floating Point Notation	71
28.1. Primitive Methods for Handling Truncation Errors.....	72
28.2. Intelligent Processing for Handling Truncation Errors	72
Module 29	73
29. Data Compression: Generic Techniques.....	73
Module 30	75

30. Data Compression: Compressing Images	75
Module 31	77
31. Data Compression: Compressing Audio and Videos.....	77
Module 32	78
32. Data Manipulation: CPU Basic	78
Module 33	80
33. Data Manipulation: Stored Program	80
Module 34	81
34. Data Manipulation: CPU Architecture Philosophies.....	81
Module 35	82
35. Data Manipulation: Machine Instruction Categories.....	82
35.1. Data Transfer Group.....	82
35.2. Arithmetic/Logic Group	82
35.3. Control Group.....	82
Module 36	83
36. Data Manipulation: Program Execution	83
Module 37	84
37. Data Manipulation: Program Execution Example	84
Module 38	87
38. Data Manipulation: Logic Operators.....	87
Module 39	89
39. Data Manipulation: Rotation and Shift	89
Module 40	90
40. Data Manipulation: Arithmetic Operators.....	90
Module 41	91
41. Data Manipulation: Role of Controller.....	91
Module 42	92
42. Data Manipulation: Direct Memory Access and Handshaking.....	92
Module 43	93
43. Data Manipulation: Communication media and communication rates.....	93
43.1. Parallel Communication	93
43.2. Serial Communication	93
Module 44	94
44. Data Manipulation: Pipelining	94
Module 45	95

45. Operating Systems: History	95
Module 46	97
46. Operating Systems: Basic Concepts (I).....	97
Module 47	98
47. Operating Systems: Basic Concepts (II)	98
Module 48	99
48. Operating Systems: Software Classification.....	99
Module 49	100
49. Operating Systems: Components (I)	100
Module 50	102
50. Operating Systems: Components (II).....	102
51. Operating Systems: Process of Booting	104
Module 52	106
52. Operating Systems: Process and its Administration	106
Module 53	107
53. Operating Systems: Process and its Administration	107
Module 54	109
54. Operating Systems: Handling Competition between Processes.....	109
Module 55	110
55. Operating Systems: Semaphores	110
Module 56	112
56. Operating Systems: Deadlock.....	112
Module 57	114
57. Operating Systems: Security Attacks from outside	114
Module 58	115
58. Operating Systems: Security Attacks from inside	115
Module 59	117
59. Networking and the Internet: Network Classification.....	117
Module 60	118
60. Networking and the Internet: Protocols	118
Module 61	120
61. Networking and the Internet: Combining Networks.....	120
Module 62	122
62. Networking and the Internet: Methods of Process Communication.....	122
Module 63	124

63. Networking and the Internet: Distributed Systems	124
Module 64	125
64. Networking and the Internet: Internet Architecture.....	125
Module 65	127
65. Networking and the Internet: Internet Addressing	127
Module 66	129
66. Networking and the Internet: Internet Applications.....	129
Module 67	130
67. Networking and the Internet: Internet Applications: Email	130
Module 68	131
68. Networking and the Internet: VoIP	131
Module 69	132
69. Networking and the Internet: Internet Multimedia Streaming.....	132
Module 70	133
70. Networking and the Internet: World Wide Web	133
Module 71	134
71. Networking and the Internet: Web implementations	134
Module 72	135
72. Networking and the Internet: HTML	135
Module 73	136
73. Networking and the Internet: HTML	136
Module 74	138
74. Networking and the Internet: More on HTML.....	138
Module 75	139
75. Networking and the Internet: XML	139
Module 76	141
76. Networking and the Internet: Client Side and Server Side.....	141
Module 77	143
77. Networking and the Internet: Layered Approach to Internet Software (I)	143
Module 78	145
78. Networking and the Internet: Layered Approach to Internet Software (II).....	145
Module 79	146
79. Networking and the Internet: Layered Approach to Internet Software (III)	146
Module 80	148
80. Networking and the Internet: TCP /IP Protocol Suite	Page 10 of 381 148

81.	Networking and the Internet: Security (Forms of Attacks)	150
------------	---	------------





Module 1**1. Introduction to Computer Science****1.1. What is Computer Science?**

Computer Science is the discipline that seeks to build a scientific foundation for such topics as:

Hardware

Computer hardware is the collection of physical parts of a computer system. This includes the computer case, monitor, keyboard, and mouse. It also includes all the parts inside the computer case, such as the hard disk drive, motherboard, video card, and many others. Computer hardware is what you can physically touch.

Software

Computer software, also called software, is a set of instructions and its documentations that tells a computer what to do or how to perform a task. Software includes all different software programs on a computer, such as applications and the operating system.

Programing

Computer programming is the process of designing and building an executable computer program for accomplishing a specific computing task.

Networks

A computer network is a set of computers connected for the purpose of sharing resources. The most common resource shared today is connection to the Internet. Other shared resources can include a printer or a file server. The Internet itself can be considered a computer network.

Graphics

Computer graphics is the discipline of generating images with the aid of computers. Today, computer graphics is a core technology in digital photography, film, video games, cell phone and computer display, and many specialized applications.

Robots

A robot is a machine—especially one programmable by a computer— capable of carrying out a complex series of actions automatically. Robots can be guided by an external control device or the control may be embedded within

Database

Database, also called electronic database, any collection of data, or information, that is specially organized for rapid search and retrieval by a computer. Databases are structured to facilitate the storage, retrieval, modification, and deletion of data in conjunction with various data-processing operations. A database management system (DBMS) extracts information from the database in response to queries.

Security

Security are those controls that are put in place to provide confidentiality, integrity, and availability for all components of computer systems. These components include data, software, hardware, and firmware.

Algorithmic Solutions

An algorithm is a set of instructions designed to perform a specific task.

Information Processing

Information processing refers to the manipulation of digitized information by computers and other digital electronic equipment, known collectively as information technology (IT). Information processing systems include business software, operating systems, computers, networks and mainframes.

We will be learning the details of these terms throughout the courses in different modules.

1.2. Computer Science Applications

Furthermore, Computer Science has applications in almost all domains such as:

- ✓ Telecom
- ✓ Banks
- ✓ Hospitals
- ✓ Software Development
- ✓ Service Industry
- ✓ Pak Army
- ✓ Freelancing
- ✓ and many more

1.3. Local Job Market

According to famous Job market website in Pakistan, most of the jobs are available in Computer Science, for example Figure 1 shows job opportunities when filtered using “By Function”, and Figure 2 represents job opportunities when filtered using “By Industry”. In both cases, the job opportunities in Computer Science are higher than rest of the fields.



Browse Jobs in Pakistan			
By Function		By Industry	By City
Software & Web (830)	Administration (180)	Computer Networking (72)	
Sales & BD (763)	Telemarketing (130)	Clerical & Front Office (68)	
Accounts & Finance (357)	Writing (127)	Hotel/Restaurant Ma... (60)	
Customer Service (326)	Engineering (113)	Quality Assurance (QA) (54)	
Marketing (298)	Health & Medicine (102)	Data Entry (48)	
Teaching & Education (210)	Human Resources (102)	More Functional Areas	
Creative Design (188)	Operations (88)		

Figure 1: Job Opportunities in Pakistan when classified as “By Function”

Browse Jobs in Pakistan

By Function

By Industry

By City

By Company

Information Technol... (1274)	Insurance / Takaful (137)	Textiles/Garments (77)
Services (341)	Healthcare/Hospital/... (113)	Consultants (76)
Manufacturing (275)	N.G.O./Social Services (98)	Importers/ Distributo... (74)
Education/Training (257)	Banking/Financial Se... (95)	Food & Beverages (71)
Call Center (200)	BPO (87)	Advertising/PR (70)
Real Estate/Property (158)	Business Development (84)	More Industries
Telecommunication/... (144)	Engineering (82)	

Figure 2: Job Opportunities in Pakistan when classified as “By Industry”

1.4. International Job Market

Similarly, internationally, jobs related to Computer Science are ranked on the top. For example, forbes magazine¹, one of the acclaimed agencies in US claims that Software Developer has been ranked as Number 1 job in the US. In Computer Science, the following areas have been ranked by the Forbes magazine.

- ✓ Artificial Intelligence and Machine Learning
- Data Science
- Virtual Reality
- IoT
- Back-End Developer
- Front-End Developer
- UI Designer
- Full-Stack Engineer
- IT Manager
- Quality Assurance Expert

All of this discussion would be helpful to make you motivated all the time and to be happy with the decision you have made to choose Computer Science and a career.

1.5. Are you not a student of Computer Science?

Those who are not studying CS, even then this course will be helpful for them too. As this course covers all basic concepts of Computer Science which you would require in whatever field of study you are working. You know studying basics of Computer Science is compulsory for everyone even you are studying Business, Engineering, or Sciences. This course has been made very easy and interactive to make sure that you learn properly the basics of Computer Science and can apply them in your own studies.

¹ <https://www.forbes.com/sites/richardgano/2018/01/29/the-best-jobs-for-2018-and-beyond/#1940306742c4>
accessed on 16th July

Module 2

2. Breadth First Learning

This course will give basic introduction to almost all courses of Computer Science. Such strategy of learning is known as Breadth First Learning. The other strategy is called Depth First Learning in which one particular course is first covered in details and then next course is covered. The Breadth First Learning helps students to first know what they will be learning in the whole degree program.

We will cover:

1. Abstract view of all major courses in CS
2. Understanding what will be studied in CS
3. Why each course is important
4. Clarifying the bigger Picture

Introduction to the topics will be covered in this course:

2.1. Search Engine Usage Techniques

We will cover the searching techniques in this topic. How can you effectively search over the internet using popular search engine?

2.2. History of Computing

We will cover in this topic that how today's computer evolved from the basic idea.

2.3. Data Storage

Whatever data you enter the computer, this data needs to be stored somewhere in the hardware, such storage science will be understood in this topic.

2.4. Data Manipulation

Then the next thing is to study how data is manipulated. For example, how basic arithmetic operations (+, -, *, /) are performed in computer and how some advanced operators are performed.

2.5. Operating System

We will study about Operating System which is the overall in-charge of the computer system.

2.6. Networking and the Internet

We will also study how different computers communicate over the internet.

2.7. Algorithms

We will then study the fundamental concept of algorithm in Computer Science. We define algorithm as: "Set of Steps in a sequence to perform a certain task".

2.8. Programming Languages

Then we will study programming tools. How algorithm can be written to achieve the said goal in a computer programming. We will cover just basics of C++.

2.9. Software Engineering

Here we will cover that how a complete software is developed starting from the requirement gathering phase to designing, implementation and testing.

2.10. Data Abstraction

In Computer Science, designing a complex and large system becomes a challenge when we discuss design at a great depth, data abstraction hides complexities from the designer and helps to comprehend things with more ease. Such things will be discussed in data abstraction like Arrays, Stack, Queue, and trees.

2.11. Database Systems

In Computer Science, we store and link data in an organized way using state-of-the-art Database Management Systems (DBMS). We will discuss database design, DBMS functionalities in these modules.

2.12. Artificial Intelligence

You know Computer is a dumb device, cannot think or take inferential decisions. However, Artificial Intelligence (AI) is a specialized field of Computer Science that aims at building such Computer Systems which can act intelligently.

2.13. CS impact on society

We will also learn the impact of Computer Science on society, social setups and on humans.

2.14. Content Filtering, Spam, International laws

Here we will cover content filtering, dealing with the spams and international laws related to Computer Science in terms of data and privacy.

2.15. Word Processing

Word processor like Microsoft Word is an application software which helps to build editable word documents, we will cover all major functionalities of Microsoft Word.

2.16. Presentations Development

Presentations are built in specialized tools like Microsoft PowerPoint. We will cover Microsoft PowerPoint tool in details to design and animate the presentation slides.

2.17. Spreadsheet

To perform certain types of calculations on data, there are some specialized software like Spreadsheet. One of such software is Microsoft Excel which will be covered in such modules.

2.18. Database MS Access

Database learnt above will be implemented using DBMS tool like Microsoft Access.

2.19. Web Page Development

We will also learn that how a web page can be made easily using built-in tools like Dreamweaver.

ZAINEE
ACADEMY

Module 3

3. Search Engines

Search Engine like Google index web pages and helps you to retrieve the relevant web pages based on your queries. Starting from today, in next few modules, we will focus on techniques and short-cuts to retrieve most relevant information from Google.

There are many search engines like Google, Yahoo, and MSN etc. However, we will focus on Google because it is most widely used search engine and has most of the market share of searching over the internet as illustrated from the Figure 3.



Figure 3: Market share of Search Engines [gs.statcounter.com/search-engine-market-share]

To search on Google, type: <https://www.google.com/> on the web browser, you will be shown the screen like shown in the Figure 4.

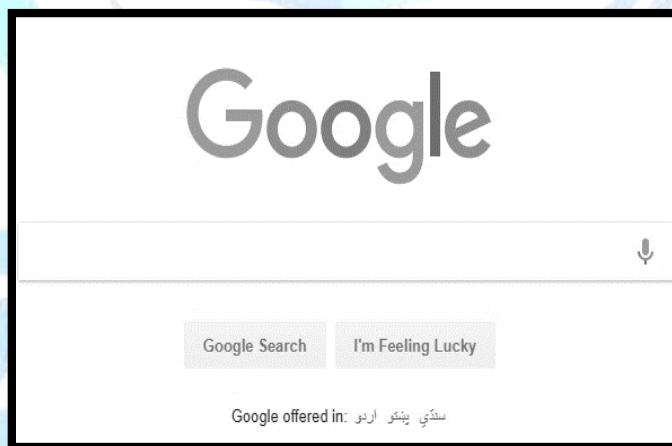


Figure 4: Google Search Engine home page

3.1. Query

Query is the set of words given to the search engine for searching for example, if you are interested to find “Virtual University” on the internet, you will provide this the text box available in the Figure 4 and you will be given the relevant results.

3.2. How Google Works

When you search in the Google by typing any query in the text box shown in the Figure 4, Google finds all those pages which contains all the terms given in the query. Try the following queries and see the results:

3.3. Try Searching on Google

1. “Airport”
2. “Where is the closest airport”
3. “Virtual University in Islamabad”

[Note: If you find any query with double quotations like “”, you should not give this query in double quotations on Google, I am attaching double quotations just to make sure that you can understand it is a query not a running text. Otherwise posting a query in double quotations has a particular meaning in Google which you will learn in next modules.]

3.4. Use of Microphone

There is another option of microphone as can be seen in the Figure 4 on the right side. If you click on you can search by speaking.

3.5. Flip a Coin

If you want to flip a coin and you do not have the coin to flip, you can use Google to do that. To achieve this just type the following on Google:

“Flip a Coin”

3.6. Query Formulation

Always remember, when you search, Google will try to identify the pages on which the provided term is available. For example, if your head is in pain and you type:

“Head Hurts”

It might not give you relevant results as when head is in pain, then its medical name is headache. So, your query should be:

“Headache “

Similarly, if you are interested to search the name of the head of the virtual university and type the query as:

“Head of the virtual university”

Google can give you wrong result for example the web page which contains the term “head” and “virtual university” that might be the head office of virtual university. The right query could be:

“Rector of virtual university”

As the head of virtual university is called rector, therefore in this way it will give you more chance to search the right person effectively.

3.7. Capitalization

Google does not distinguish the capital letters or small letters. This is called case-insensitive in computer science. For example, search

“COMPUTER SCIENCE” or “computer science”

Will not make a difference.

Module 4

4. Searching Tricks

In this module we will learn about searching tricks while searching on Google.

4.1. Weather Searching

You can simply add the location with the word “Weather” to form such query for example:

Query “Weather Lahore” will give you a screen shown in the Figure 5.

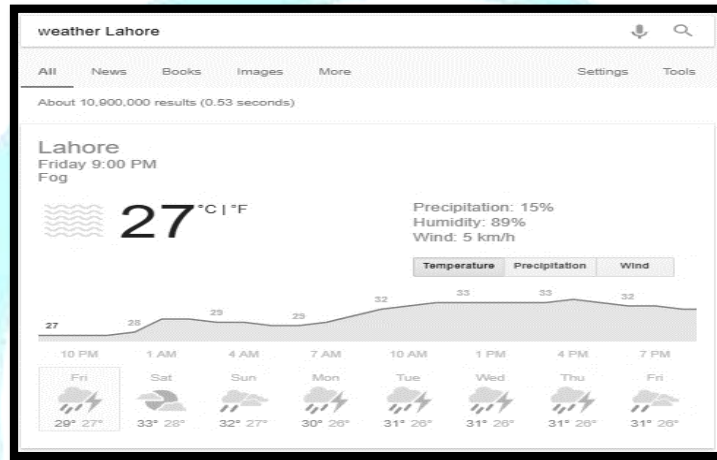


Figure 5: Weather searching on Google

4.2. Performing Calculations on Google

To perform calculations on Google, just type whatever you want to perform like if we want to multiply 12 with 391, we will type the query like:

$$12 * 391$$

and you can see in the screen shown in the Figure 6 that the Google has automatically started the calculator, have understood our query, have translated our query and given this query to calculator application and have shown the result as well.

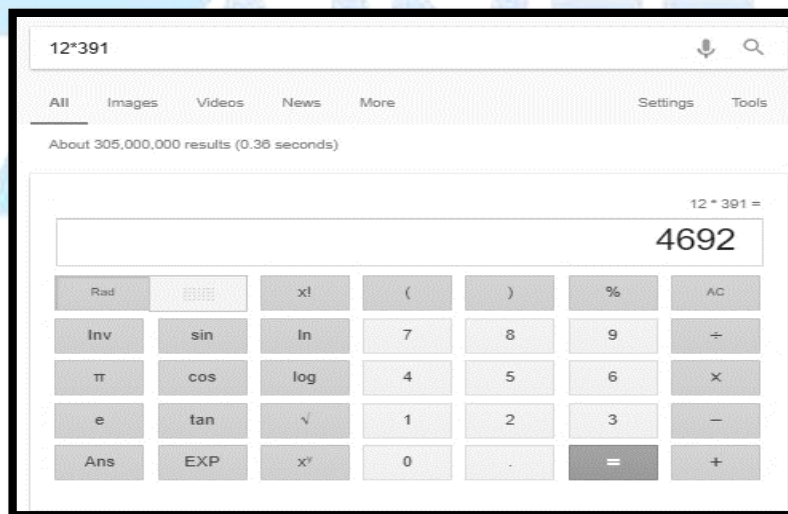


Figure 6: Calculations on Google

Perform the following queries on your computer:

1. 12-5
2. Sin 90
3. Tan 80
4. $80/100*200$
5. Subtract 10 from 30

4.3. Currency Conversion

You can also see latest currency conversion on Google. For example, you want to see how much 100 Euros will be when converted to Pakistani rupees. Here is the query and result are shown in the Figure 7.

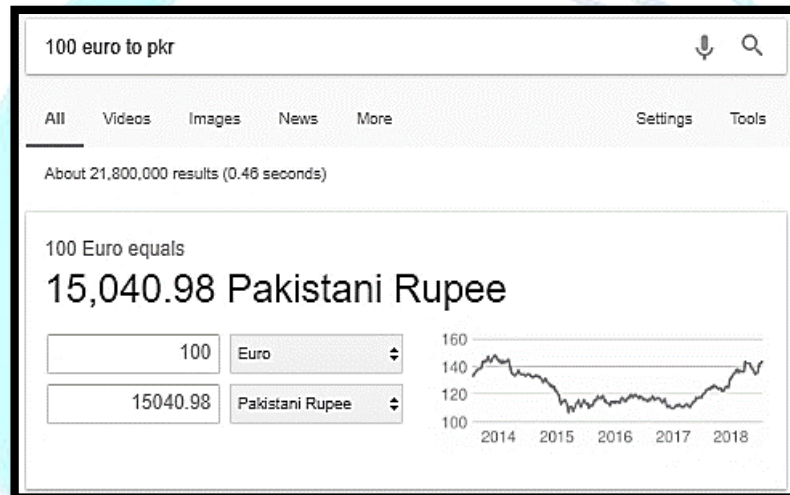


Figure 7: Currency conversion on Google

Try the following queries and see the results:

1. Kph in Mph
2. m in cm
3. Pakistan Cricket team
4. Baadshahi Mosque
5. Minar e Pakistan

Module 5

5. Search Operators (1)

In this module, we will further learn about different search operators that can be used on the Google to retrieve more relevant and focused results.

5.1. Search on Social Media

When you want to use Google and want to search the query on a particular social media, you can add “@” symbol. For example, when we give queries:

“Fifa World cup @facebook” and “Fifa World Cup @Twitter”, the results can be seen in the Figure 8.

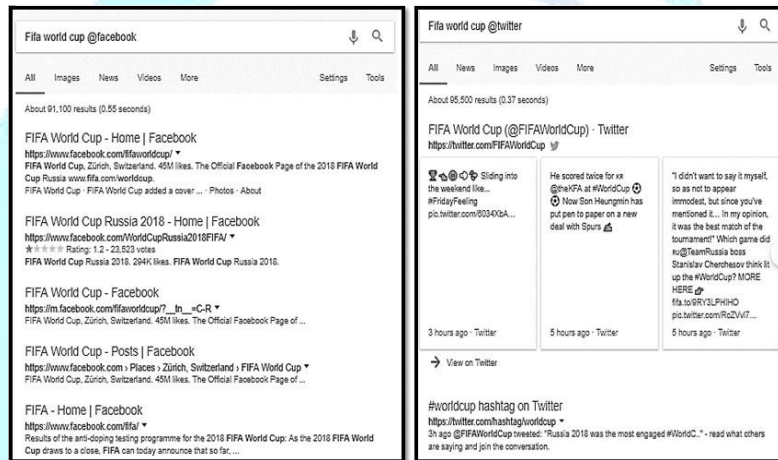


Figure 8: Searching in Social media over Google

5.2. Search for a price

Put pkr in front of a number. For example, give a query like:

“Laptop pkr 50000” it will give you laptops whose price is around 50000 as shown in the Figure 9.

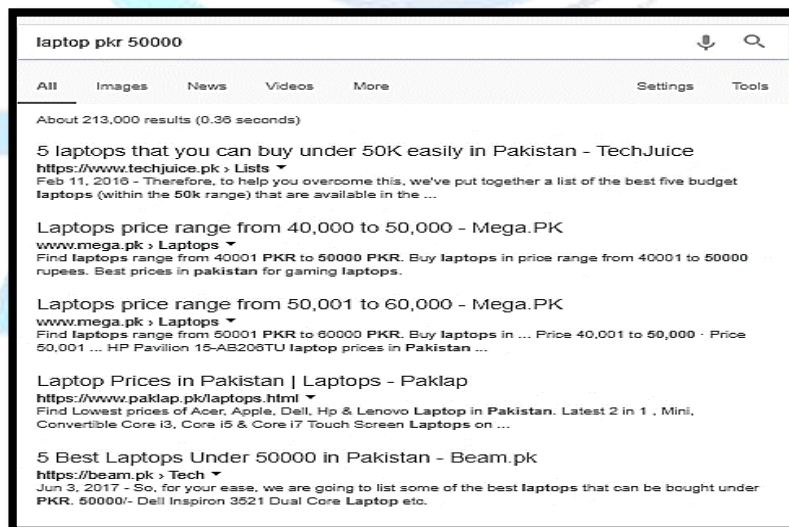


Figure 9: searching for a price at Google

5.3. Searching Hash tags

On Social media, sometimes, a hash tag is very popular, if you are interested to identify such pages using hash tag, you can use the query like:

“#education”

You will get result like shown in the Figure 10.

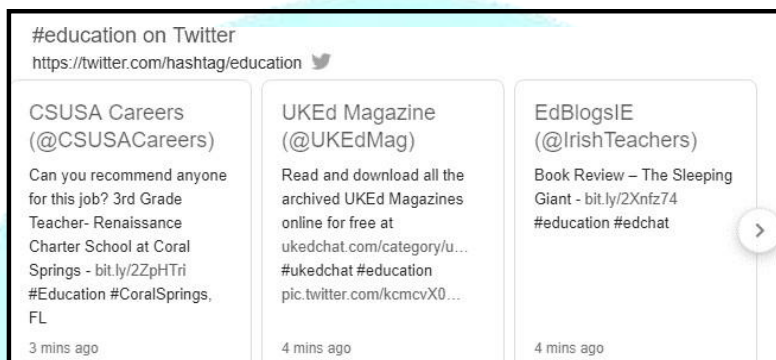


Figure 10: Searching Hash tag

5.4. Exclude words from Query

One term could mean more than one thing for example word: “Jaguar” is used in two meanings such as: animal and car. If you want to exclude all pages containing pages of cars, you can write the query like “Jaguar -cars”.

5.5. Exact Match

If you want to search for an exact phrase enclose it in double quotes, in this case, the search engine will search where it can find the exact phrase as it is. For example, searching

“Tallest Building in Pakistan” will give only those pages which contains exact this phrase, however, if you give the query like:

Tallest Building in Pakistan

Google will give you all those pages which contains all or any of these words.

5.6. Wild Card based Searching

You can search a phrase when even you do not know what word would be there at a particular location for example, you want to search all those pages which contains any word in the start and then contains the exact phrase “is thicker than water”. You can write:

“* is thicker than water”

This query will give you all those pages which contains any word in the start and then have the exact phrase “is thicker than water”

Please note if you search “is thicker than water”, it might give you those pages where such question has been posed or there is no word in the start, but the phrase is written like “is thicker than water”.

Module 6

6. Search Operators (2)

This module will further explore more search operators.

6.1. Searching within the range

If you want to search within the range of number, you can use two dots. For example: “laptop pkr25000.pkr35000” will try to give those pages which contains laptop options between the specified ranges.

6.2. Boolean Operators

We can use Boolean operators specially “And” “Or” to find relevant pages. For example, writing a query:

Computer and Science

Will give you all those pages which have both words but not necessarily they occur together.

Similarly writing a query:

Computer Or science

Will give you those pages in which either the term “computer” is written, or “science” is written. Off course, it will also return those pages in which both terms are also written.

6.3. Search with a Specific Site

If you are interested to search the query in a website, we can write like:

- ✓ “virtual university site: youtube.com”

This will give us those pages of virtual university which are contained within the youtube.com website.

6.4. Searching Related Websites

For example, you like the videos on YouTube and interested to know are there some more websites like YouTube? you can type the query:

- ✓ “related:youtube.com”

Furthermore, you can also search information for a particular website like if you write:

- ✓ “info:youtube.com”

It will provide the information about this website.

6.5. Searching a cached version

If the website is down and you are interested to see the cached version of the website, you can use the following query:

- ✓ cache:youtube.com

Searching on a file type

If you are interested to search a query for a specific file type, you can include such file type in the query. For example, we are interested to search for “virtual university” but all those pages having file type PDF, so you can write a query:

- ✓ “Virtual University” filetype:pdf
- ✓ “Virtual University” ext:pdf

Module 7

7. Search Operators (3)

In this module we will further learn about some remaining search operators.

7.1. Stocks Operator

Using this operator, we can find the trend of the stock market for a particular company for example, if we want to see what the stock market trend for the company “Apple” is, we can write the following query:

✓ stocks:aapl

7.2. Map Operator

If we are interested to see the map of some location, we can use map operator in the following way:

✓ map:Lahore

7.3. Movie Operator

Using movie operator, we can find information about any movie. For example, try:

✓ movie:steve jobs

7.4. Compare Food

If you want to compare two foods based on their nutrient values, you can go and explore the following website:

✓ <https://www.myfooddata.com/>

7.5. Define Operator

If you are interested to find the definition of a particular term, type:

✓ Define:Computer

7.6. Image Search

There is a **dedicated web link** at Google:

<https://images.google.com/>

Using this link, you can search images even by giving a query of another image. Google will provide you other images which look quite similar to the queried image.

7.7. Tilt

This is an interesting option available at Google. Using this option, you can perform number of funny things with the shown screen of the Google. For example, you can rotate the **Google screen to 360 degree etc.** To do it type the following query:

✓ Tilt

On the show results, select the appropriate link like: <https://elgoog.im/tilt/> and then perform the interesting tricks with the Google screen.

Module 8

8. Advanced Search Operators

In this module, we will learn some advanced search operators.

8.1. Intitle

This operator will give us only those web pages in which the searched term appears in the title of the web pages. For example, if we are interested to search web pages in which the phrase “iPhone vs. android” appears in the title, we can write the following query.

- ✓ Intitle:“iPhone vs. android”

8.2. Allintitle

The next similar operator is allintitle without double quotation. This will find all those web pages in which any of the word “iPhone” or “android” is found. The query would be:

- ✓ allintitle:iphone vs. android

8.3. inurl and allinurl

The “inurl” operator finds all those web pages which contains the mentioned query in the url of the web pages. For example, if we try the following query:

- ✓ inurl:2018 “virtual university”

This will give us all those web pages in which 2018 is written and the phrase “virtual university” is written.

As we had allintitle above, here again we have allinurl which means it will find any of the those mentioned terms within the url and will fetch all those pages in which any of the mentioned query terms are present.

8.4. Intext and allintext

If we are interested to search anywhere in the body of the document, we can use intext operator, for example:

- ✓ intext:“virtual university admissions 2018”

this will give all those pages in which the whole phrase is present as it is. Furthermore, we can try:

- ✓ allintext:virtual university admissions 2018

Which will return all those pages in which any of those terms are available.

8.5. Proximity Search

Suppose we are interested to search two words:

- Education
- Virtual university

But both words should be collocated on the web page within the margin of three words. To achieve this, we can use the operator around.

- ✓ education AROUND(3) "virtual university"

This query will give all those web pages on which both words are available within the margin of three words.

8.6. Let's solve a complex Query

As we have learnt many operators, now let's solve a bit complex query. The query is:

“We are interested to see how many of pages of a website are not secured?”

- ✓ site:youtube.com –inurl:https

Module 9

9. What we should not search on Internet

We have learnt different tricks and operators to ease the task of searching on the Google. However, there are many things which we should not search on Google. But Why?

There are three main reasons:

- 1) **Google Adds:** Google give you advertisements based on your search queries, history, geographical location etc.
- 2) **Security Agencies:** law and enforcement agencies might see your queries and can enquire you or can reach at your location if they see a sensitive term searching again and again.
- 3) Sometimes, when we search like free music, we can be trapped to provide a virus which might be harmful for our computer.

By keeping in view, the above three points, the followings and related things must not be searched over the internet:

9.1. Avoiding Ads:

Searching the followings can avoid un-necessary ads from the search engines.

- ✓ Your email
- ✓ Medical issues and drugs
- ✓ Your name
- ✓ Your location
- ✓ Your favorite things

9.2. Dangerous to Search

Searching the followings can alert the law enforcement agencies or any organization who is keeping eye on the security threats. There is one typical such Pressure Cooker bomb story. When someone searched this term and the security agency representatives reached to the particular location from where such searching was performed. Therefore, avoid searching the followings:

- ✓ pressure cooker bomb story
- ✓ Attacks
- ✓ Suicide bomb
- ✓ Killers/Underworld
- ✓ Terrifying insects
- ✓ Killing animals
- ✓ Poisons
- ✓ Murder
- ✓ Medical Symptoms
- ✓ How to make computer Virus
- ✓ Hacking
- ✓ About Religion
- ✓ About Politics

9.3. Avoid Cyber-security attacks

Avoid searching the followings:

- ✓ Free Music

The hackers normally add many viruses to such websites as the information seeker of free music etc.

9.4. Unpleasant Results

You might search the followings or similar thing and might get unpleasant results which might depress you.

- ✓ Smokers Lungs
- ✓ Skin Condition



Module 10

10. Roots of Computing

Here we will see how we moved from mechanical driven machines to electronics driven machine (Computer).

10.1. Abacus

One of the earlier computing devices was the abacus. History tells us that it probably had its roots in ancient China and was used in the early Greek and Roman civilizations. The machine is quite simple, consisting of beads strung on rods that are in turn mounted in a rectangular frame as shown in the Figure 11.

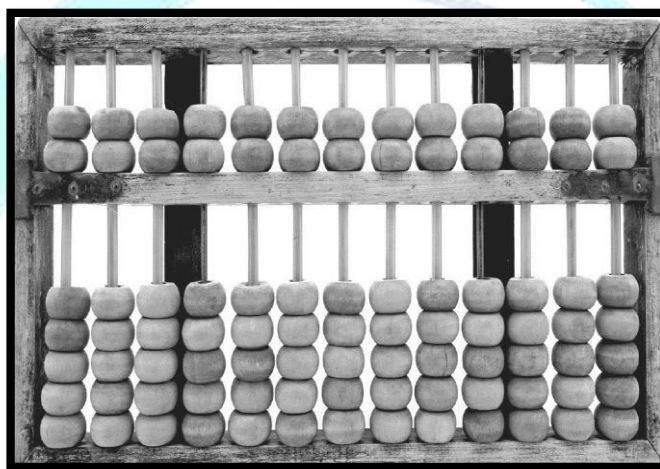


Figure 11: Abacus

10.2. Technology of Gears

In the time period after the Middle Ages and before the Modern Era, the quest for more sophisticated computing machines was seeded. A few inventors began to experiment with the technology of gears. Among these were Blaise Pascal (1623–1662) of France, Gottfried Wilhelm Leibniz (1646–1716) of Germany, and Charles Babbage (1792–1871) of England. These machines represented data through gear positioning, with data being entered mechanically by establishing initial gear positions. Output from Pascal's and Leibniz's machines was achieved by observing the final gear positions. Babbage, on the other hand, envisioned machines that would print results of computations on paper so that the possibility of transcription errors would be eliminated. One typical diagram is shown in the Figure 12.

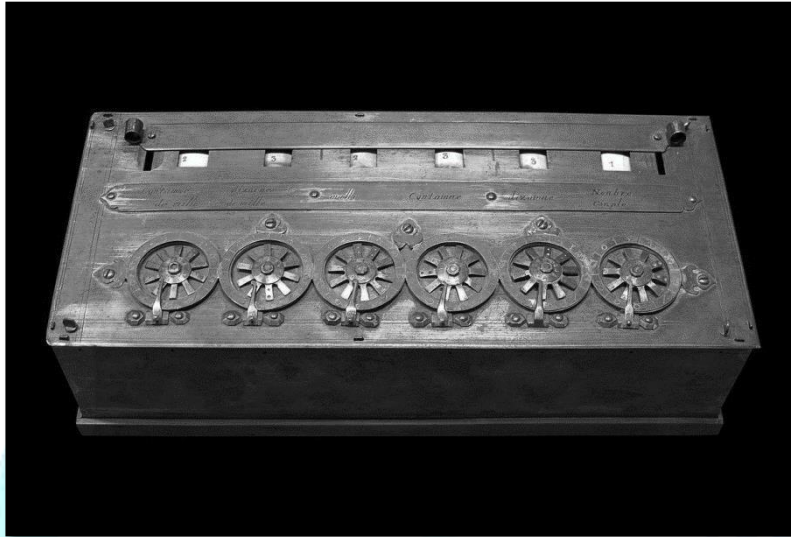


Figure 12: Gear Technology

10.3. Punch Cards

Herman Hollerith (1860–1929), who applied the concept of representing information as holes in paper cards to speed up the tabulation process in the 1890 U.S. census. (It was this work by Hollerith that led to the creation of IBM.) Such cards ultimately came to be known as punched cards and survived as a popular means of communicating with computers well into the 1970s. One typical punch card machine is illustrated in the Figure 13.



Figure 13: Punch Cards

Mechanical-driven to Electronics-driven machines

Nineteenth-century technology was unable to produce the complex gear-driven machines of Pascal, Leibniz, and Babbage cost-effectively. But with the advances in electronics in the early 1900s, this barrier

was overcome. Examples of this progress include the electromechanical machine of George Stibitz, completed in 1940 at Bell Laboratories, and the Mark I, completed in 1944 at Harvard University by Howard Aiken and a group of IBM engineers. These machines made heavy use of electronically controlled mechanical relays. In this sense they were obsolete almost as soon as they were built, because other researchers were applying the technology of vacuum tubes to construct totally electronic computers.

10.4. ENIAC

More flexible machines, such as the ENIAC (electronic numerical integrator and calculator) developed by John Mauchly and J. Presper Eckert at the Moore School of Electrical Engineering, University of Pennsylvania, soon followed as shown in the Figure 14.

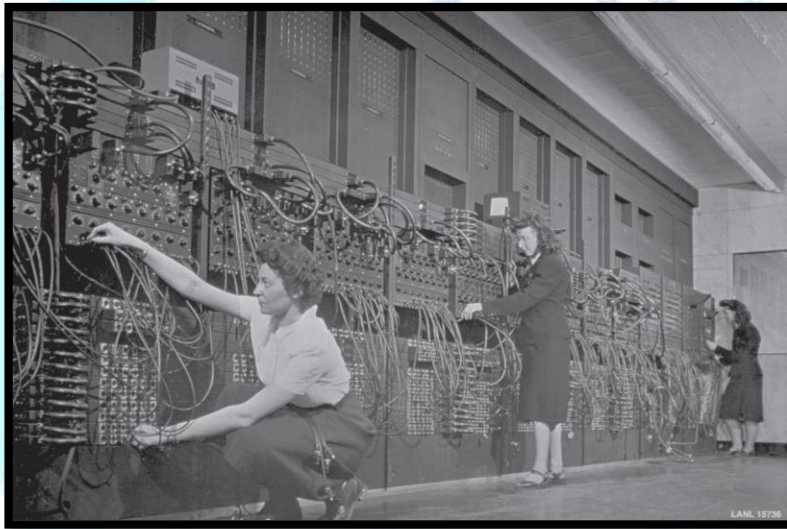


Figure 14: ENIAC

10.5. Factsheet of ENIAC

- ✓ Occupied 1800 square feet
- ✓ 20,000 vacuum tubes
- ✓ 1500 relays
- ✓ 10,000 capacitors
- ✓ 70,000 registers
- ✓ 200 Kilo Watt electricity
- ✓ Weight 30tons
- ✓ Cost = \$487,000
- ✓ PKR = 62.5 millions

[<https://www.computerhope.com/jargon/e/eniac.htm>]

10.6. Rapid Advancement

Since the invention of ENIAC, there was a rapid advancement. Some key milestones are listed below:

- ✓ Transistor
- ✓ Integrated Circuits
- ✓ Size reduction
- ✓ Processing Power doubling in 2 years
- ✓ IBM launched PC in 1981

- ✓ Web
- ✓ Smart Phones



Module 11

11. Bits

In Computer, all kind of information is stored in bits. Bit is the basic unit of storage.

11.1. Basics

- ✓ Information is coded as pattern of 0 or 1
- ✓ Short form of Binary Digits
- ✓ One bit can contain only one value 0 or 1

11.2. What Bits can represent:

- ✓ Representing numbers, text, audio, video, image etc.
- ✓ In Chip electric Charge 0/1

11.3. Bits units

Table 1 shows the bits and their equivalent units.

Table 1: Bits Units

Unit	Equivalent
1 Byte	8 bits
1 Kilo Byte (KB)	1024 Bytes
1 Mega Byte (MB)	1024 KB
1 Giga Byte (GB)	1024 MB
1 Tera Byte (TB)	1024 GB

11.4. Patterns Using Bits

In one bit we can represent two patterns either 0 or 1, if we have 2 bits then 4 different patterns can be representing as shown in the Table 2.

Table 2: bits patterns for two bits

00
01
10
11

Let's comprehend all bit patterns, we will have the Table 3.

Table 3: Bits and their number of patterns

Bits	Number of Patterns
1	2
2	4
3	8
4	16
5	32
6	64
7	128
8 = 1 byte	256

At page 577 of your book², you can find ASCII codes, using which we can represent different characters. For example, representing character “A” we need to represent it using digit 65. The bit pattern of 65 would be:

01000001.

How did we get this, we will learn in details, at the moment you can just try to learn very basic method of converting decimal to binary. As the binary is represented using base 2. To represent any number, please form the representation as shown in the Table 4.

Table 4: representing decimal to Binary

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1

Now whatever number you want to represent, put 1 the respective bit. For example if you want to represent the decimal 2, just put 1 on the second bit from the right side and put zeros otherwise, you will get the number 2 in binary as shown in the Table 5.

Table 5: Representing Decimal 2 in Binary

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1
0	0	0	0	0	0	1	0

Similarly, 8 will be represented as: 00001000,

65 will be represented as: 01000001

² Computer Science – An Overview by J. Glenn Brookshear and Dennis Brylow

Module 12

12. Boolean Operations

To understand how individual bits are stored and manipulated inside a computer, it is convenient to imagine that the bit value *false* and the bit 1 represents the value *true*. Operations that manipulate true/false values are called

Boolean operations, in honor of the mathematician George Boole (1815–1864), who was a pioneer in the field of mathematics called logic. Three of the basic Boolean operations are AND, OR, and XOR (exclusive or). The “AND” operation and “OR” are summarized in the Figure 15.

12.1. AND Boolean Operation

It produces the output as 1 when both of the inputs are 1. Otherwise, it gives output as 0 as shown in the Figure 15.

12.2. OR Boolean Operation

It produces output of 1 when any of the inputs are 1, otherwise 0 as shown in the Figure 15.

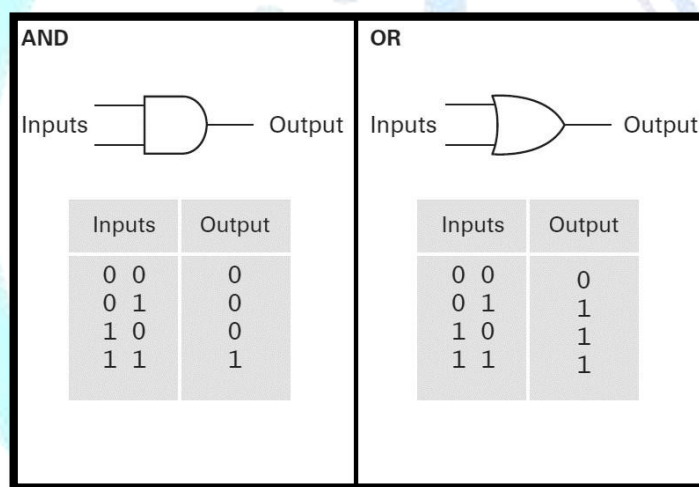


Figure 15: AND, OR Boolean operation

12.3. XOR Boolean Operation

XOR (Exclusive or) is another Boolean operation which produces the output of 1 when both inputs are different. It produces 0 when both inputs are same as shown in the Figure 16.

12.4. Not Operation

Not operation takes one input, produces 1 when input is 0 and produces 0, when the input is 1 as shown in the Figure 16.

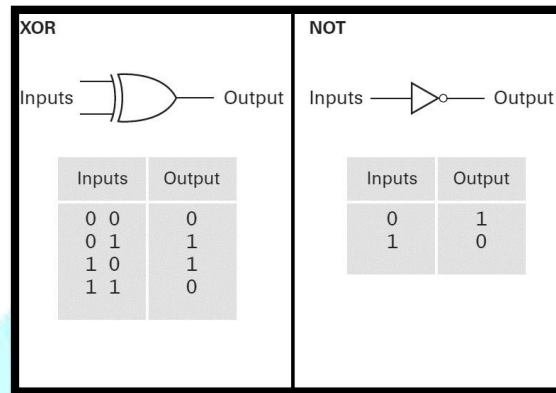


Figure 16: XoR, Not operation

12.5. Boolean Operation Example

To demonstrate the working of the Boolean operations, an example has been shown in the Table 6. First two rows represent the bit patterns as “A” and “B” and next rows are self-explanatory to perform “AND”, “OR”, “XOR”, “Not” on both “A” and “B”.

Table 6: Example of Boolean operation

A	0	0	1	1	0	1	1	0
B	1	0	0	1	1	0	0	1
AND	0	0	0	1	0	0	0	0
OR	1	0	1	1	1	1	1	1
XOR	1	0	1	0	1	1	1	1
Not (A)	1	1	0	0	1	0	0	1
Not (B)	0	1	1	0	0	1	1	0

ZAINEE
ACADEMY

Module 13

13. Hexadecimal Notation

13.1. Why we need Hexadecimal Notation

When considering the internal activities of a computer, we must deal with patterns of bits, which we will refer to as a string of bits, some of which can be quite long. A long string of bits is often called a **stream**. Unfortunately, streams are difficult for the human mind to comprehend. Merely transcribing the pattern 101101010011 is tedious and error prone. To simplify the representation of such bit patterns, therefore, we usually use a shorthand notation called **hexadecimal notation**, which takes advantage of the fact that bit patterns within a machine tend to have lengths in multiples of four. In particular, hexadecimal notation uses a single symbol to represent a pattern of four bits. For example, a string of twelve bits can be represented by three hexadecimal symbols.

13.2. Hexadecimal Representation

Table 7 presents the hexadecimal notation.

Table 7: Hexadecimal Notation

Bit pattern	Hexadecimal representation
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

13.3. Hexadecimal Example

Let's solve an example from the book, on page 38, Q#5 (b), we want to represent the following bit pattern in hexadecimal.

111010000101010100010111

Our strategy would be to divide it in four bit pattern as shown in the Table 8.

Table 8: hexadecimal example

1110	1000	0101	0101	0001	0111
E	8	5	5	1	7

So the hexadecimal notation for:

111010000101010100010111

Is E85517.

Module 14

14. Storing a Bit

14.1. Main Memory

For the purpose of storing data, a computer contains a large collection of circuits (such as flip-flops), each capable of storing a single bit. This bit reservoir is known as the machine's **main memory**. A typical Flip-flop is shown in the Figure 17

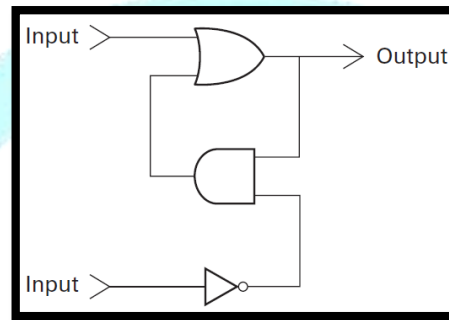


Figure 17: Flip flop

14.2. Main Memory Organization

- ✓ Manageable unit Cells – 8 bits
- ✓ Household devices have few hundreds of cells
- ✓ Large computer may have billions of cells

14.3. Byte Size Organization

In Computer Science, there is no left or right, however, to understand the significance of bits, we represent most significant bits on left side while least significant bits on the right side as shown in the Figure 18.

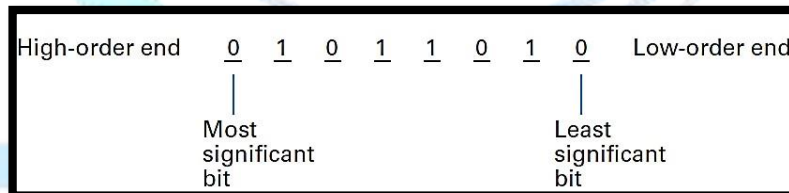


Figure 18: Byte organization in memory

14.4. Memory Address

To identify individual cells in a Computer's main memory, each cell is assigned a unique "name," called its **address**. It has been shown in the Figure 19.

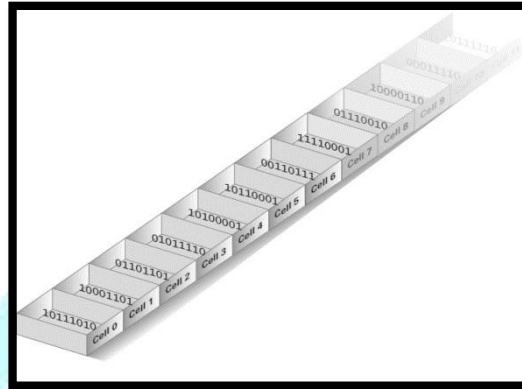


Figure 19: Memory arranged with respect to addresses

14.5. RAM

The cells (represented in Figure 19) can be accessed independently as required. To reflect the ability to access cells in any order, a computer's main memory is often called **random access memory (RAM)**.

14.6. DRAM

Dynamic ram – Stores bits as tiny electric Charge, refreshes many times a second.

14.7. SDRAM

Synchronous DRAM is used in reference to DRAM that applies additional techniques to decrease the time needed to retrieve the contents from its memory cells.

ZAINEE
ACADEMY

Module 15

15. Magnetic Systems

15.1. Mass Storage

Due to the volatility and limited size of a computer's main memory, most computers have additional memory devices called **mass storage** (or **secondary storage**) systems, including magnetic disks, CDs, DVDs, magnetic tapes, flash drives, and secondary storage of which we will discuss shortly). The

advantages of mass storage systems over main memory include less volatility, large storage capacities, low cost, and in many cases, the ability to remove the storage medium from the machine for archival purposes.

15.2. How it works

One typical device has been shown in the Figure 19.

- ✓ Thin spinning disk with magnetic coating to hold data
- ✓ Read/Write Heads placed above or below disk
- ✓ Each head traverses a circle called track.
- ✓ Normally, each track is divided into equal sectors
- ✓ Sectors have equal number of bits: 512 bytes to few KBs.
- ✓ Outer tracks contain more information.

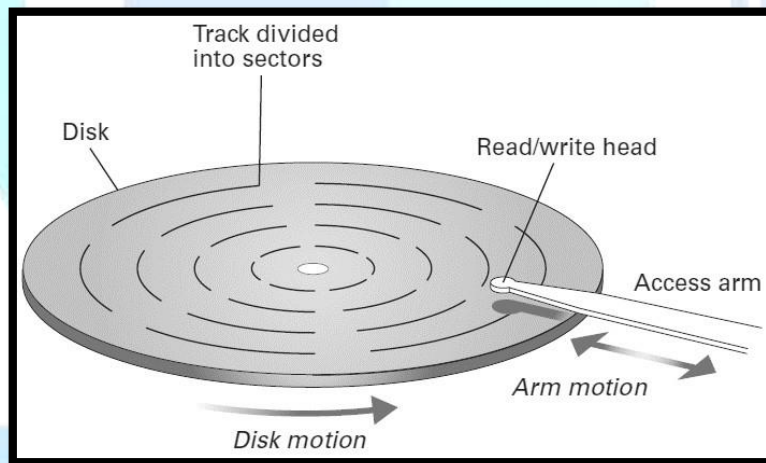


Figure 20: Disk Storage System

15.3. Zoned-bit recording

- ✓ Adjacent tracks form Zones.
- ✓ A typical disk contains 10 zones
- ✓ All tracks within a zone have equal number of sectors.

15.4. Seek Time

Time required to move the read/write heads from one track to another.

15.5. Rotation Delay

Average amount of time required for the desired data to rotate around to the read/write head once the head has been positioned over the desired track.

15.6. Access Time

The sum of seek time and rotation delay

15.7. Transfer rate

The rate at which data can be transferred to or from the disk.



Module 16

16. Optical System

Another class of mass storage system applies optical technology. An example is the **compact disk (CD)**. Digital Versatile Disks (DVDs) and Blu-ray Disks (BDs) are also popular examples of optical systems.

16.1. Compact Disk

Data storage in terms of tracks has been shown in the Figure 21. The characteristics of a CD are:

- ✓ 12 centimeter in diameter
- ✓ Consists of reflective material
- ✓ Information is recorded on them by creating variations in their reflective surfaces.
- ✓ Data is stored on a single-track spiral from inside outside.
- ✓ Each track is divided into sectors,
- ✓ Capacity of a sector is 2KB of data, 1/75 of a second audio music.
- ✓ Retrieval using a Laser that detects the irregularities on the reflective surface of the CD as it spins
- ✓ Initially applied on audio recording using format CD-DA (compact disk-digital audio)
- ✓ Format is being used even today to store computer data
- ✓ Single Track
- ✓ As all sectors are not individually accessible in Optical system having one track, therefore, the data retrieval is faster in magnetic systems than optical systems.
- ✓ Optical System is best suited for long continuous string of data.

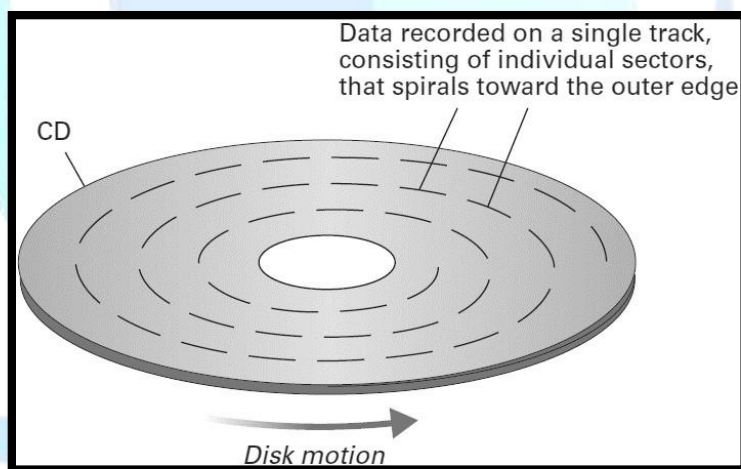


Figure 21: Data storage in CD

16.2. DVD (Digital Versatile Disks)

CDs capacity is 600 to 700 MB; however, DVD has space in GBs as have multiple semi-transparent layers, which can be viewed by precise focused laser. Its shape is like CD.

16.3. BDs (Blue Ray Disks)

BDs use a blue-violet spectrum of light (instead of red), which is able to focus its laser beam with very fine precision. BDs provide five-time capacity than DVDs.

Module 17

17. Flash Drives

17.1. Issues in Magnetic and Optical Systems

- ✓ Physical motion is required

- ✓ Moving Read/Write heads
- ✓ Aiming Laser beams
- ✓ All this takes lot of time in comparison with electronic circuitry

17.2. Flash Drive Technology

- ✓ Bits are stored by sending electronic signals directly to the storage medium where they cause electrons to be trapped in tiny chambers of **silicon dioxide**
- ✓ Chambers are able to store data for years without external power.
- ✓ Repeated erasing slowly damages the silicon dioxide chambers
- ✓ Not suitable in place of Main memory
- ✓ Suitable where alterations can be controlled like cameras, smart phones, portable devices
- ✓ Not reliable as optical disks for long term

17.3. SSDs (Solid State Disks)

- ✓ Larger Flash memory devices designed to take place of magnetic disks
- ✓ Quite operations and low access time
- ✓ However, costly than magnetic systems

17.4. SDs (Secure Digital Memory Cards)

- ✓ Provide up to few GBs of storage
- ✓ Available in smaller, mini, and micro sizes
- ✓ SDHC (High Capacity) can provide 32 GBs
- ✓ SDXC (Extendable Capacity) may exceed to TB.
- ✓ Compact physical size, suitable for car navigation, cameras etc.

ZAINEE
ACADEMY

Module 18

18. Representing Text

This module highlights that how a text is stored with the computer memory.

18.1. Representation as Code

- ✓ Each Textual Symbol is represented with a unique bit pattern
- ✓ Normally 8 bits for each character
- ✓ “Virtual University”
- ✓ 18 characters = 18×8 (144) bits or 18 bytes
- ✓ In 1940’s and 1950’s many such codes were designed
- ✓ American National Standards Institute (ANSI)
- ✓ American Standard Code for Information Interchange (ASCII)

18.2. ASCII Codes

- ✓ 7 bits for information and most significant bit has zero
- ✓ 2^7 combinations = 128
- ✓ Uppercase, lower case, punctuation marks, digits 0 to 9, line feed, carriage returns, and tabs
- ✓ Available at page 577 of your book.

18.3. ASCII code example

Suppose we want to represent “Hello”, by looking at the ASCII table available in your book, the solution is represented in the Figure 22:

01001000	01100101	01101100	01101100	01101111	00101110
H	e	l	l	o	.

Figure 22: ASCII code example

18.4. Limitation of ASCII codes

- ✓ Only 128 characters
- ✓ International Organization for Standardization come-up with many extensions to ASCII
- ✓ One to support western language symbols.

18.5. Limitations of ASCII-extensions

- ✓ 256 are still insufficient to denote all language symbols
- Document having multiple languages could not be read as it should follow a one standard

18.6. Unicode

- ✓ Internationalization of codes by manufacturers of hardware and software
- ✓ Unique patterns of 21 bits
- ✓ Compliance with ASCII
- ✓ Supporting thousands of character sets of Chinese, Hebrew, Japanese

18.7. UTF-8

- ✓ Uses 24 to 32 bits having large possibilities for expansion
- ✓ 2^{24} = 16,777,216 unique symbols
- ✓ File consisting of long symbols encoded with ASCII or Unicode is called text file.

Module 19

19. Representing Numeric Values

This module explains how the numeric values can be represented.

19.1. Issues in storing numeric as Unicode

- ✓ Inefficient suppose you want to store 12, you would need 16 bits to do that
- ✓ 99 could be stored in 16 bits
- ✓ We will learn 16 bits can store 65,535 numeric values

19.2. Binary Notation

Binary notation is a way of representing numeric values using only the digits 0 and 1 rather than the digits 0, 1, 2, 3, 4, 5, 6, 7, 8, and 9. Let's represent numeric values using three bits as shown in the Table 9.

Table 9: Three-bit representation of Numeric values

Numeric Value	Binary
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

If we add another bit and takes 4 bits then you can observe in the Table 10, we can represent 16 values [ranging from 0 to 15]

Table 10: 4 bits representation of Numeric Values

Numeric	Binary	Numeric	Binary
0	0000	8	1000
1	0001	9	1001
2	0010	10	1010
3	0011	11	1011
4	0100	12	1100
5	0101	13	1101
6	0110	14	1110
7	0111	15	1111

If we closely look into the representation, we can represent 2^n numbers if we have n bits. Therefore, if we have 16 bits, we can represent $2^{16} = 65536$ numeric values.

19.3. Binary Notation Variations

Binary notation has two following representations, we will learn them in detail in the upcoming modules.

- ✓ Two's complement for storing whole numbers
- ✓ Floating point notation for fractional numbers

Module 20**20. Representing Images**

In this module, we will learn how images are stored in the memory.

20.1. Pixel

- ✓ Collection of dots – Pixel short for Picture Element.
- ✓ Appearance of each pixel is encoded to form bit map.
- ✓ Many display devices, printers work on Pixel concept.

20.2. Encoding Method: Pixel to Bitmap

- ✓ In black and white images, each pixel is represented as one bit – e.g. 0 for black and 1 for white
- ✓ Often used in Facsimile

20.3. Encoding Method: Handling shades

- ✓ 8 bits are used instead of 1 bit to store the shades of grayness.

20.4. Encoding Method: Colorful Images

- ✓ RGB encoding
- ✓ One byte for Red, one byte for Green, and one-byte Blue – Three bytes to represent one pixel.

20.5. Brightness Chrominance

- ✓ One brightness component and two-color components.
- ✓ Brightness component= Pixel's luminance (sum of red, green, and blue)
- ✓ Blue Chrominance and Red Chrominance
- ✓ Difference between luminance and amount of blue or red.

20.6. Image Scaling

- ✓ Scaling to a larger size needs more pixels
- ✓ Digital Zoom

20.7. Geometric Structures for Image Scaling

- ✓ Collection of lines and curves
- ✓ Using Analytical Geometry
- ✓ Technique: How geometric structures should be displayed rather Pixel reproduction

20.8. Scalable Fonts

- ✓
 - ✓ TrueType by Microsoft and Apple
 - ✓ PostScript by Adobe
- Also popular in Computer Aided Design (CAD)

Module 21

21. Representing Sound

21.1. Sound Amplitude

It is the extent to which air particles are displaced, and this **amplitude of sound** or **sound amplitude** is experienced as the loudness of **sound**.

21.2. How to encode the Sound

- ✓ Sample the amplitude of sound at regular intervals and record the values as shown in the Figure 23.
- ✓ 8000 samples per second for long distance telephone communication

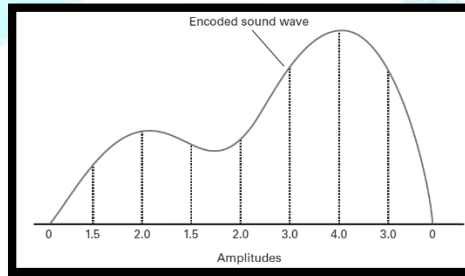


Figure 23: Encoding the amplitude of the sound

21.3. How Sound data communication takes place

- ✓ At one end, it stores amplitude numeric values for each eight thousand of a second
- ✓ Transmitted over the network
- ✓ Received on other end and sound is produced using amplitude.

21.4. Sample Intervals

After how long we should take the sample depends on how accurate and high definition sound recordings are required.

- ✓ 8000 is not enough for high fidelity music recordings
- ✓ 44,100 samples per second are recorded in today's CD's.
- ✓ Data from each sample is recorded in 16 bits (32 bits for Stereo)
- ✓ $32 \times 44100 =$ million bits/sec

21.5. Alternative Method: MIDI

- ✓ Musical Instrument Digital Interface.
- ✓ Used in music synthesizers found in electronic keyboards.
- ✓ Encode directions for producing music rather than storing music itself.
- ✓ 2 seconds sound can be stored in 3 bytes rather than 2 million bits.
- ✓ Encoding the "Sheet music" read by performer rather than the performance itself.
- ✓ MIDI recordings could be significantly different when performed on different synthesizer.

Module 22

22. Binary Notation

Quantity associated with each position is twice the quantity associated with the position to its right.

22.1. Power Method

To understand binary notation first recall decimal numbers we use in our daily life, when we say 375. The representation of 375 in decimal system is available in the Table 11.

Table 11: Decimal System representation

Hundreds	Tens	Units
$10^2 = 100$	$10^1 = 10$	$10^0 = 1$
3	7	5

In each of the position we can use any number from 0 to 9. However, in binary system we can use only 0 or 1 at each position and from right side it represents 2^0 and onwards as illustrated in the Table 12. For example, if we want to represent 32, we will just put 1 in that particular location and rest are 0s.

Table 12: Binary System Representation

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1
0	0	1	0	0	0	0	0

Similarly, if we want to represent 255, we will put 1 in all locations as:

11111111

If you count ($128+64+32+16+8+4+2+1 = 255$).

22.2. Algorithm for Finding Binary Representation of a Positive Decimal Number

Remember, algorithm was a set of steps to perform a certain task. To convert a positive decimal number, the set of steps are shown in the Figure 24.



- Step 1. Divide the value by two and record the remainder.
- Step 2. As long as the quotient obtained is not zero, continue to divide the newest quotient by two and record the remainder.
- Step 3. Now that a quotient of zero has been obtained, the binary representation of the original value consists of the remainders listed from right to left in the order they were recorded.

ER

22-06-24 09:09:29

These steps keep in the mind

Figure 24: Algorithm for converting positive decimal to binary number

Let's apply this algorithm, suppose we have the number 13 and wants to convert it to the equivalent binary, the working of this algorithm is shown in the Figure 25.

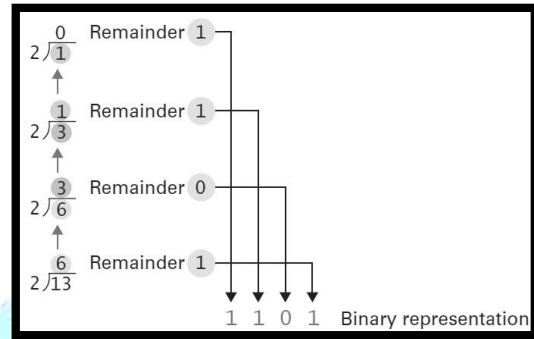


Figure 25: Algorithm working for converting positive decimal to binary

ZAINEE
ACADEMY

Module 23

23. Binary Addition

In this module, you will learn the binary addition.

As we are dealing with two bits while adding them, there could be 4 possibilities, firstly whether both bits are zero, secondly first bit is one and second is zero, thirdly first bit is zero and second bit is one, fourthly both are one. All four cases and their sum is illustrated in the Figure 26.

0	1	0	1
+0	+0	+1	+1
0	1	1	10

Figure 26: Binary Addition basics

We will apply these rules from right to left as we do the addition of decimal numbers, when we come across 10, we will place 0 and takes 1 as a carry. Different examples have been shown in the Figures 27 to 29.

2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	
128	64	32	16	8	4	2	1	
0	0	1	1	1	0	1	0	58
0	0	0	1	1	0	1	1	27
0	1	0	1	0	1	0	1	85

Figure 27: Example 1 of Binary Addition

2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	
128	64	32	16	8	4	2	1	
0	0	0	0	1	0	1	1	11
0	0	0	0	1	1	1	0	14
0	0	0	1	1	0	0	1	25

Figure 28: Example 2 of Binary Addition

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
128	64	32	16	8	4	2	1	
0	1	0	0	0	0	0	0	64
0	1	0	0	0	0	0	0	64
1	0	0	0	0	0	0	0	128

Figure 29: Example 3 of Binary Addition



Module 24

24. Fraction in Binary

24.1. Radix Point

- ✓ As we have decimal point
- ✓ Digits on left side represent the whole number
- ✓ Digits on right side represent the fractional part

24.2. Example of Radix Point

2^2	2^1	2^0	radix	2^{-1}	2^{-2}	2^{-3}
4	2	1	.	$1/2$	$1/4$	$1/8$
1	0	1		1	0	1

The example has been shown in the Figure 30.

Figure 30: Radix point example

How to read the data represented in the Figure 30, consider the Figure 31.

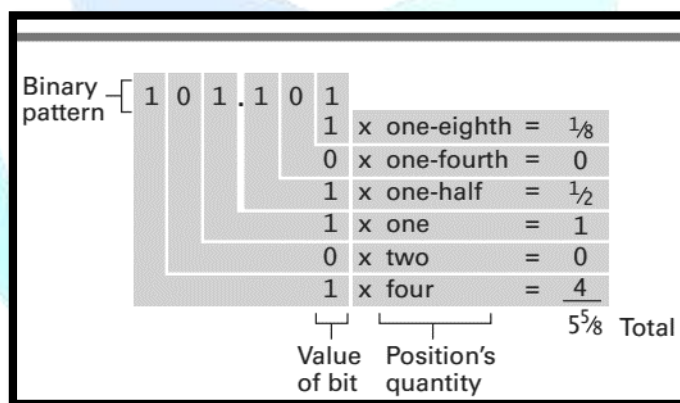


Figure 31: Understanding Fraction represented in Binary

24.3. Addition in fraction

It is same as the addition in decimal system, you need to align the radix point and perform the addition as we did in simple binary or decimal system illustrated in the previous modules. For example, 10.011 added to 100.11 produces 111.001, as shown in the Figure 32.

$$\begin{array}{r}
 10.011 \\
 + 100.110 \\
 \hline
 111.001
 \end{array}$$

Figure 32: Binary addition

Module 25

25. 2's Complement Notation to Store Numbers

The most popular system for representing integers within today's computers is **two's complement** notation.

25.1. Integer Representation in 2's Complement

- ✓ Fixed number of bits to represent each value
- ✓ Normally 32 bits are used
- ✓ We will discuss smaller examples for demonstration purpose.
- ✓ For +ve integers, starting from zero going upward until a single zero is reached followed by all 1's
- ✓ For -ve integers, starting from all 1's going downwards until a single 1 is reached followed by all 0's.
- ✓ Left most bit = sign bit.

Integers are represented in the pattern of three lengths in the Figure 33 and patterns of four lengths have been represented in the Figure 34.

a. Using patterns of length three

Bit pattern	Value represented
011	3
010	2
001	1
000	0
111	-1
110	-2
101	-3
100	-4

Figure 33: 2's complement notation for three bits patterns

b. Using patterns of length four

Bit pattern	Value represented
0111	7
0110	6
0101	5
0100	4
0011	3
0010	2
0001	1
0000	0
1111	-1
1110	-2
1101	-3
1100	-4
1011	-5
1010	-6
1001	-7
1000	-8

Figure 34: 2's complement notation for four bits patterns

25.2. Conversion between positive and negative representations

One trick to convert between positive and negative numbers represented in 2's complement is as follows:

1. Start from right most bit.
2. Start copying until first 1 arrives
3. After first 1 is found, complement all numbers i.e 0 to 1 and 1 to 0

For example, you know the binary of 7 in 2's complement notation which is 0111, now if you want to find the binary form of -7, apply the above rule and you will get 1001.

25.3. Addition in 2's complement notation

The beauty of 2's complement notation is that it works perfectly fine by using the same method of addition as we studied earlier, for more understanding carefully look at the Figure 35.

Problem in base 10	Problem in two's complement	Answer in base 10
$\begin{array}{r} 3 \\ + 2 \\ \hline \end{array}$	$\begin{array}{r} 0011 \\ + 0010 \\ \hline 0101 \end{array}$	→ 5
$\begin{array}{r} -3 \\ + -2 \\ \hline \end{array}$	$\begin{array}{r} 1101 \\ + 1110 \\ \hline 1011 \end{array}$	→ -5
$\begin{array}{r} 7 \\ + -5 \\ \hline \end{array}$	$\begin{array}{r} 0111 \\ + 1011 \\ \hline 0010 \end{array}$	→ 2

Figure 35: Addition in 2's complement notation

25.4. Problem of Overflow

- ✓ Using 4 bits, maximum positive number 7 can be represented and -8 on the negative side.
- ✓ $5+4 = 9$ which cannot be stored in four bits
- ✓ Result would be as -7.
- ✓ Can be deducted using sign bit.
- ✓ Today's computer have longer bit pattern normally 32 bits
- ✓ Maximum positive value = 2,147,483,647
- ✓ If still overflow occurs, we can use even more bits, or changing the units like calculating answer in Kilometer than meter.
- ✓ Previously 16 bits were used to store 2's complement notation, maximum number represented was 32,768
- ✓ On "September 19, 1989", Hospital system malfunctioned as 32, 768 days had been passed after "January 1, 1900", and thus it produced negative value.

ZAINEE
ACADEMY

Module 26

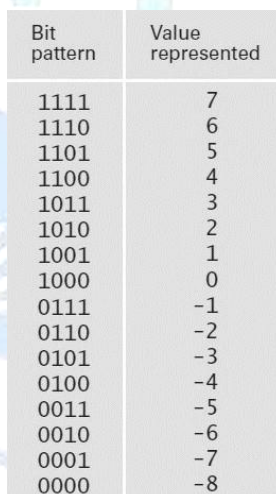
26. Excess Notation

This is another method of representing integer values.

26.1. Integer Representations in Excess Notation

- ✓ Fixed number of bits to represent each value
- ✓ Write down all bit patterns of the same length
- ✓ First bit pattern having 1 in the most significant bit is used to represent Zero
- ✓ Following values used to represent positive numbers and Preceding values to represent negative numbers.
- ✓ Each value in Excess notation is Excess of its original value in Binary
- ✓ 1011 represent 11 in Binary but here it is representing 3 (excess of 8).
- ✓ Excess 16 to represent 10000 as Zero
- Excess 4 to represent 100 as Zero

Excess 8 notation is shown in the Figure 36.

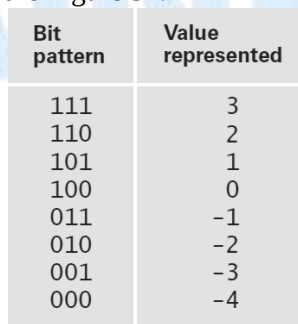


Bit pattern	Value represented
1111	7
1110	6
1101	5
1100	4
1011	3
1010	2
1001	1
1000	0
0111	-1
0110	-2
0101	-3
0100	-4
0011	-5
0010	-6
0001	-7
0000	-8

Figure 36: Excess eight notation

26.2. Excess Four Notation

- ✓ Excess 4 to represent 100 as Zero
- ✓ Values are Excess of 4 as shown in the Figure 37.



Bit pattern	Value represented
111	3
110	2
101	1
100	0
011	-1
010	-2
001	-3
000	-4

Figure 37: Excess Four notation

26.3. Excess Four Notation Comparisons

Let's see the differences of bits patterns represented in Binary, Excess 4, and 2's complement. Such comparison is shown in the Figure 38.

Binary	Decimal	Excess 4	2's Complement
111	7	3	-1
110	6	2	-2
101	5	1	-3
100	4	0	-4
011	3	-1	3
010	2	-2	2
001	1	-3	1
000	0	-4	0

Figure 38: Excess four notation comparisons



Module 27

27. Floating Point Notation

In contrast to the storage of integers, the storage of a value with a fractional part requires that we store not only the pattern of 0s and 1s representing its binary representation but also the position of the radix point. A popular way of doing this is based on scientific notation and is called floating-point notation.

27.1. Storing Radix

- ✓ Numbers with fractional part have a radix, so its important to store the position of Radix
- ✓ One popular way is floating-point notation
- ✓ For demonstration purpose we will use examples of 8 bits storage system.

27.2. Storing Fractions

We first designate the high-order bit of the byte as the sign bit. Once again, at the value stored is nonnegative, and a 1 will mean that the value is negative. Next, we divide the remaining 7 bits of the byte into two groups, or fields: the **exponent field** and the **mantissa field**. Let us designate the 3 bits following the sign bit as the exponent field and the remaining 4 bits as the mantissa field. Figure 39 illustrates how the byte is divided. We can explain the meaning of the fields by considering the following example. Suppose a byte consists of the bit pattern 01101011. Analyzing this pattern with the preceding format, we see that the sign bit is 0, the exponent is 110, and the mantissa is 1011. To decode the byte, we first extract the mantissa and place a radix point on its left side, obtaining

.1011

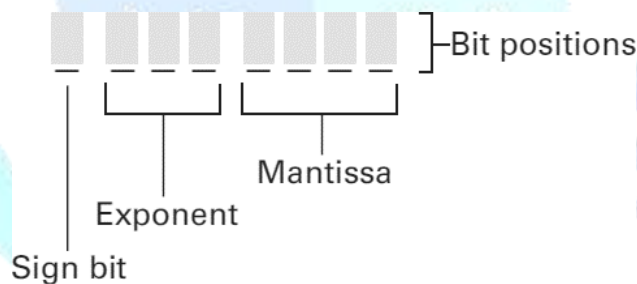


Figure 39: Floating-point notation components

Next, we extract the contents of the exponent field (110) and interpret it as an integer stored using the 3-bit excess method (see again Figure 39). Thus the pattern in the exponent field in our example represents a positive 2. This tells us to move the radix in our solution to the right by 2 bits. (A negative exponent would mean to move the radix to the left.) Consequently, we obtain:

10.11

Which is the binary representation for $2\frac{3}{4}$. (Recall the representation of binary fractions from Figure 1.18.) Next, we note that the sign bit in our example is 0; the value represented is thus nonnegative. We conclude that the byte 01101011 represents $2\frac{3}{4}$. Had the pattern been 11101011 (which is the same as before except for the sign bit), the value represented would have been $-2\frac{3}{4}$. As another example, consider the byte 00111100. We extract the mantissa to obtain

.1100

and move the radix 1 bit to the left, since the exponent field (011) represents the value -1. We therefore have

.01100

Which represents $3/8$. Since the sign bit in the original pattern is 0, the value stored is nonnegative. We conclude that the pattern 00111100 represents $3/8$. To store a value using floating-point notation, we reverse the preceding process. For example, to encode $11/8$, first we express it in binary notation and obtain 1.001. Next, we copy the bit pattern into the mantissa field from left to right, starting with the leftmost 1 in the binary representation. At this point, the byte looks like this:

-	-	-	-	1	0	0	1
---	---	---	---	---	---	---	---

We must now fill in the exponent field. To this end, we imagine the contents of the mantissa field with a radix point at its left and determine the number of bits and the direction the radix must be moved to obtain the original binary number. In our example, we see that the radix in .1001 must be moved 1 bit to the right to obtain 1.001. The exponent should therefore be a positive one, so we place 101 (which is positive one in excess four notation as shown in Figure 1.23) in the exponent field. Finally, we fill the sign bit with 0 because the value being stored is nonnegative. The finished byte looks like this:

0 1 0 1 1 0 0 1

There is a subtle point you may have missed when filling in the mantissa field. The rule is to copy the bit pattern appearing in the binary representation from left to right, starting with the leftmost 1. To clarify, consider the process of storing the value $3/8$, which is .011 in binary notation. In this case the mantissa will be

-	-	-	-	1	1	0	0
---	---	---	---	---	---	---	---

It will not be

-	-	-	-	0	1	1	0
---	---	---	---	---	---	---	---

This is because we fill in the mantissa field starting with the leftmost 1 that appears in the binary representation. Representations that conform to this rule are said to be in normalized form.

Using normalized form eliminates the possibility of multiple representations for the same value. For example, both 00111100 and 01000110 would decode to the value $3/8$, but only the first pattern is in normalized form. Complying with normalized form also means that the representation for all nonzero values will

have a mantissa that starts with 1. The value zero, however, is a special case; its floating-point representation is a bit pattern of all 0s.

Module 28

28. Truncation Errors in Floating Point Notation

Let us consider the annoying problem that occurs if we try to store the value 2^{58} with our one-byte floating-point system. We first write 2^{58} in binary, which gives us 10.101. But when we copy this into the mantissa field, we run out of room, and the rightmost 1 (which represents the last $1/8$) is lost (Figure 40). If we ignore this

problem for now and continue by filling in the exponent field and the sign bit, we end up with the bit pattern 01101010, which represents $2^{1/2}$ instead of 2^{58} . What has occurred is called a **truncation error**, or **round-off error**—meaning that part of the value being stored is lost because the mantissa field is not large enough.

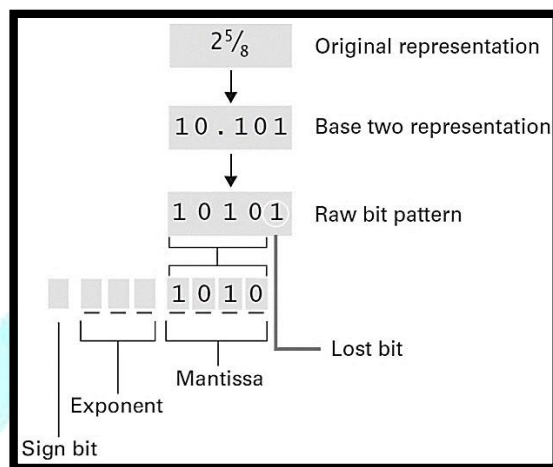


Figure 40: Encoding the value $2^5/8$

28.1. Primitive Methods for Handling Truncation Errors

The significance of such errors can be reduced by using a longer mantissa field. In fact, most computers manufactured today use at least 32 bits for storing values in floating-point notation instead of the 8 bits we have used here. This also allows for a longer exponent field at the same time. Even with these longer formats, however, there are still times when more accuracy is required. Another source of truncation errors is a phenomenon that you are already accustomed to in base 10 notation: the problem of nonterminating expansions, such as those found when trying to express $1/3$ in decimal form. Some values cannot be accurately expressed regardless of how many digits we use. The difference between our traditional base 10 notation and binary notation is that more values have nonterminating representations in binary than in decimal notation. For example, the value $1/10$ is nonterminating when expressed in binary. Imagine the problems this might cause the unwary person using floating-point notation to store and manipulate dollars and cents. In particular, if the dollar is used as the unit of measure, the value of a dime could not be stored accurately. A solution in this case is to manipulate the data in units of pennies so that all values are integers that can be accurately stored using a method such as two's complement.

28.2. Intelligent Processing for Handling Truncation Errors

Let's suppose we want to store

$+1/8 + 1/8$

- ✓ If we add $2^{1/2}$ to $1/8$ we end up with $2^{5/8}$ which is 10.101 which cannot be stored in 4 bit mantissa.
- ✓ The result 10.10 would be $2^{1/2}$ this means the $1/8$ is truncated

In this situation, the following can be done:

- ✓ Let's add first $1/8$ to $1/8$ which is $1/4$ or $.01$, can be stored and result would be 00111000
- ✓ Now add this to $2^{1/2}$ now we got $2^{3/4} = 01101011$ which is accurate.
- ✓ Order is important, adding small quantities together first might give significant quantity to be added to large quantity.

Module 29

29. Data Compression: Generic Techniques

Data compression schemes fall into two categories. Some are **lossless**, others are **lossy**. Lossless schemes are those that do not lose information in the compression process. Lossy schemes are those that may lead to the loss of information. Lossy techniques often provide more compression than lossless ones and are therefore popular in settings in which minor errors can be tolerated, as in the case of images and audio.

In cases where the data being compressed consist of long sequences of the same value, the compression technique called **run-length encoding**, which is a lossless method, is popular. It is the process of replacing sequences of identical data elements with a code indicating the element that is repeated and the number of times it occurs in the sequence. For example, less space is required to indicate that a bit pattern consists of 253 ones, followed by 118 zeros, followed by 87 ones than to actually list all 458 bits.

Another lossless data compression technique is **frequency-dependent encoding** a system in which the length of the bit pattern used to represent a data item is inversely related to the frequency of the item's use. Such codes are examples of variable-length codes, meaning that items are represented by patterns of different lengths. David Huffman is credited with discovering an algorithm that is commonly used for developing frequency-dependent codes, and it is common practice to refer to codes developed in this manner as **Huffman codes**. In turn, most frequency-dependent codes in use today are Huffman codes.

As an example of frequency-dependent encoding, consider the task of encoding English language text. In the English language the letters **e, t, a, and i** are used more frequently than the letters **z, q, and x**. So, when constructing a code for text in the English language, space can be saved by using short bit patterns to represent the former letters and longer bit patterns to represent the latter ones. The result would be a code in which English text would have shorter representations than would be obtained with uniform-length codes.

In some cases, the stream of data to be compressed consists of units, each of which differs only slightly from the preceding one. An example would be consecutive frames of a motion picture. In these cases, techniques using **relative encoding**, also known as **differential encoding**, are helpful. These techniques record the differences between consecutive data units rather than entire units; that is, each unit is encoded in terms of its relationship to the previous unit. **Relative encoding** can be implemented in **either lossless or lossy** form depending on whether the differences between consecutive data units are encoded precisely or approximated.

Still other popular compression systems are based on **dictionary encoding** techniques. Here the term **dictionary** refers to a collection of building blocks from which the message being compressed is constructed, and the message itself is encoded as a sequence of references to the dictionary. We normally think of dictionary encoding systems as lossless systems, but as we will see in our discussion of image compression, there are times when the entries in the dictionary are only approximations of the correct data elements, resulting in a **lossy compression system**.

Dictionary encoding can be used by word processors to compress text documents because the dictionaries already contained in these processors for the purpose of spell checking make excellent compression dictionaries. In particular, an entire word can be encoded as a single reference to this dictionary rather than as a sequence of individual characters encoded using a system such as UTF-8. A typical dictionary in a word processor contains approximately **25,000** entries, which means an individual entry can be identified by an integer in the range of 0 to 24,999. This means that a particular entry in the **dictionary can be identified by a pattern of only 15 bits**. In contrast, if the word being referenced consisted of six letters, its character-by-character encoding would require 48 bits using UTF-8.

A variation of dictionary encoding is **adaptive dictionary encoding** (also known as dynamic dictionary encoding). In an adaptive dictionary encoding system, the dictionary is allowed to change during the encoding process. A popular example is **Lempel-Ziv-Welsh (LZW) encoding** (named after its creators, Abraham Lempel, Jacob Ziv, and Terry Welsh). To encode a message using LZW, one starts with a dictionary containing the basic building blocks from which the message is constructed, but as larger units are found in the message, they are added to the dictionary –meaning that future occurrences of those units can be encoded as single, rather than multiple, dictionary references. For example, when encoding English text, one could start with a dictionary containing individual characters, digits, and punctuation marks. But as words in the message are identified, they could be added to the dictionary. Thus, the dictionary would grow as the message is encoded, and as the dictionary grows, more words (or recurring patterns of words) in the message could be encoded as single references to the dictionary.

The result would be a message encoded in terms of a rather large dictionary that is unique to that particular message. But this large dictionary would not have to be present to decode the message. Only the original small dictionary would be needed. Indeed, the decoding process could begin with the same small dictionary with which the encoding process started. Then, as the decoding process continues, it would encounter the same units found during the encoding process, and thus be able to add them to the dictionary for future reference just as in the encoding process.

To clarify, consider applying LZW encoding to the message

xyx xyx xyx xyx

starting with a dictionary with three entries, the first being x, the second being y, and the third being a space. We would begin by encoding xyx as 121, meaning that the message starts with the pattern consisting of the first dictionary entry, followed by the second, followed by the first. Then the space is encoded to produce 1213. But, having reached a space, we know that the preceding string of characters forms a word, and so we add the pattern xyx to the dictionary as the fourth entry. Continuing in this manner, the entire message would be encoded as 121343434.

If we were now asked to decode this message, starting with the original three entry dictionary, we would begin by decoding the initial string 1213 as xyx followed by a space. At this point we would recognize that the string xyx forms a word and add it to the dictionary as the fourth entry, just as we did during the encoding process. We would then continue decoding the message by recognizing that the 4 in the message refers to this new fourth entry and decode it as the word xyx, producing the pattern

xyx xyx

Continuing in this manner we would ultimately decode the string 121343434 as

xyx xyx xyx xyx

Which is the original message.

Module 30

30. Data Compression: Compressing Images

Numerous compression schemes have been developed specifically for image representations. One system known as **GIF** (short for **Graphic Interchange Format** and pronounced “Giff” by some and “Jiff” by others) is a dictionary encoding system that was developed by **CompuServe**. It approaches the compression problem by reducing the number of colors that can be assigned to a pixel to only **256**. The red-green-blue combination for each of these colors is encoded using three bytes, and these 256 encodings are stored in a table (a dictionary) called the palette. Each pixel in an image can then be represented by a single byte whose value indicates which of the 256 palette entries represents the pixel’s color. (Recall that a single byte can contain any one of 256 different bit patterns.) Note that **GIF is a lossy compression system** when applied to arbitrary images because the colors in the palette may not be identical to the colors in the original image.

GIF can obtain additional compression by extending this simple dictionary system to an adaptive dictionary system using **LZW** techniques. In particular, as patterns of pixels are encountered during the encoding process, they are added to the dictionary so that future occurrences of these patterns can be encoded more efficiently. Thus, the final dictionary consists of the original palette and a collection of pixel patterns.

One of the colors in a GIF palette is normally assigned the value “transparent,” which means that the background is allowed to show through each region assigned that “color.” This option, combined with the relative simplicity of the GIF system, makes GIF a logical choice in simple animation applications in which multiple images must move around on a computer screen. On the other hand, its ability to encode only 256 colors renders it unsuitable for applications in which higher precision is required, as in the field of photography.

Another popular compression system for images is **JPEG** (pronounced “JAY-peg”). It is a standard developed by the **Joint Photographic Experts Group** (hence the standard’s name) within ISO. JPEG has proved to be an effective standard for compressing color photographs and is widely used in the photography industry, as witnessed by the fact that most digital cameras use JPEG as their default compression technique.

The JPEG standard actually encompasses several methods of image compression, each with its own goals. In those situations that require the utmost in precision, **JPEG provides a lossless mode**. However, JPEG’s lossless mode does not produce high levels of compression when compared to other JPEG options. Moreover, other JPEG options have proven very successful, meaning that JPEG’s lossless mode is rarely used. Instead, the option known as JPEG’s baseline standard (also known as JPEG’s lossy sequential mode) has become the standard of choice in many applications.

Image compression using the JPEG baseline standard requires a sequence of steps, some of which are designed to take advantage of a human eye’s limitations. In particular, the human eye is more sensitive to changes in brightness than to changes in color. So, starting from an image that is encoded in terms of luminance and chrominance components, the first step is to average the chrominance values over two-by-two-pixel squares. This reduces the size of the chrominance information by a factor of four while preserving all the original brightness information. The result is a significant degree of compression without a noticeable loss of image quality.

The next step is to divide the image into eight-by-eight-pixel blocks and to compress the information in each block as a unit. This is done by applying a mathematical technique known as the discrete cosine transform, whose details need not concern us here. The important point is that this transformation converts the original eight-by-eight block into another block whose entries reflect how the pixels in the original block relate to each other rather than the actual pixel values. Within this new block, values below a predetermined threshold are then replaced by zeros, reflecting the fact that the changes represented by these values are too subtle to be detected by the human eye. For example, if the original block contained a checkerboard pattern, the new block might reflect a uniform average color. (A typical eight-by-eight-pixel

block would represent a very small square within the image so the human eye would not identify the checkerboard appearance anyway.)

At this point, more traditional run-length encoding, relative encoding, and variable-length encoding techniques are applied to obtain additional compression. Altogether, JPEG's baseline standard normally compresses color images by a factor of at least 10, and often by as much as 30, without noticeable loss of quality.

Still another data compression system associated with images is **TIFF** (short for **Tagged Image File Format**). However, the most popular use of TIFF is not as a means of data compression but instead as a standardized format for storing photographs along with related information such as date, time, and camera settings. In this context, the image itself is normally stored as red, green, and blue pixel components without compression.

The TIFF collection of standards does include data compression techniques, most of which are designed for compressing images of text documents in facsimile applications. These use variations of run-length encoding to take advantage of the fact that text documents consist of long strings of white pixels. The color image compression option included in the TIFF standards is based on techniques similar to those used by GIF and are therefore not widely used in the photography community.



Module 31**31. Data Compression: Compressing Audio and Videos**

The most commonly used standards for encoding and compressing audio and video were developed by the **Motion Picture Experts Group (MPEG)** under the leadership of ISO. In turn, these standards themselves are called MPEG.

MPEG encompasses a variety of standards for different applications. For example, the demands for high definition television (HDTV) broadcast are distinct from those for video conferencing, in which the broadcast signal must find its way over a variety of communication paths that may have limited capabilities. Both of these applications differ from that of storing video in such a manner that sections can be replayed or skipped over.

The techniques employed by MPEG are well beyond the scope of this text, but in general, videocompression techniques are based on video being constructed as a sequence of pictures in much the same way that motion pictures are recorded on film. To compress such sequences, only some of the pictures, called I-frames, are encoded in their entirety. The pictures between the I-frames are encoded using relative encoding techniques. That is, rather than encode the entire picture, only its distinctions from the prior image are recorded. The I-frames themselves are usually compressed with techniques similar to JPEG.

The best-known system for compressing audio is **MP3**, which was developed within the MPEG standards. In fact, the acronym MP3 is short for MPEG layer 3. Among other compression techniques, MP3 takes advantage of the properties of the human ear, removing those details that the human ear cannot perceive. One such property, called **temporal masking**, is that for a short period after a loud sound, the human ear cannot detect softer sounds that would otherwise be audible. Another, called **frequency masking**, is that a sound at one frequency tends to mask softer sounds at nearby frequencies. By taking advantage of such characteristics, MP3 can be used to obtain significant compression of audio while maintaining near CD quality sound.

Using MPEG and MP3 compression techniques, video cameras are able to record as much as an hour's worth of video within 128MB of storage, and portable music players can store as many as 400 popular songs in a single GB. But, in contrast to the goals of compression in other settings, the goal of compressing audio and video is not necessarily to save storage space. Just as important is the goal of obtaining encodings that allow information to be transmitted over today's communication systems fast enough to provide timely presentation. If each video frame required a MB of storage and the frames had to be transmitted over a communication path that could relay only one KB per second, there would be no hope of successful video conferencing. Thus, in addition to the quality of reproduction allowed, audio and video compression systems are often judged by the transmission speeds required for timely data communication. These speeds are normally measured in **bits per second (bps)**. Common units include **Kbps** (kilo-bps, equal to one thousand bps), **Mbps** (mega-bps, equal to one million bps), and **Gbps** (giga-bps, equal to one billion bps). Using MPEG techniques, video presentations can be successfully relayed over communication paths that provide transfer rates of 40 Mbps. MP3 recordings generally require transfer rates of no more than 64 Kbps.

Module 32

32. Data Manipulation: CPU Basic

A CPU consists of three parts (Figure 41): the **arithmetic/logic unit**, which contains the circuitry that performs operations on data (such as addition and subtraction); the **control unit**, which contains the circuitry for coordinating the machine's activities; and the **register unit**, which contains data storage cells (similar to main memory cells), called **registers**, that are used for temporary storage of information within the CPU.

Some of the registers within the register unit are considered **general-purpose registers**, whereas others are **special-purpose registers**. For now, we are concerned only with the general purpose registers.

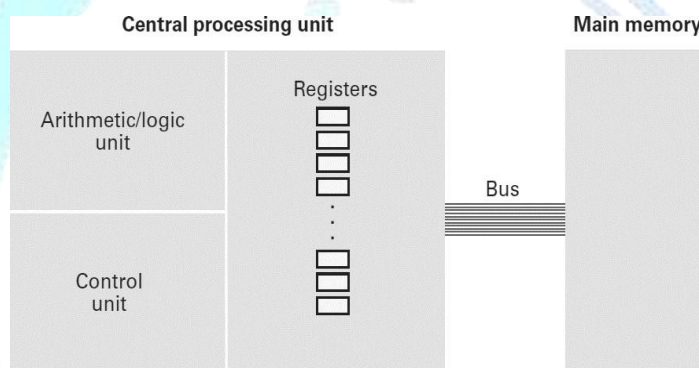


Figure 41: CPU and main memory connected via a bus

General-purpose registers serve as temporary holding places for data being manipulated by the CPU. These registers hold the inputs to the arithmetic/logic unit's circuitry and provide storage space for results produced by that unit. To perform an operation on data stored in main memory, the control unit transfers the data from memory into the general purpose registers, informs the arithmetic/logic unit which registers hold the data, activates the appropriate circuitry within the arithmetic/logic unit, and tells the arithmetic/logic unit which register should receive the result.

For the purpose of transferring bit patterns, a machine's CPU and main memory are connected by a collection of wires called a **bus** (see again Figure 41). Through this bus, the CPU extracts (reads) data from main memory by supplying the address of the pertinent memory cell along with an electronic signal telling the memory circuitry that it is supposed to retrieve the data in the indicated cell. In a similar manner, the CPU places (writes) data in memory by providing the address of the destination cell and the data to be stored together with the appropriate electronic signal telling main memory that it is supposed to store the data being sent to it.

Based on this design, the task of adding two values stored in main memory involves more than the mere execution of the addition operation. The data must be transferred from main memory to registers within the CPU, the values must be added with the result being placed in a register, and the result must then be stored in a memory cell. The entire process is summarized by the five steps listed in Figure 42.

Adding values stored in memory

- Step 1. Get one of the values to be added from memory and place it in a register.
- Step 2. Get the other value to be added from memory and place it in another register.
- Step 3. Activate the addition circuitry with the registers used in Steps 1 and 2 as inputs and another register designated to hold the result.
- Step 4. Store the result in memory.
- Step 5. Stop.

Figure 42: Adding values stored in memory

ZAINEE
ACADEMY

Module 33**33. Data Manipulation: Stored Program**

Early computers were not known for their flexibility—the steps that each device executed were built into the control unit as a part of the machine. To gain more flexibility, some of the early electronic computers were designed so that the CPU could be conveniently rewired. This flexibility was accomplished by means of a pegboard arrangement similar to old telephone switchboards in which the ends of jumper wires were plugged into holes.

A breakthrough (credited, apparently incorrectly, to John von Neumann) came with the realization that a program, just like data, can be encoded and stored in main memory. If the control unit is designed to extract the program from memory, decode the instructions, and execute them, the program that the machine follows can be changed merely by changing the contents of the computer's memory instead of rewiring the CPU.

The idea of storing a computer's program in its main memory is called the **stored-program concept** and has become the standard approach used today—so standard, in fact, that it seems obvious. What made it difficult originally was that everyone thought of programs and data as different entities: Data were stored in memory; programs were part of the CPU. The result was a prime example of not seeing the forest for the trees. It is easy to be caught in such ruts, and the development of computer science might still be in many of them today without our knowing it. Indeed, part of the excitement of the science is that new insights are constantly opening doors to new theories and applications.

ZAINEE
ACADEMY

Module 34

34. Data Manipulation: CPU Architecture Philosophies

To apply the stored program concept, CPUs are designed to recognize instructions encoded as bit patterns. This collection of instructions along with the encoding system is called the machine language. An instruction expressed in this language is called a machine level instruction or, more commonly, a machine instruction.

The list of machine instructions that a typical CPU must be able to decode and execute is quite short. In fact, once a machine can perform certain elementary but well-chosen tasks, adding more features does not increase the machine's theoretical capabilities. In other words, beyond a certain point, additional features may increase such things as convenience but add nothing to the machine's fundamental capabilities.

The degree to which machine designs should take advantage of this fact has led to two philosophies of CPU architecture. One is that a CPU should be designed to execute a minimal set of machine instructions. This approach leads to what is called a reduced instruction set computer (RISC). The argument in favor of RISC architecture is that such a machine is efficient, fast, and less expensive to manufacture. On the other hand, others argue in favor of CPUs with the ability to execute a large number of complex instructions, even though many of them are technically redundant. The result of this approach is known as a complex instruction set computer (CISC). The argument in favor of CISC architecture is that the more complex CPU can better cope with the ever-increasing complexities of today's software. With CISC, programs can exploit a powerful rich set of instructions, many of which would require a multi-instruction sequence in a RISC design. In the 1990s and into the millennium, commercially available CISC and RISC processors were actively competing for dominance in desktop computing. Intel processors, used in PCs, are examples of CISC architecture; PowerPC processors (developed by an alliance between Apple, IBM, and Motorola) are examples of RISC architecture and were used in the Apple Macintosh. As time progressed, the manufacturing cost of CISC was drastically reduced; thus Intel's processors (or their equivalent from AMD—Advanced Micro Devices, Inc.) are now found in virtually all desktop and laptop computers (even Apple is now building computers based on Intel products).

While CISC secured its place in desktop computers, it has an insatiable thirst for electrical power. In contrast, the company Advanced RISC Machine (ARM) has designed a RISC architecture specifically for low power consumption. (Advanced RISC Machine was originally Acorn Computers and is now ARM Holdings.) Thus, ARM based processors, manufactured by a host of vendors including Qualcomm and Texas Instruments, are readily found in game controllers, digital TVs, navigation systems, automotive modules, cellular telephones, smartphones, and other consumer electronics.

Module 35

35. Data Manipulation: Machine Instruction Categories

Regardless of the choice between RISC and CISC, a machine's instructions can be categorized into three groupings: (1) the data transfer group, (2) the arithmetic/logic group, and (3) the control group.

35.1. Data Transfer Group

The data transfer group consists of instructions that request the movement of data from one location to another. Steps 1, 2, and 4 in Figure 42 fall into this category. We should note that using terms such as transfer or move to identify this group of instructions is actually a misnomer. It is rare that the data being transferred is erased from its original location. The process involved in a transfer instruction is more like copying the data rather than moving it. Thus, terms such as copy, or clone better describe the actions of this group of instructions. While on the subject of terminology, we should mention that special terms are used when referring to the transfer of data between the CPU and main memory. A request to fill a general-purpose register with the contents of a memory cell is commonly referred to as a LOAD instruction; conversely, a request to transfer the contents of a register to a memory cell is called a STORE instruction. In Figure 2.2, Steps 1 and 2 are LOAD instructions, and Step 4 is a STORE instruction.

An important group of instructions within the data transfer category consists of the commands for communicating with devices outside the CPU main memory context (printers, keyboards, display screens, disk drives, etc.). Since these instructions handle the input/output (I/O) activities of the machine, they are called **I/O instructions** and are sometimes considered as a category in their own right.

35.2. Arithmetic/Logic Group

The arithmetic/logic group consists of the instructions that tell the control unit to request an activity within the arithmetic/logic unit. Step 3 in Figure 42 falls into this group. As its name suggests, the arithmetic/logic unit is capable of performing operations other than the basic arithmetic operations.

35.3. Control Group

The control group consists of those instructions that direct the execution of the program rather than the manipulation of data. Step 5 in Figure 42 falls into this category,

ZAINEE
ACADEMY

Module 36

36. Data Manipulation: Program Execution

- ✓ Machine Instruction is fetched from main memory to CPU as illustrated in the Figure 43.
- ✓ Each instruction is decoded and obeyed.
- ✓ The order is as the instructions are stored in memory otherwise specified by a JUMP.

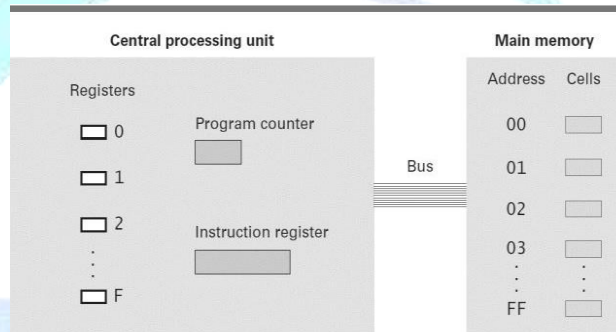


Figure 43: CPU and Main memory linkage using Bus

Special Purpose Registers

- ✓ **Instruction Register:** Holding instruction being executed now.

Program Counter: contains the address of next instruction to be executed.

Machine cycle is shown in the Figure 44.

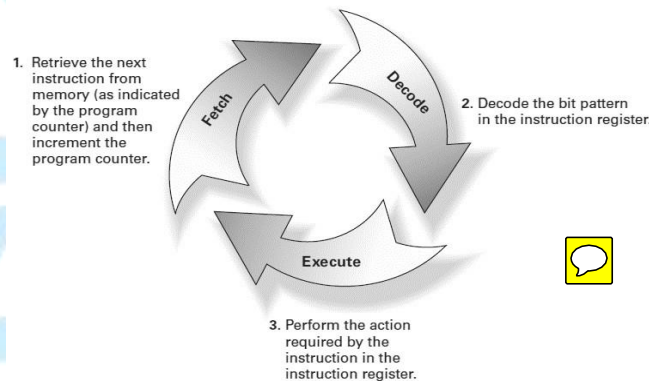


Figure 44: Machine Cycle



USER

2022-06-23 15:49:44

3 steps keep in the mind
which step is not is the
mcqss

Module 37**37. Data Manipulation: Program Execution Example**

In your book an example of machine instructions with their meanings are available in Appendix C at page 581. let's discuss it.

Let's execute a program that reads two numbers from memory, adds them and store in the memory. The instructions for such an activity can be seen in the Figure 45.

Encoded instructions	Translation
6C	Load register 5 with the bit pattern found in the memory cell at address 6C.
166D	Load register 6 with the bit pattern found in the memory cell at address 6D.
5056	Add the contents of register 5 and 6 as though they were two's complement representation and leave the result in register 0.
306E	Store the contents of register 0 in the memory cell at address 6E.
	Halt.

Figure 45: Machine Instructions for adding two numbers

When this instruction will be loaded in the memory. The snapshot of the main memory is shown in the Figure 46.

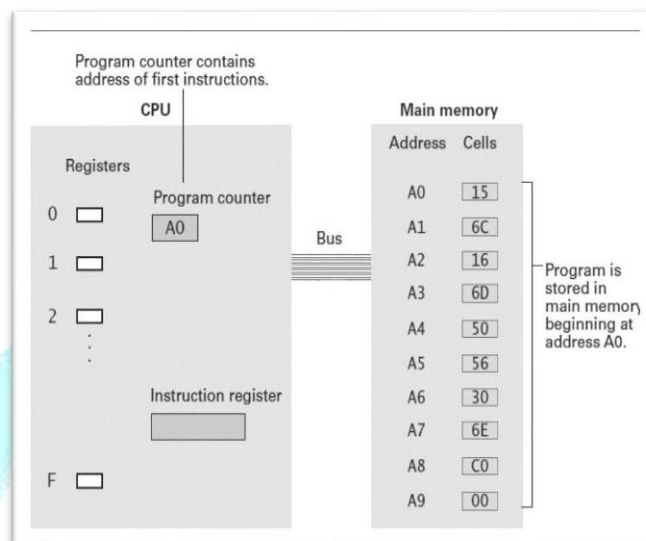


Figure 46: Machine instructions loaded in the main memory

The CPU begins the fetch step of the machine cycle by extracting the instruction stored in main memory at location A0 and placing this instruction (156C) in its instruction register. Notice that, in our machine, instructions are 16 bits (two bytes) long. Thus, the entire instruction to be fetched occupies the memory cells at both address A0 and A1. The CPU is designed to take this into account, so it retrieves the contents of both cells and places the bit patterns received in the instruction register, which is 16 bits long. The CPU then adds two to the program counter so that this register contains the address of the next instruction. At the end of the fetch step of the first machine cycle, the program counter and instruction register contain the following data:

Program Counter: A2

Instruction Register: 156C

Next, the CPU analyzes the instruction in its instruction register and concludes that it is to load register 5 with the contents of the memory cell at address 6C. This load activity is performed during the execution step of the machine cycle, and the CPU then begins the next cycle.

This cycle begins by fetching the instruction 166D from the two memory cells starting at address A2. The CPU places this instruction in the instruction register and increments the program counter to A4. The values in the program counter and instruction register therefore become the following:

Program Counter: A4

Instruction Register: 166D

Now the CPU decodes the instruction 166D and determines that it is to load register 6 with the contents of memory address 6D. It then executes the instruction. It is at this time that register 6 is actually loaded.

Since the program counter now contains A4, the CPU extracts the next instruction starting at this address. The result is that 5056 is placed in the instruction register, and the program counter is incremented to A6. The CPU now decodes the contents of its instruction register and executes it by activating the two's complement addition circuitry with inputs being registers 5 and 6.

During this execution step, the arithmetic/logic unit performs the requested addition, leaves the result in register 0 (as requested by the control unit), and reports to the control unit that it has finished. The CPU then begins another machine cycle. Once again, with the aid of the program counter, it fetches the next instruction (306E) from the two memory cells starting at memory location A6 and increments the program counter to A8. This instruction is then **decoded and executed**. At this point, the sum is placed in memory location 6E.

The next instruction is fetched starting from memory location A8, and the program counter is incremented to AA. The contents of the instruction register (C000) are now decoded as the halt instruction. Consequently, the machine stops during the execute step of the machine cycle, and the program is completed.



Module 38

38. Data Manipulation: Logic Operators

Bitwise operations that combine two strings of bits to produce a single output string by applying the basic operation to individual columns. For example, the result of ANDing the patterns 10011010 and 11001001 results in

```

10011010
AND 11001001
10001000

```

where we have merely written the result of ANDing the two bits in each column at the bottom of the column. Likewise, ORing and XORing these patterns would produce

```

10011010      10011010
OR 11001001   XOR 11001001
11011011      01010011

```

One of the major uses of the AND operation is for placing 0s in one part of a bit pattern while not disturbing the other part. There are many applications for this in practice, such as filtering certain colors out of a digital image represented in the RGB format, as described in the previous chapter. Consider, for example, what happens if the byte 00001111 is the first operand of an AND operation without knowing the contents of the second operand, we still can conclude that the four most significant bits of the result will be 0s. Moreover, the four least significant bits of the result will be a copy of that part of the second operand, as shown in the following example:

```

00001111
AND 10101010
00001010

```

This use of the AND operation is an example of the process called **masking**. Here one operand, called a **mask**, determines which part of the other operand will affect the result. In the case of the AND operation, masking produces a result that is a partial replica of one of the operands, with 0s occupying the non-duplicated positions. One trivial use of the AND operation in this context would be to mask off all of the bits associated with the red component of the pixels in an image, leaving only the blue and green components. This transformation is frequently available as an option in image manipulation software.

AND operations are useful when manipulating other types of **bit map**

besides images, whenever a string of bits is used in which each bit represents the presence or absence of a particular object. As a non-graphical example, a string of 52 bits, in which each bit is associated with a particular playing card, can be used to represent a poker hand by assigning 1s to those five bits associated with the cards in the hand and 0s to all the others. Likewise, a bit map of 52 bits, of which thirteen are 1s, can be used to represent a hand of bridge, or a bit map of 32 bits can be used to represent which of thirty-two ice cream flavors are available.

Suppose, then, that the eight bits in a memory cell are being used as a bit map, and we want to find out whether the object associated with the third bit from the high-order end is present. We merely need to AND the entire byte with the mask 00100000, which produces a byte of all 0s if and only if the third bit from the high-order end of the bit map is itself 0. A program can then act accordingly by following the AND operation with a conditional branch instruction. Moreover, if the third bit from the high-order end of the bit

map is a 1, and we want to change it to a 0 without disturbing the other bits, we can AND the bit map with the mask 11011111 and then store the result in place of the original bit map.

Where the AND operation can be used to duplicate a part of a bit string while placing 0s in the non-duplicated part, the OR operation can be used to duplicate a part of a string while putting 1s in the non-duplicated part. For this we again use a mask, but this time we indicate the bit positions to be duplicated with 0s and use 1s to indicate the non-duplicated positions. For example, ORing any byte with 11110000 produces a result with 1s in its most significant four bits while its remaining bits are a copy of the least significant four bits of the other operand, as demonstrated by the following example:

```
11110000
OR 10101010
11111010
```

Consequently, whereas the mask 11011111 can be used with the AND operation to force a 0 in the third bit from the high-order end of a byte, the mask 00100000 can be used with the OR operation to force a 1 in that position.

A major use of the XOR operation is in forming the complement of a bit string. XORing any byte with a mask of all 1s produces the complement of the byte. For example, note the relationship between the second operand and the result in the following example:

```
11111111
XOR 10101010
01010101
```

The XOR operation can be used to invert all of the bits of an RGB bitmap image, resulting in an “inverted” color image in which light colors have been replaced by dark colors, and vice versa.

Module 39

39. Data Manipulation: Rotation and Shift

The operations in the class of rotation and shift operations provide a means for moving bits within a register and are often used in solving alignment problems. These operations are classified by the direction of motion (right or left) and whether the process is circular. Within these classification guidelines are numerous variations with mixed terminology. Let us take a quick look at the ideas involved.

Consider a register containing a byte of bits. If we shift its contents one bit to the right, we imagine the rightmost bit falling off the edge and a hole appearing at the leftmost end. What happens with this extra bit and the hole is the distinguishing feature among the various shift operations. One technique is to place the bit that fell off the right end in the hole at the left end. The result is a **circular shift, also called a rotation**. Thus, if we perform a right circular shift on a byte size bit pattern **eight times**, we obtain the same bit pattern we started with.

Another technique is to discard the bit that falls off the edge and always fill the hole with a 0. The term **logical shift** is often used to refer to these operations. Such shifts to the left can be used for multiplying two's complement representations by two. After all, shifting binary digits to the left corresponds to multiplication by two, just as a similar shift of decimal digits corresponds to multiplication by ten. Moreover, division by two can be accomplished by shifting the binary string to the right. In either shift, care must be taken to preserve the sign bit when using certain notational systems. Thus, we often find right shifts that always fill the hole (which occurs at the sign bit position) with its original value. Shifts that leave the sign bit unchanged are sometimes called **arithmetic shifts**.

ZAINEE
ACADEMY

Module 40**40. Data Manipulation: Arithmetic Operators**

Subtraction can be simulated by **addition and negation** like in 2's complement notation 7 -5 would be 7 + (-5) which means the binary of 7 will be added to binary of -5 as demonstrated in the Figure 47.

Problem in base 10	Problem in two's complement	Answer in base 10
$\begin{array}{r} 3 \\ + 2 \\ \hline \end{array}$	$\begin{array}{r} 0011 \\ + 0010 \\ \hline 0101 \end{array}$	→ 5
$\begin{array}{r} -3 \\ + -2 \\ \hline \end{array}$	$\begin{array}{r} 1101 \\ + 1110 \\ \hline 1011 \end{array}$	→ -5
$\begin{array}{r} 7 \\ + -5 \\ \hline \end{array}$	$\begin{array}{r} 0111 \\ + 1011 \\ \hline 0010 \end{array}$	→ 2

Figure 47: Arithmetic Operations Examples

Multiplication

✓ **Multiplication is repetitive addition**

- ✓ For example, 8 multiplied by 3, we will add 8 three times

Division

✓ **Can be achieved through subtraction**

- ✓ Some small CPUs are designed to have just add, or just add and subtract to do all arithmetic operations.

Remember

- ✓ If its stores as 2's complement, it is straight forward
- ✓ If it save in floating point notation, then you know you need to use mantissa, exponent and sign bit.

Although both are additions, but both have different workout.

Module 41

41. Data Manipulation: Role of Controller

Communication between a computer and other devices is normally handled through an intermediary apparatus known as a **controller**. In the case of a personal computer, a controller may consist of circuitry permanently mounted on the computer's motherboard or, for flexibility, it may take the form of a circuit board that plugs into a slot on the motherboard. In either case, the controller connects via cables to peripheral devices within the computer case or perhaps to a connector, called a **port**, on the back of the computer where external devices can be attached. These controllers are sometimes small computers themselves, each with its own memory circuitry and simple CPU that performs a program directing the activities of the controller.

A controller translates messages and data back and forth between forms compatible with the internal characteristics of the computer and those of the peripheral device to which it is attached. Originally, each controller was designed for a particular type of device; thus, purchasing a new peripheral device often required the purchase of a new controller as well.

Recently, steps have been taken within the personal computer arena to develop standards, such as the **universal serial bus (USB)** and **FireWire**, by which a single controller is able to handle a variety of devices. For example, a single USB controller can be used as the interface between a computer and any collection of USB-compatible devices. The list of devices on the market today that can communicate with a USB controller includes mice, printers, scanners, mass storage devices, digital cameras, and smartphones.

Each controller communicates with the computer itself by means of connections to the same bus that connects the computer's CPU and main memory (Figure 48). From this position it is able to monitor the signals being sent between the CPU and main memory as well as to inject its own signals onto the bus.

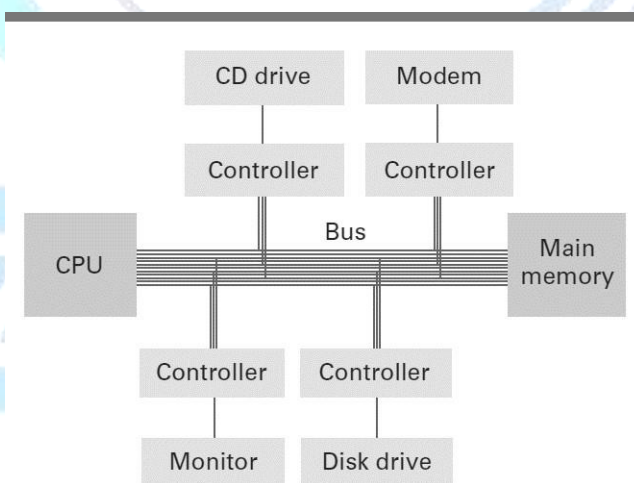


Figure 48: Controllers attached to a machine's bus

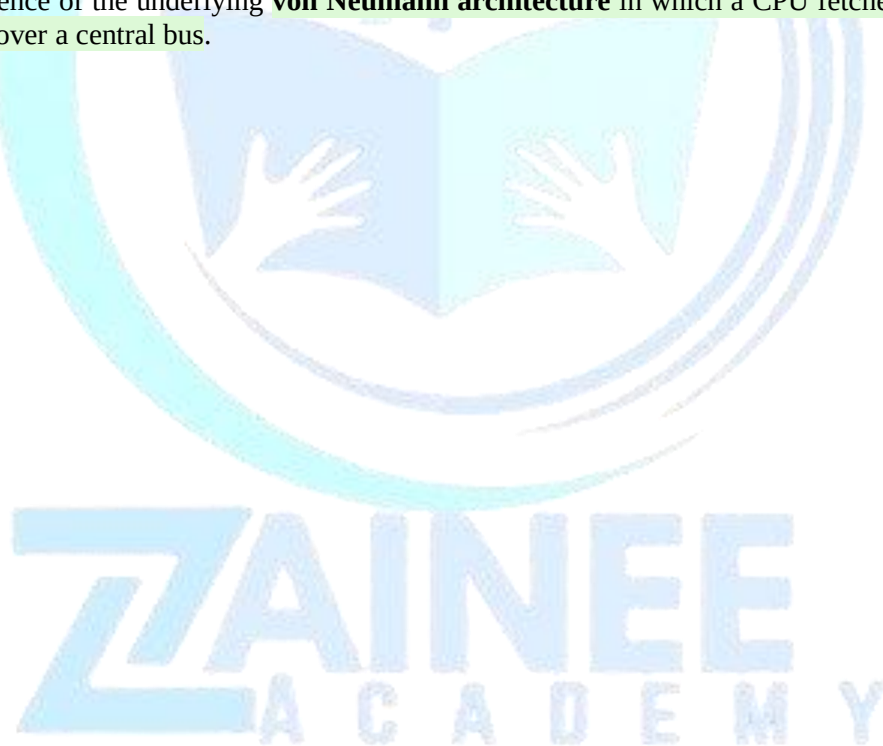
With this arrangement, the CPU is able to communicate with the controllers attached to the bus in the same manner that it communicates with main memory. To send a bit pattern to a controller, the bit pattern is first constructed in one of the CPU's general-purpose registers. Then an instruction similar to a **STORE instruction** is executed by the CPU to "store" the bit pattern in the controller. Likewise, to receive a bit pattern from a controller, an instruction similar to a **LOAD instruction** is used.

Module 42

42. Data Manipulation: Direct Memory Access and Handshaking

Since a controller is attached to a computer's bus, it can carry on its own communication with main memory during those **nanoseconds** in which the CPU is not using the bus. This **ability of a controller to access main memory is known as direct memory access (DMA)**, and it is a **significant asset to a computer's performance**. For instance, to retrieve data from a sector of a disk, the CPU can send encoded bit patterns to the controller attached to the disk asking the controller to read the sector and place the data in a specified area of main memory. The CPU can then continue with other tasks while the controller performs the read operation and deposits the data in main memory via DMA. Thus, two activities will be performed at the same time. The CPU will be executing a program and the controller will be overseeing the transfer of data between the disk and main memory. In this manner, the computing resources of the CPU are not wasted during the relatively slow data transfer.

The use of DMA also has the **detrimental effect of complicating** the communication taking place over a computer's bus. Bit patterns must move between the CPU and main memory, between the CPU and each controller, and between each controller and main memory. Coordination of all this activity on the bus is a major design issue. Even with excellent designs, the central bus can become an **impediment as the CPU** and the controllers compete for bus access. This impediment is known as the **von Neumann bottleneck** because it is a consequence of the underlying **von Neumann architecture** in which a CPU fetches its instructions from memory over a central bus.



Module 43

43. Data Manipulation: Communication media and communication rates

Communication between computing devices is handled over **two types** of paths:

- ✓ Parallel communication
- ✓ Serial communication

43.1. Parallel Communication

- ✓ **Several signals are transferred at the same time, each on a separate line.**
- ✓ Data transfer is good
- ✓ Requires complex architecture

It was illustrated in the Figure 48.

43.2. Serial Communication

- ✓ **Transfer data one after the other,**
- ✓ Requires a simple data path
- ✓ Data transfer rate is relatively slower than parallel communication.
- ✓ **USB and FireWire are examples of high-speed data transfer over a short distance.**
- ✓ **Ethernet connections for slightly longer distances.**

It has been shown in the Figure 49.



Figure 49: Serial communication

- ✓ Traditional voice lines dominated the PC arena for many years.
- ✓ Converting bit patterns to audible tones using **Modem (Modulator-demodulator)**
- ✓ For faster long-distance communication, **DSL (Digital Subscriber Line)**
- ✓ **Cable Modems:**
- ✓ **Fiber Optics and Coaxial cables for high definition TV and computer network.**
- ✓ Rate at which bits are transferred from one computing component to another measured in bits per second (**bps**)
 - ✓ **Kbps (Kilo bits)**
 - ✓ **Mbps (Million bits)**
 - ✓ **Gbps (Billion bits)**
 - ✓ **8kbps = 1KB per second.**
 - ✓ **USB and Firewire provide several hundred Mbps.**

Module 44**44. Data Manipulation: Pipelining**

Electric pulses travel through a wire no faster than the speed of light. Since light travels approximately 1 foot in a nanosecond (one billionth of a second), it requires at **least 2 nanoseconds for the CPU** to fetch an instruction from a memory cell that is 1 foot away. (The read request must be sent to memory, requiring at least 1 nanosecond, and the instruction must be sent back to the CPU, requiring at least another nanosecond.) Consequently, to fetch and execute an instruction in such a machine requires several nanoseconds—which means that increasing the execution speed of a machine ultimately becomes **a miniaturization problem**. However, increasing execution speed is not the only way to improve a computer's performance. The real goal is to improve the machine's **throughput**, which refers to the total amount of work the machine can accomplish in a given amount of time.

An example of how a computer's throughput can be increased without requiring an increase in execution speed involves **pipelining**, which is the technique of allowing the steps in the machine cycle to overlap. In particular, while one instruction is being executed, the next instruction can be fetched, which means that more than one instruction can be in “the pipe” at any one time, each at a different stage of being processed. In turn, the total throughput of the machine is increased even though the time required to fetch and execute each individual instruction remains the same. (Of course, when a JUMP instruction is reached, any gain that would have been obtained by prefetching is not realized because the instructions in “the pipe” are not the ones needed after all.)

Modern machine designs push the pipelining concept beyond our simple example. They are often capable of fetching several instructions at the same time and actually executing more than one instruction at a time when those instructions do not rely on each other.

ZAINEE
ACADEMY

Module 45

45. Operating Systems: History

Today's operating systems are large, complex software packages that have grown from humble beginnings. The computers of the **1940s and 1950s were not very flexible or efficient**. Machines occupied entire rooms. Program execution required significant preparation of equipment in terms of mounting magnetic tapes, placing punched cards in card readers, setting switches, and so on. The execution of each program, called a **job**, was handled as an isolated activity—the machine was prepared for executing the program, the program was executed, and then all the tapes, punched cards, etc. had to be retrieved before the next program preparation could begin. When several users needed to share a machine, sign-up sheets were provided so that users could reserve the machine for blocks of time. During the time period allocated to a user, the machine was totally under that user's control. The session usually began with program setup, followed by short periods of program execution. It was often completed in a hurried effort to do just one more thing ("It will only take a minute") while the next user was impatiently starting to set up.

In such an environment, operating systems began as systems for simplifying program setup and for streamlining the transition between jobs. One early development was the separation of users and equipment, which eliminated the physical transition of people in and out of the computer room. For this purpose, a computer operator was hired to operate the machine. Anyone wanting a program run was required to submit it, along with any required data and special directions about the program's requirements, to the operator and return later for the results. The operator, in turn, loaded these materials into the machine's mass storage where a program **called the operating system** could read and execute them one at a time. This was the beginning of **batch processing**—the execution of jobs by collecting them in a single batch, then executing them without further interaction with the user.

In batch processing systems, the jobs residing in mass storage wait for execution in a **job queue** (Figure 50). A **queue** is a storage organization in which objects (in this case, jobs) are ordered in **first-in, first-out (abbreviated FIFO and pronounced "FI-foe")** fashion. That is, the objects are removed from the queue in the order in which they arrived. In reality, most job queues do not rigorously follow the FIFO structure, since most operating systems provide for consideration of job priorities. As a result, a job waiting in the job queue can be bumped by a higher-priority job.

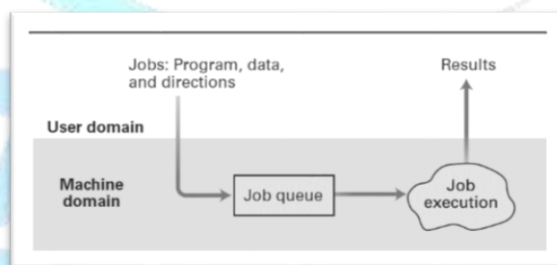


Figure 50: Batch Processing

In early batch-processing systems, each job was accompanied by a set of instructions explaining the steps required to prepare the machine for that particular job. These instructions were encoded, using a system known as a **job control language (JCL)**, and stored with the job in the job queue. When the job was selected for execution, the operating system printed these instructions at a printer where they could be read and followed by the computer operator. This communication between the operating system and the computer operator is still seen today, as witnessed by PC operating systems that report such errors as "network not available" and "printer not responding."

A major drawback to using a computer operator as an intermediary between a computer and its users is that the users have no interaction with their jobs once they are submitted to the operator. This approach is acceptable for some applications, such as payroll processing, in which the data and all processing decisions are established in advance. However, it is not acceptable when the user must interact with a program during its execution. Examples include reservation systems in which reservations and cancellations must be reported as they occur; word processing systems in which documents are developed in a dynamic write and rewrite manner; and computer games in which interaction with the machine is the central feature of the game.

To accommodate these needs, new operating systems were developed that allowed a program being executed to carry on a dialogue with the user through remote terminals—a feature known as **interactive processing** (Figure 51). (A terminal consisted of little more than an electronic typewriter by which the user could type input and read the computer's response that was printed on paper. Today terminals have evolved into more sophisticated devices called workstations and even into complete PCs that can function as stand-alone computers when desired.)

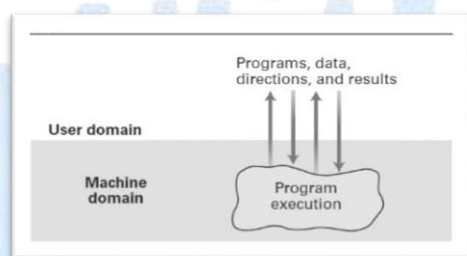


Figure 51: Interactive Processing

Module 46**46. Operating Systems: Basic Concepts (I)****Coordination with User**

- ✓ Successfully interactive processing is to respond the users with sufficiently **fast time**.
- ✓ Printing record of all students VU versus word processing (typing characters)
- ✓ Execution of tasks under a deadline

Real-time Processing

- ✓ Computer performs the tasks in accordance with the deadlines in its external real-world environment.
- ✓ Example of Cruise Missile, Radar etc.

Interactive system and Real-time Processing

- ✓ If a system is servicing only one user, real-time processing was relatively easier to implement, however, computers in 60s, 70s were expensive,
- ✓ Each machine had to serve more than one user at remote terminals.

Time Sharing

- ✓ Based on this problem, such OS were designed to service multiple users at the same time called time-sharing. Active, Ready, waiting states as illustrated in the Figure 52.

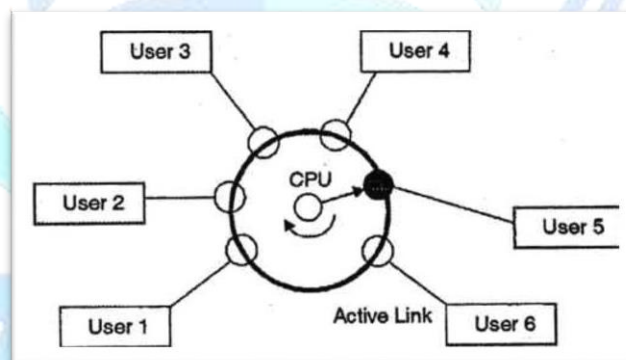


Figure 52: Time Sharing

Multi-programming

- ✓ One way of implementing time-sharing, small time intervals, and each job is executed for such a small interval.
- ✓ Early systems were able to service 30 users simultaneously.

Multitasking

- ✓ One user executing several tasks simultaneously.
- ✓ Today Multiprogramming is **used in single user and multiple users**.

Module 47**47. Operating Systems: Basic Concepts (II)**

With the development of multiuser, time-sharing operating systems, a typical computer installation was configured as a large central computer connected to numerous workstations. From these workstations, users could communicate directly with the computer from outside the computer room rather than submitting requests to a computer operator. Commonly used programs were stored in the machine's mass storage devices, and operating systems were designed to execute these programs as requested from the workstations. In turn, the role of a computer operator as an intermediary between the users and the computer begins to fade.

Today, the existence of a computer operator has essentially disappeared, especially in the arena of personal computers where the computer user assumes all of the responsibilities of computer operation. Even most large computer installations run essentially unattended. Indeed, the job of computer operator has given way to that of a system administrator who manages the computer system—obtaining and overseeing the installation of new equipment and software, enforcing local regulations such as the issuing of new accounts and establishing mass storage space limits for the various users, and coordinating efforts to resolve problems that arise in the system—rather than operating the machines in a hands-on manner.

In short, operating systems have grown from simple programs that retrieved and executed programs one at a time into complex systems that coordinate time-sharing, maintain programs and data files in the machine's mass storage devices, and respond directly to requests from the computer's users.

But the evolution of operating systems continues. The development of multiprocessor machines has led to operating systems that provide time-sharing/ multitasking capabilities by assigning different tasks to different processors as well as by sharing the time of each single processor. These operating systems must wrestle with such problems as **load balancing** (dynamically allocating tasks to the various processors so that all processors are used efficiently) as well as **scaling** (breaking tasks into a number of subtasks compatible with the number of processors available).

Still another direction of research in operating systems focuses on devices that are dedicated to specific tasks such as medical devices, vehicle electronics, home appliances, cell phones, or other hand-held computers. The computer systems found in these devices are known as **embedded systems**. Embedded operating systems are often expected to conserve battery power, meet demanding real-time deadlines, or operate continuously with little or no human oversight. Successes in this endeavor are marked by systems such as **VxWORKS**, developed by Wind River Systems and used in the Mars Exploration Rovers named Spirit and Opportunity; **Windows CE** (also known as Pocket PC) developed by Microsoft; and Palm OS developed by PalmSource, Inc., especially for use in hand-held devices.

Module 48

48. Operating Systems: Software Classification

Let us begin by dividing a machine's software into two broad categories: **application software** and **system software** (Figure 53). **Application software** consists of the programs for performing tasks particular to the machine's utilization. A machine used to maintain the inventory for a manufacturing company will contain different application software from that found on a machine used by an electrical engineer. Examples of application software include spreadsheets, database systems, desktop publishing systems, accounting systems, program development software, and games.

In contrast to application software, system software performs those tasks that are common to computer systems in general. In a sense, the system software provides the infrastructure that the application software requires, in much the same manner as a nation's infrastructure (government, roads, utilities, financial institutions, etc.) provides the foundation on which its citizens rely for their individual lifestyles.

Within the class of system software are two categories: **One is the operating system** itself and the other consists of **software units collectively known as utility software**. The majority of an installation's utility software consists of programs for performing activities that are fundamental to computer installations but not included in the operating system. In a sense, utility software consists of software units that extend (or perhaps customize) the capabilities of the operating system. For example, the ability to format a magnetic disk or to copy a file from a magnetic disk to a CD is often not implemented within the operating system itself but instead is provided by means of a utility program. Other instances of utility software include software to compress and decompress data, software for playing multimedia presentations, and software for handling network communication.

Implementing certain activities as utility software allows system software to be customized to the needs of a particular installation more easily than if they were included in the operating system. Indeed, it is common to find companies or individuals who have modified, or added to, the utility software that was originally provided with their machine's operating system.

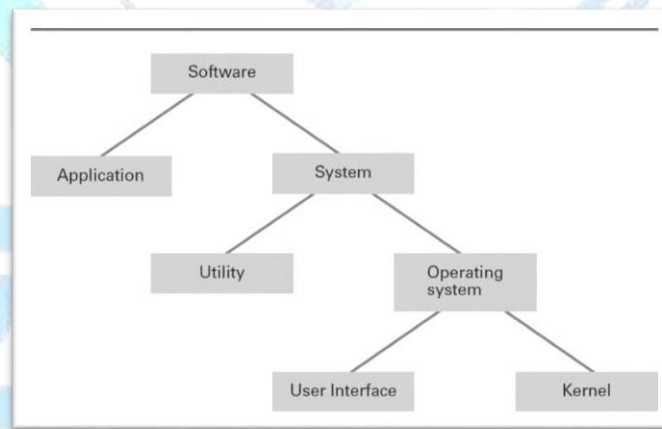


Figure 53: Software Classification

Module 49

49. Operating Systems: Components (I)

Let us focus now on components that are within the domain of an operating system. In order to perform the actions requested by the computer's users, an operating system must be able to communicate with those users. The portion of an operating system that handles this communication is often called the **user interface**. Older user interfaces, called **shells**, communicated with users through textual messages using a keyboard and monitor screen. More modern systems perform this task by means of a **graphical user interface (GUI—pronounced “GOO-ee”)** in which objects to be manipulated, such as files and programs, are represented pictorially on the display as icons. These systems allow users to issue commands by using one of several common input devices. For example, a computer mouse, with one or more buttons, can be used to click or drag icons on the screen. In place of a mouse, special purpose pointing devices or styluses are often used by graphic artists or on several types of handheld devices. More recently, advances in fine-grained touch screens allow users to manipulate icons directly with their fingers. Whereas today's GUIs use two-dimensional image projection systems, three-dimensional interfaces that allow human users to communicate with computers by means of 3D projection systems, tactile sensory devices, and surround sound audio reproduction systems are subjects of current research.

Although an operating system's user interface plays an important role in establishing a machine's functionality, this framework merely acts as an intermediary between the computer's user and the real heart of the operating system (Figure 54). This distinction between the user interface and the internal parts of the operating system is emphasized by the fact that some operating systems allow a user to select among different interfaces to obtain the most comfortable interaction for that particular user. Users of the UNIX operating system, for example, can select among a variety of shells including the Bourne shell, the C shell, and the Korn shell, as well as a GUI called X11. The earliest versions of Microsoft Windows were a GUI application program that could be loaded from the MS-DOS operating system's command shell. The DOS cmd.exe shell can still be found as a utility program in the latest versions of Windows, although this interface is almost never required by casual users. Similarly, Apple's OS X retains a Terminal utility shell that harkens back to that system's UNIX ancestors.

An important component within today's GUI shells is the **window manager**, which allocates blocks of space on the screen, called windows, and keeps track of which application is associated with each window. When an application wants to display something on the screen, it notifies the window manager, and the window manager places the desired image in the window assigned to the application. In turn, when a mouse button is clicked, it is the window manager that computes the mouse's location on the screen and notifies the appropriate application of the mouse action. Window managers are responsible for what is generally called the “style” of a GUI, and most managers offer a range of configurable choices. Linux users even have a range of choices for a window manager, with popular choices including KDE and Gnome.

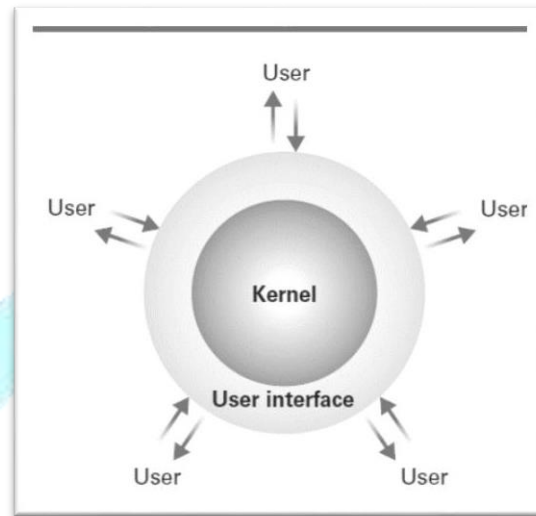


Figure 54: Components of OS

ZAINEE
ACADEMY

Module 50

50. Operating Systems: Components (II)

In contrast to an operating system's user interface, **the internal part of an operating system is called the kernel.** An operating system's kernel contains those software components that perform the very basic functions required by the computer installation. One such unit is the **file manager**; whose job is to coordinate the use of the machine's mass storage facilities. More precisely, the file manager maintains records of all the files stored in mass storage, including where each file is located, which users are allowed to access the various files, and which portions of mass storage are available for new files or extensions to existing files. These records are kept on the individual storage medium containing the related files so that each time the medium is placed online, the file manager can retrieve them and thus know what is stored on that particular medium.

For the convenience of the machine's users, most file managers allow files to be grouped into a bundle called a **directory** or **folder**. This approach allows a user to organize his or her files according to their purposes by placing related files in the same directory. Moreover, by allowing directories to contain other directories, called subdirectories, a hierarchical organization can be constructed. For example, a user may create a directory called MyRecords that contains sub-directories called FinancialRecords, MedicalRecords, and HouseHoldRecords. Within each of these subdirectories could be files that fall within that particular category. (Users of a Windows operating system can ask the file manager to display the current collection of folders by executing the utility program Windows Explorer.)

A chain of directories within directories is called a **directory path**. Paths

are often expressed by listing the directories along the path separated by slashes. For instance, animals/prehistoric/dinosaurs would represent the path starting at the directory named animals, passing through its subdirectory named prehistoric, and terminating in the sub-subdirectory dinosaurs. (For Windows users the slashes in such a path expression are reversed as in animals\ prehistoric\dinosaurs.)

Any access to a file by other software units is obtained at the discretion of the file manager. The procedure begins by requesting that the file manager grant access to the file through a procedure known as opening the file. If the file manager approves the requested access, it provides the information needed to find and to manipulate the file.

Another component of the kernel consists of a collection of **device drivers**, which are the software units that communicate with the controllers (or at times, directly with peripheral devices) to carry out operations on the peripheral devices attached to the machine. Each device driver is uniquely designed for its particular type of device (such as a printer, disk drive, or monitor) and translates generic requests into the more technical steps required by the device assigned to that driver. For example, a device driver for a printer contains the software for reading and decoding that particular printer's status word as well as all the other hand-shaking details. Thus, other software components do not have to deal with those technicalities in order to print a file. Instead, the other components can merely rely on the device driver software to print the file and let the device driver take care of the details. In this manner, the design of the other software units can be independent of the unique characteristics of particular devices. The result is a generic operating system that can be customized for particular peripheral devices by merely installing the appropriate device drivers.

Still another component of an operating system's kernel is the **memory manager**, which is charged with the task of coordinating the machine's use of main memory. Such duties are minimal in an environment in which a computer is asked to perform only one task at a time. In these cases, the program for performing the current task is placed at a predetermined location in main memory, executed, and then replaced by the program for performing the next task. However, in multiuser or multitasking environments in which the computer is asked to address many needs at the same time, the

duties of the memory manager are extensive. In these cases, many programs and blocks of data must reside in main memory concurrently. Thus, the memory manager must find and assign memory space for these needs and ensure that the actions of each program are restricted to the program's allotted space. Moreover, as the needs of different activities come and go, the memory manager must keep track of those memory areas no longer occupied. The task of the memory manager is complicated further when the total main memory space required exceeds the space actually available in the computer. In this case the memory manager may create the illusion of additional memory space by rotating programs and data back and forth between main memory and mass storage (a technique called **paging**). Suppose, for example, that a main memory of 8GB is required but the computer only has 4GB. To create the illusion of the larger memory space, the memory manager reserves 4GB of storage space on a magnetic disk. There it records the bit patterns that would be stored in main memory if main memory had an actual capacity of 8GB. This data is divided into uniform sized units called **pages**, which are typically a few KB in size. Then the memory manager shuffles these pages back and forth between main memory and mass storage so that the pages that are needed at any given time are actually present in the 4GB of main memory. The result is that the computer is able to function as though it actually had 8GB of main memory. This large "fictional" memory space created by paging is called **virtual memory**.

Two additional components within the kernel of an operating system are the **scheduler** and **dispatcher**, which we will study in the next section. For now, we merely note that in a multiprogramming system the scheduler determines which activities are to be considered for execution, and the dispatcher controls the allocation of time to these activities.

ZAINEE
ACADEMY

Module 51

51. Operating Systems: Process of Booting

We have seen that an operating system provides the software infrastructure required by other software units, but we have not considered how the operating system gets started. This is accomplished through a procedure known as **boot strapping** (often shortened to **booting**) that is performed by a computer each time it is **turned on**. It is this procedure that transfers the operating system from mass storage (where it is permanently stored) into main memory (which is essentially empty when the machine is first turned on). To understand the boot strap process and the reason it is necessary, we begin by considering the machine's CPU.

A CPU is designed so that its program counter starts with a particular predetermined address each time the CPU is turned on. It is at this location that the CPU expects to find the beginning of the program to be executed. Conceptually, then, all that is needed is to store the operating system at this location. However, for technical reasons, a computer's main memory is typically constructed from **volatile technologies**—meaning that the memory loses the data stored in it when the computer is turned off. Thus, the contents of main memory must be replenished each time the computer is restarted.

In short, we need a program (preferably the operating system) to be present in main memory when the computer is first turned on, but the computer's volatile memory is erased each time the machine is turned off. To resolve this dilemma, a small portion of a computer's main memory where the CPU expects to find its initial program is constructed from special nonvolatile memory cells. Such memory is known as **read-only memory (ROM)** because its contents can be read but not altered. As an analogy, you can think of storing bit patterns in ROM as blowing tiny fuses (some blown open—ones—and some blown closed—zeros), although the technology used is more advanced. More precisely, most ROM in today's PCs is constructed with flash memory technology (which means that it is not strictly ROM because it can be altered under special circumstances).

In a general-purpose computer, a program called the **boot loader** is permanently stored in the machine's ROM. This, then, is the program that is initially executed when the machine is turned on. The instructions in the boot loader direct the CPU to transfer the operating system from a predetermined location into the volatile area of main memory (Figure 55). Modern boot loaders can copy an operating system into main memory from a variety of locations. For example, in embedded systems, such as smartphones, the operating system is copied from special flash (nonvolatile) memory; in the case of **small workstations** at large companies or universities, the operating system may be copied from a distant machine over a network. Once the operating system has been placed in main memory, the boot loader directs the CPU to execute a jump instruction to that area of memory. At this point, the operating system takes over and begins controlling the machine's activities. The overall process of executing the boot loader and thus starting the operating system is called **booting** the computer.

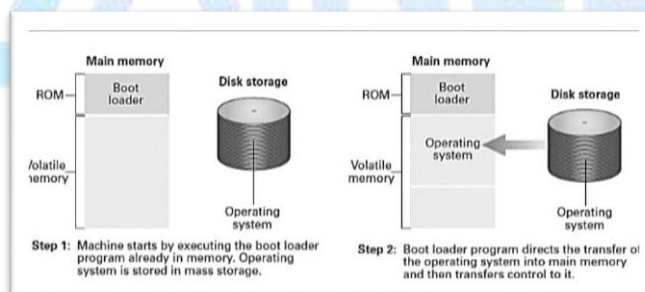


Figure 55: process of Booting

You may ask why desktop computers are not provided with enough ROM to hold the entire operating system so that booting from mass storage would not be necessary. While this is feasible for embedded

systems with small operating systems, devoting large blocks of main memory in general-purpose computers to nonvolatile storage is not efficient with today's technology. Moreover, computer operating systems undergo frequent updates in order to maintain security and keep abreast of new and improved device drivers for the latest hardware. While it is possible to update operating systems and boot loaders stored in ROM (often called a **firmware update**), the technological limits make mass storage the most common choice for more traditional computer systems



Module 52**52. Operating Systems: Process and its Administration****Coordinating Machine's activities**

- ✓ How OS coordinates the execution of application software, utility software, or OS itself?

Program vs Process

- ✓ Fundamental concepts of modern operating systems is the distinction between a program and the activity of executing a program.
- ✓ Static vs dynamic activity.
- ✓ Analogy of music sheet.

Process

- ✓ The activity of executing a program under the control of the operating system is known as a process.
- ✓ Associated with a process is the current status of the activity, called the process state.

Process state

- ✓ The value of Program counter
- ✓ Values in other CPU registers
- ✓ Values in associated memory cell.
- ✓ Snapshot of the machine at particular time.

ZAINEE
ACADEMY

Module 53

53. Operating Systems: Process and its Administration

The tasks associated with coordinating the execution of processes are handled by the scheduler and dispatcher within the operating system's kernel. The scheduler maintains a record of the processes present in the computer system, introduces new processes to this pool, and removes completed processes from the pool. Thus when a user requests the execution of an application, it is the scheduler that adds the execution of that application to the pool of current processes.

To keep track of all the processes, the scheduler maintains a block of information in main memory called **the process table**. Each time the execution of a program is requested, the scheduler creates a new entry for that process in the process table. This entry contains such information as the memory area assigned to the process (obtained from the memory manager), the priority of the process, and whether the process is ready or waiting. A process is **ready** if it is in a state in which its progress can continue; it is **waiting** if its progress is currently delayed until some external event occurs, such as the completion of a mass storage operation, the pressing of a key at the keyboard, or the arrival of a message from another process.

The dispatcher is the component of **the kernel** that oversees the execution of the scheduled processes. In a time-sharing/multitasking system this task is accomplished by **multiprogramming**; that is, dividing time into short segments, each **called a time slice** (typically measured in milliseconds or microseconds), and then switching the CPU's attention among the processes as each is allowed to execute for one time slice (Figure 56). The procedure of changing from one process to another is called a **process switch** (or a **context switch**).

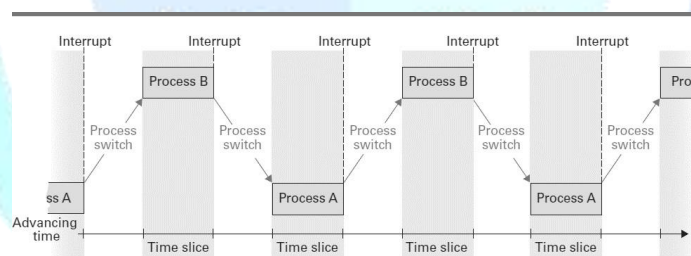


Figure 56: Multiprogramming between process A and process B

Each time the dispatcher awards a time slice to a process, it initiates a timer circuit that will indicate the end of the slice by generating a signal called an **interrupt**. The CPU reacts to this interrupt signal in much the same way that you react when interrupted from a task. You stop what you are doing, record where you are in the task (so that you will be able to return at a later time) and take care of the interrupting entity. When the CPU receives an interrupt signal, it completes its current machine cycle, saves its position in the current process, and begins executing a program, called an **interrupt handler**, which is stored at a predetermined location in main memory. This interrupt handler is a part of the dispatcher, and it describes how the dispatcher should respond to the interrupt signal.

Thus, the effect of the interrupt signal is to preempt the current process and transfer control back to the dispatcher. At this point, the dispatcher selects the process from the process table that has the highest priority among the ready processes (as determined by the scheduler), restarts the timer circuit, and allows the selected process to begin its time slice.

Paramount to the success of a multiprogramming system is the ability to stop, and later restart, a process. If you are interrupted while reading a book, your ability to continue reading at a later time depends on your ability to remember your location in the book as well as the information that you had accumulated to that point. In short, you must be able to re-create the environment that was present immediately prior to the interruption.

In the case of a process, the environment that must be re-created is the process's state, which as already mentioned, includes the value of the program counter as well as the contents of **the registers and pertinent memory cells**. CPUs designed for multiprogramming systems incorporate the task of saving this information as part of the CPU's reaction to the interrupt signal. These CPUs also tend to have machine-language instructions for reloading a previously saved state. Such features simplify the task of the dispatcher when performing a process switch and exemplify how the design of modern CPUs is influenced by the needs of today's operating systems.



Module 54

54. Operating Systems: Handling Competition between Processes

An important task of an operating system is the allocation of the **machine's resources** to the processes in the system. Here we are using the term *resource* in a broad sense, including the machine's peripheral devices as well as features within the machine itself. The **file manager** allocates access to files as well and allocates mass storage space for the construction of new files; the **memory manager** allocates memory space; the **scheduler** allocates space in **the process table**; and **the dispatcher allocates time slices**. As with many problems in computer systems, this allocation task may appear simple at first glance. Below the surface, however, lie several subtleties that can lead to malfunctions in a poorly designed system. Remember, a machine does not think for itself; it merely follows directions. Thus, to construct reliable operating systems, we must develop **algorithms** that cover every possible contingency, regardless of how minuscule it may appear.

Resources

- ✓ File manager allocates access to files
- ✓ Memory manager allocates the memory space.
- ✓ Scheduler allocates space in the process table
- ✓ Dispatcher allocates timeslots,

Resources Allocation

- ✓ Such resource allocation looks quite simpler, however, it's not so straight forward.
- ✓ Machine does not think about itself, it's You, who can/cannot design reliable OS

Resource Allocation Issues

- ✓ What if two processes demand the same resource at one time.
- ✓ What if 'Process A' is utilizing 'Resource 1' and waiting for 'Resource 2', while 'Process B' is using 'Resource 2' and waiting for 'Resource 1'. It has been shown in the Figure 57.

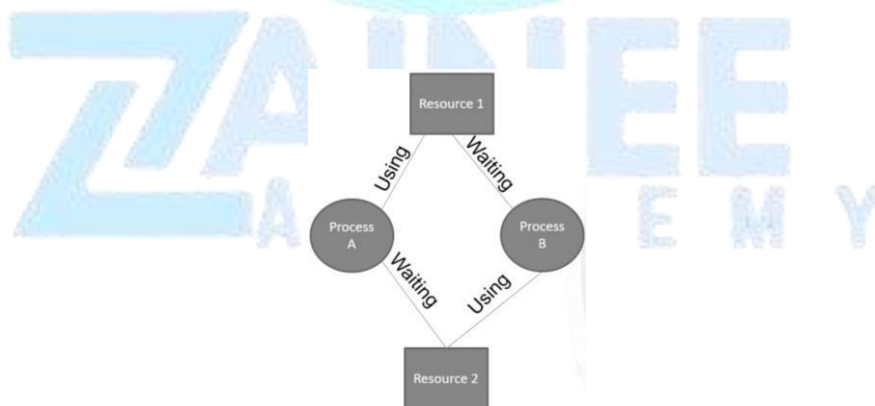


Figure 57: Resource Allocation Issues

Solutions to this problem will be discussed in next modules.

Module 55

55. Operating Systems: Semaphores

Let us consider a time-sharing/multitasking operating system controlling the activities of a computer with a single printer. If a process needs to print its results, it must request that the operating system give it access to the printer's device driver. At this point, the operating system must decide whether to grant this request, depending on whether the printer is already being used by another process. If it is not, the operating system should grant the request and allow the process to continue; otherwise, the operating system should deny the request and perhaps classify the process as a waiting process until the printer becomes available. After all, if two processes were given simultaneous access to the computer's printer, the results would be worthless to both.

To control access to the printer, **the operating system** must keep track of whether the printer has been allocated. One approach to this task would be to use a flag, which in this context refers to a bit in memory whose states are often referred **to as set and clear**, rather than 1 and 0. A clear flag (value 0) indicates that the printer **is available** and a set flag (value 1) indicates that the printer is **currently allocated**. On the surface, this approach seems well-founded. The operating system merely checks the flag each time a request for printer access is made. If it is clear, the request is granted, and the operating system sets the flag. If the flag is set, the operating system makes the requesting process wait. Each time a process finishes with the printer, the operating system either allocates the printer to a waiting process or, if no process is waiting, merely **clears the flag**.

However, this simple flag system has a problem. The task of testing and possibly setting the flag may require several machine instructions. (The value of the flag must be retrieved from main memory, manipulated within the CPU, and finally stored back in memory.) It is therefore possible for a task to be interrupted after a clear flag has been detected but before the flag has been set. In particular, suppose the printer is currently available, and a process requests use of it. The flag is retrieved from main memory and found to be clear, indicating that the printer is available. However, at this point, the process is interrupted, and another process begins its time slice. It too requests the use of the printer. Again, the flag is retrieved from main memory and found still clear because the previous process was interrupted before the operating system had time to set the flag in main memory. Consequently, the operating system allows the second process to begin using the printer. Later, the original process resumes execution where it left off, which is immediately after the operating system found the flag to be clear. Thus the operating system continues by setting the flag in main memory and granting the original process access to the printer. Two processes are now using the same printer.

The solution to this problem is to insist that the task of testing and possibly setting the flag be completed without interruption. One approach is to use the interrupt disable and interrupt enable instructions provided in most machine languages. When executed, an interrupt disable instruction causes future interrupts to be blocked, whereas an interrupt enable instruction causes the CPU to resume responding to interrupt signals. Thus, if the operating system starts the flag-testing routine with a disable interrupt instruction and ends it with an enable interrupt instruction, no other activity can interrupt the routine once it starts.

Another approach is to use the **test-and-set instruction** that is available in many machine languages. This instruction directs the CPU to retrieve the value of a flag, note the value received, and then set the flag—all within a single machine instruction. The advantage here is that because the CPU always completes an instruction before recognizing an interrupt, the task of testing and setting the flag cannot be split when it is implemented as a single instruction.

A properly implemented flag, as just described, is called a **semaphore**, in reference to the railroad signals used to control access to sections of track. In fact, semaphores are used in software systems in much the same way as they are in railway systems. Corresponding to the section of track that can contain only one train at a time is a sequence of instructions that should be executed by **only one process at a time**. Such a sequence of instructions is called **a critical region**. The requirement that only one process at a time be

allowed to execute a critical region is known as **mutual exclusion**. In summary, a common way of obtaining mutual exclusion to a critical region is to guard the critical region with a semaphore. To enter the critical region, a process must find the **semaphore clear** and then set the semaphore before entering the critical region; then upon exiting the critical region, the process must clear the semaphore. If the semaphore is found in its set state, the process trying to enter the critical region **must wait until** the semaphore has been cleared.

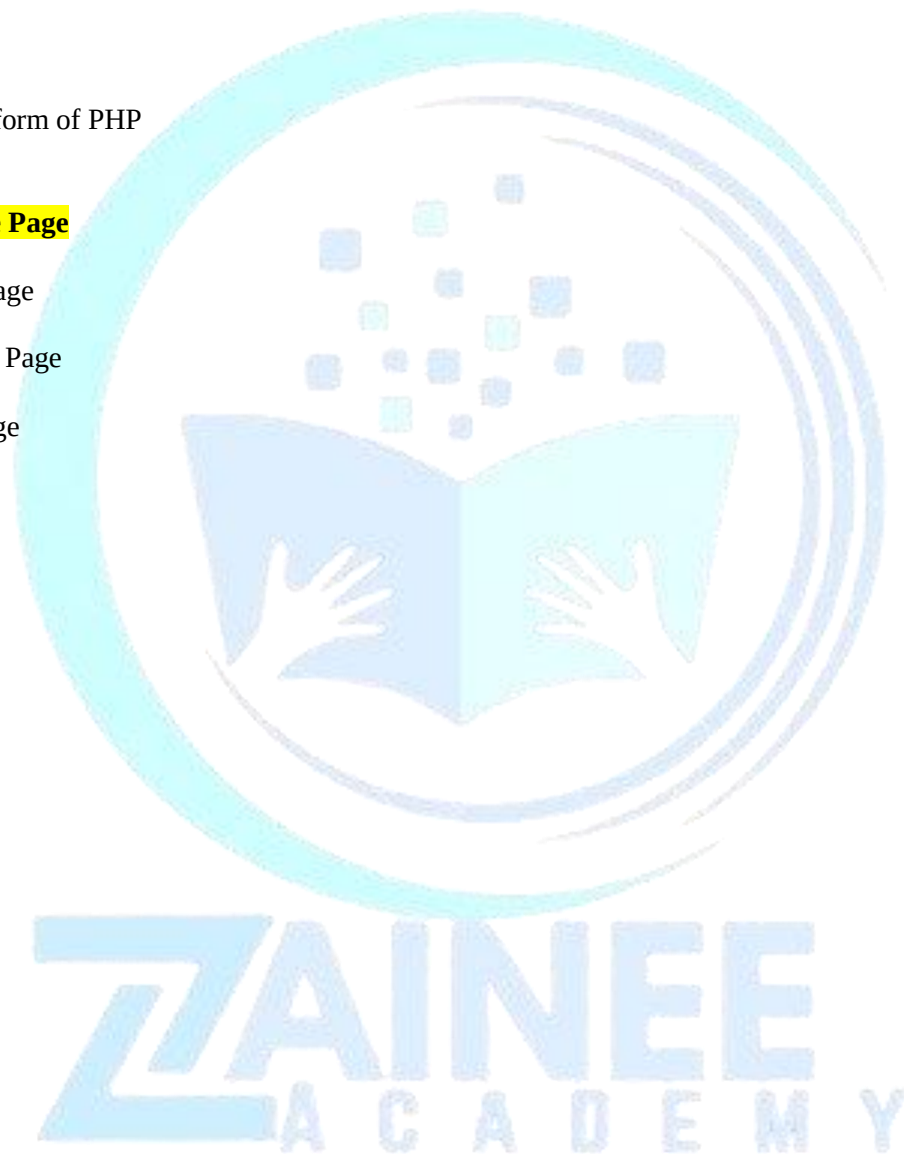
what is the full form of PHP
correct option

Personal Home Page

private Home Page

protected Home Page

Public home page



Module 56

56. Operating Systems: Deadlock

Another problem that can arise during resource allocation is **deadlock**, the conditions in which two or processes are blocked from progressing because each is waiting for a resource that is **allocated to another**. For example, one process may have access to the computer's printer but be waiting for access to the computer's CD player, while another process has access to the CD player but is waiting for the printer. Another example occurs in systems in which processes are allowed to create new processes (an action called **forking** in the UNIX vernacular) to perform subtasks. If the scheduler has no space left in the process table and each process in the system must create an additional process before it can complete its task, then no process can continue. Such conditions, as in other settings (Figure 58), can severely degrade a system's performance. Analysis of deadlock has revealed that it cannot occur unless all three of the following conditions are satisfied:

1. There is competition for non-sharable resources.
2. The resources are requested on a partial basis; that is, having received some resources, a process will return later to request more.
3. Once a resource has been allocated, it cannot be forcibly retrieved.

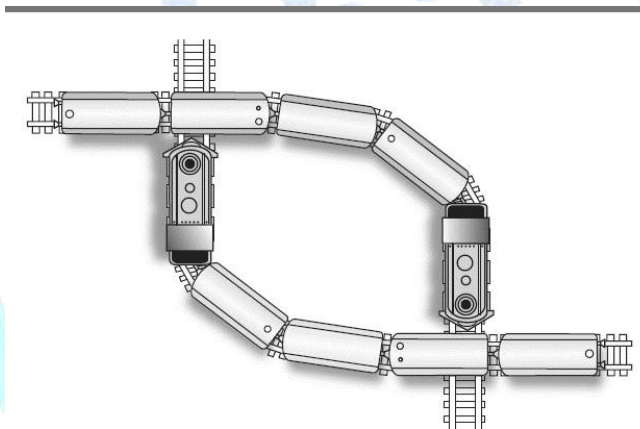


Figure 58: Deadlock Situation

The point of isolating these conditions is that the deadlock problem can be removed by attacking any one of the three. Techniques that attack the third condition fall into the category known as **deadlock detection** and correction schemes. In these cases, the occurrence of deadlock is considered so remote that no effort is made to avoid the problem. Instead, the approach is to detect it should it occur and then correct it by forcibly retrieving some of the allocated resources. Our example of a full process table might fall in this class. If deadlock should occur due to a full table, routines within the operating system (or perhaps a human administrator using his or her powers as “super user”) **can remove (the technical term is killing)** some of the processes. This releases space in the process table, breaking the deadlock and allowing the remaining processes to continue their tasks.

Techniques that attack the first two conditions are known as deadlock avoidance schemes. One, for example, attacks the second condition by requiring each process to **request all its resources** at one time. Another scheme attacks the first condition, not by removing the competition directly but by converting non-sharable resources into sharable ones. For example, suppose the resource in question is a printer and a variety of processes require its use. Each time a process requests the printer, the operating system could grant the request. However, instead of connecting the process to the printer's device driver, the operating system would connect it to a device driver that stores the information to be printed in mass storage rather than sending it to the printer. Thus each process, thinking it has access to the printer, could execute in its normal way. Later, when the printer is available, the operating system could transfer the data from mass

storage to the printer. In this manner, the operating system would make the non-sharable resource appear sharable by creating the illusion of more than one printer. This technique of holding data for output at a later but more convenient time is called **spooling**.



Module 57**57. Operating Systems: Security Attacks from outside**

An important task performed by operating systems is to protect the computer's resources from access by unauthorized personnel. In the case of computers used by multiple people, this is usually approached by means of establishing "accounts" for the various authorized users—an account being essentially a record within the operating system containing such entries as the user's name, password, and privileges to be granted to that user. The operating system can then use this information during each **login procedure** (a sequence of transactions in which the user **establishes initial contact** with a computer's operating system) to control access to the system.

MCQ

Accounts are established by a person known as the **super user or the administrator**. This person gains highly privileged access to the operating system by identifying him- or herself as the administrator (usually by name and password) during the login procedure. Once this contact is established, the administrator can alter settings within the operating system, modify critical software packages, adjust the privileges granted to other users, and perform a variety of other maintenance activities that are denied normal users.

From this "lofty perch," the administrator is also able to monitor activity within the computer system in an effort to detect destructive behavior, whether malicious or accidental. To assist in this regard, numerous software utilities, called **auditing software**, have been developed that record and then analyze the activities taking place within the computer system. In particular, **auditing software** may expose a flood of attempts to login using incorrect passwords, indicating that an unauthorized user may be trying to gain access to the computer.

Auditing software may also identify activities within a user's account that do not conform **to that user's past behavior**, which may indicate that an unauthorized user has gained access to that account. (It is unlikely that a user who traditionally uses only word processing and spreadsheet software will suddenly begin to access highly technical software applications or try to execute utility packages that lie outside that user's privileges.)

http. An old, well-known example is a program that simulates the operating system's login procedure. Such a program can be used to trick authorized users into thinking they are communicating with the operating system, whereas they are actually supplying their names and passwords to an impostor.

With all the technical complexities associated with computer security, it is surprising to many that one of the major obstacles to the security of computer systems is **the carelessness** of the users themselves. They select passwords that are relatively easy to guess (such as names and dates), they share their passwords with friends, they fail to change their passwords on a timely basis, they subject offline mass storage devices to potential degradation by transferring them back and forth between machines, and they import unapproved software into the system that might subvert the system's security. For problems like these, most institutions with large computer installations adopt and enforce policies that catalog the requirements and responsibilities of the users.

Module 58

58. Operating Systems: Security Attacks from inside

Once an intruder (or perhaps an authorized user with malicious intent) gains access to a computer system, the **next step is usually to explore, looking for information** of interest or for places to insert **destructive software**. This is a straight-forward process if the prowler has gained access to the administrator's account, which is why the administrator's password is closely guarded. If, however, access is through a **general user's account**, it becomes necessary to trick the operating system into allowing the intruder to reach beyond the privileges granted to that user. For example, the intruder may try to trick the memory manager into allowing a process to access main memory cells out Babbage's Analyticalide its allotted area, or the prowler may try to trick the file manager into retrieving files whose access should be denied.

Today's CPUs are enhanced with features that are designed to foil such attempts. As an example, consider the need to restrict a process to the area of main memory assigned to it by the memory manager. Without such restrictions, a process could erase the operating system from main memory and take control of the computer itself. To counter such attempts, CPUs designed for multiprogramming systems typically contain special-purpose registers in which the operating system **can store the upper and lower limits of a process's allotted memory area**. Then, while performing the process, **the CPU compares** each memory reference to these registers to ensure that the reference is within the designated limits. If the reference is found to be outside the process's designated area, the CPU automatically transfers control back to the operating system (by performing an interrupt sequence) so that the operating system can take appropriate action.

Embedded in this illustration is a subtle but significant problem. Without further security features, a process could still gain access to memory cells outside of its designated area merely by changing the special-purpose registers that **contain its memory limits**. That is, a process that wanted access to additional memory could merely increase the value in the register containing the upper memory limit and then proceed to use the additional memory space without approval from the operating system.

To protect against such actions, CPUs for multiprogramming systems are designed to operate in one of two **privilege levels**; we will call one "privileged mode," the other we will call "non-privileged mode." When in privileged mode, the CPU is able to execute all the instructions in its machine language. However, when in non-privileged mode, the list of acceptable instructions is limited. The instructions that are available only in privileged mode are called **privileged instructions**. (Typical examples of privileged instructions include instructions that change the contents of memory limit registers and instructions that change the current privilege mode of the CPU.) An attempt to execute a privileged instruction when the **CPU is in non-privileged mode** causes **an interrupt**. This interrupt converts the CPU to privileged mode and transfers control to **an interrupt handler** within the operating system.

When first turned on, the CPU is in privileged mode. Thus, when the operating system starts at the end of the boot process, all instructions are executable. However, each time the operating system allows a process to start a time slice, it switches the CPU to non-privileged mode by executing a "change privilege mode" instruction. In turn, the operating system will be notified if the process attempts to execute a privileged instruction, and thus the operating system will be in position to maintain the integrity of the computer system.

Privileged instructions and the control of privilege levels is the major tool available to operating systems for maintaining security. However, the use of these tools is a complex component of an operating system's design, and errors continue to be found in current systems. A single flaw in privilege level control can open the door to disaster from malicious programmers or from inadvertent programming errors. If a process is allowed to alter the timer that controls the system's multiprogramming system, that process can extend its time slice and dominate the machine. If a process is allowed to access peripheral devices directly, then it can read files without supervision by the system's file manager. If a process is allowed to access memory cells outside its allotted area, it can read and even alter data being used by other processes. Thus,

maintaining security continues to be an important task of an administrator as well as a goal in operating system design.



Module 59

59. Networking and the Internet: Network Classification

A computer network is often classified as being either a **personal area network (PAN)**, a **local area network (LAN)**, a **metropolitan area network (MAN)**, or a **wide area network (WAN)**. A PAN is normally used for short-range communications—typically less than **a few meters**—such as between a wireless headset and a smartphone or between a wireless mouse and its PC. In contrast, a LAN normally consists of a collection of computers in a single building or building complex. For example, the computers on a university campus or those in a manufacturing plant might be connected by a LAN. A MAN is a network of intermediate size, such as one spanning a local community. Finally, a WAN links machines over a greater distance—perhaps in neighboring cities or on opposite sides of the world.

Another means of classifying networks is based on whether the network's internal operation is based on designs that are in the public domain or on innovations owned and controlled by a particular entity such as an individual or a corporation. A network of the former type is called an **open network**; a network of the latter type is called a **closed**, or sometimes a **proprietary**, network. Open network designs are freely circulated and often grow in popularity to the point that they ultimately prevail over proprietary approaches whose applications are restricted by license fees and contract conditions.

The Internet (a popular worldwide network of networks that we will study in this chapter) is an **open system**. In particular, communication throughout the Internet is governed by an open collection of standards known **as the TCP/IP protocol suite**. Anyone is free to use these standards without paying fees or signing license agreements. In contrast, a company such as Novell Inc. might develop **proprietary systems** for which it chooses to maintain ownership rights, allowing the company to draw income from selling or leasing these products.

Still another way of classifying networks is based on the topology of the network, **which refers to the pattern in which the machines are connected**. Two of the more popular topologies are the bus, in which the machines are all connected to a common communication line called a bus (Figure 59a), and the star, in which one machine serves as a central focal point to which all the others are connected (Figure 59b). The bus topology was popularized in the 1990s. When it was implemented under a set of standards known as Ethernet, and Ethernet networks remain one of the most popular networking systems in use today.

The star topology has roots as far back as the 1970s. It evolved from the paradigm of a large central computer serving many users. As the simple terminals employed by these users grew into small computers themselves, a star network emerged. Today, **the star configuration** is popular in wireless networks where communication is conducted by means of radio broadcast and the central machine, called the **access point (AP)**, serves as a focal point around which all communication is coordinated.

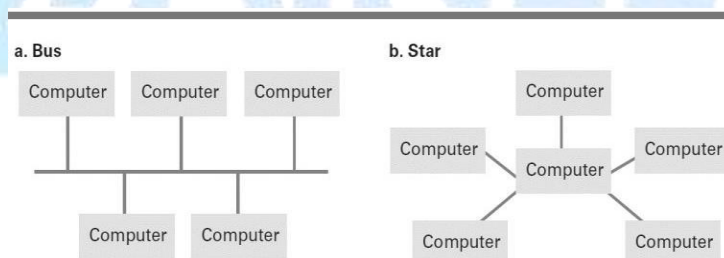


Figure 59: Two popular network topologies

Module 60

60. Networking and the Internet: Protocols

For a network to function reliably, it is important to establish rules by which activities are conducted. Such rules **are called protocols**. By developing and adopting protocol standards, **vendors are able to build products for network applications that are compatible with products from other vendors**. Thus, the development of protocol standards is an indispensable process in the development of networking technologies. As an introduction to the protocol concept, let us consider the problem of coordinating the transmission of messages among computers in a network. Without rules governing this communication, all the computers might insist on transmitting messages at the same time or fail to assist other machines when that assistance is required.

In a bus network based on the **Ethernet standards**, the right to transmit **messages is controlled by** the protocol known as **Carrier Sense, Multiple Access with Collision Detection (CSMA/CD)**. This protocol dictates that each **message be broadcast** to all the machines on the bus (Figure 60). Each machine monitors all the messages but keeps only those addressed to itself. To transmit a message, a machine waits until the bus is silent, and at this time it begins transmitting while continuing to monitor the bus. If another machine also begins transmitting, both machines detect the clash and pause for a brief, independently random period of time before trying to transmit again. The result is a system similar to that used by a small group of people in a conversation. If two people start to talk at once, they both stop. The difference is that people might go through a series such as, “I’m sorry, what were you going to say?”, “No, no. You go first,” whereas under the CSMA/CD protocol each machine merely tries again later.

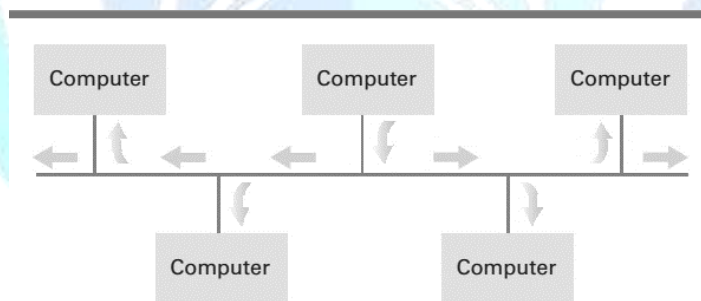


Figure 60: Communication over a bus network

Note that CSMA/CD is not compatible with wireless star networks in which all machines communicate through a central AP. This is because a machine may be unable to detect that its transmissions are colliding with those of another. For example, the machine may not hear the other because its own signal drowns out that of the other machine. Another cause might be that the signals from the different machines are blocked from each other by objects or distance even though they can all communicate with the central AP (a condition known as the **hidden terminal problem**, Figure 61). The result is that wireless networks adopt the policy of trying to avoid collisions rather than trying to detect them. Such policies are classified as **Carrier Sense, Multiple Access with Collision Avoidance (CSMA/CA)**, many of which are standardized by IEEE.

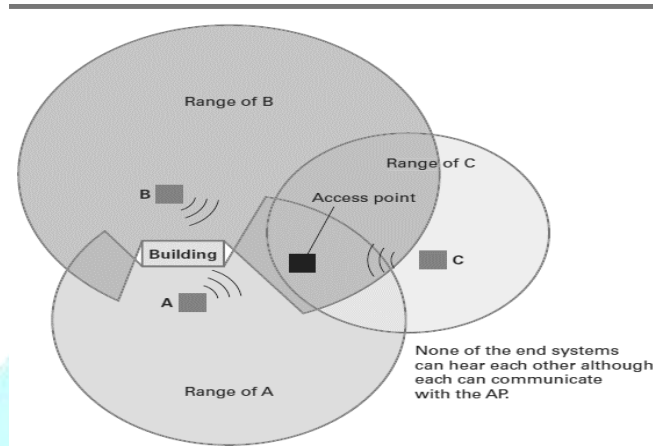


Figure 61: The hidden terminal problem

The most common approach to collision avoidance is based on giving advantage to machines that have already been waiting for an opportunity to transmit. The protocol used is similar to Ethernet's CSMA/CD. The basic difference is that when a machine first needs to transmit a message and finds the communication channel silent, it does not start transmitting immediately. Instead, it waits for a short period of time and then starts transmitting only if the channel has remained silent throughout that period. If a busy channel is experienced during this process, the machine waits for a randomly determined period before trying again. Once this period is exhausted, the machine is allowed to claim a silent channel without hesitation. This means that collisions between “newcomers” and those that have already been waiting are avoided because a “newcomer” is not allowed to claim a silent channel until any machine that has been waiting is given the opportunity to start.

This protocol, however, does not solve the hidden terminal problem. After all, any protocol based on distinguishing between a silent or busy channel requires that each individual station be able to hear all the others. To solve this problem, some WiFi networks require that each machine send a short “request” message to the AP and wait until the AP acknowledges that request before transmitting an entire message. If the AP is busy because it is dealing with a “hidden terminal,” it will ignore the request, and the requesting machine will know to wait. Otherwise, the AP will acknowledge the request, and the machine will know that it is safe to transmit. Note that all the machines in the network will hear all acknowledgments sent from the AP and thus have a good idea of whether the AP is busy at any given time, even though they may not be able to hear the transmissions taking place.

Module 61

61. Networking and the Internet: Combining Networks

Sometimes it is necessary to connect existing networks to form an extended communication system. This can be done by connecting the networks to form a larger version of the same “type” of network. For example, in the case of bus networks based on the Ethernet protocols, it is often possible to connect the buses to form a single long bus. This is done by means of different devices known as repeaters, bridges, and switches, the distinctions of which are subtle yet informative. The simplest of these is the **repeater**, which is little more than a device that passes signals back and forth between the two original buses (usually with some form of amplification) without considering the meaning of the signals (Figure 62a).

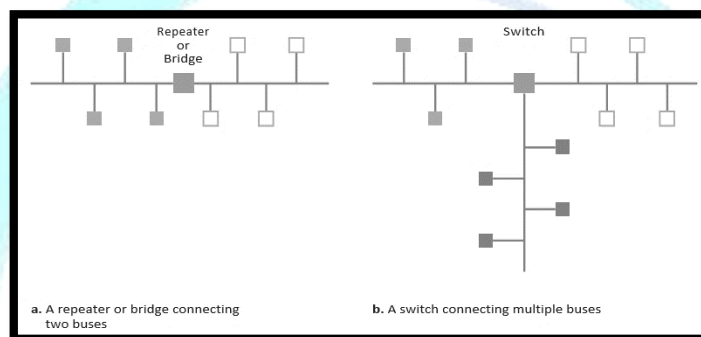


Figure 62: Building a large bus network from smaller ones

A **bridge** is similar to, but more complex than, a repeater. Like a repeater, it connects two buses, but it does not necessarily pass all messages across the connection. Instead, it looks at the destination address that accompanies each message and forwards a message across the connection only when that message is destined for a computer on the other side. Thus, two machines residing on the same side of a bridge can exchange messages without interfering with communication taking place on the other side. A bridge produces a more efficient system than that produced by a repeater.

A **switch is essentially a bridge with multiple connections, allowing it to connect several buses rather than just two**. Thus, a switch produces a network consisting of several buses extending from the switch as spokes on a wheel (Figure 62b). As in the case of a bridge, a switch considers the destination addresses of all messages and forwards only those messages destined for other spokes. Moreover, each message that is forwarded is relayed only into the appropriate spoke, thus minimizing the traffic in each spoke.

It is important to note that when networks are connected via repeaters, bridges, and switches, the result is a single large network. The entire system operates in the same manner (using the same protocols) as each of the original smaller networks.

Sometimes, however, the networks to be connected have incompatible characteristics. For instance, the characteristics of a WiFi network are not readily compatible with an Ethernet network. In these cases the networks must be connected in a manner that builds a **network of networks, known as an internet**, in which the original networks maintain their individuality and continue to function as autonomous networks. (Note that the generic term *internet* is distinct from *the Internet*. The Internet, written with an uppercase *I*, refers to a particular, world-wide internet that we will study in later sections of this chapter. There are many other examples of internets. Indeed, traditional telephone communication was handled by worldwide internet systems well before the Internet was popularized.) The connection between networks to form an internet is handled by devices **known as routers**, which are special purpose computers used for forwarding messages. Note that the task of a router is different from that of repeaters, bridges, and switches in that routers provide links between networks while allowing each network to maintain its unique internal characteristics. As an example, Figure 63 depicts two WiFi star networks and an Ethernet bus network connected by routers.

When a machine in one of the Wi-Fi networks wants to send a message to a machine in the Ethernet network, it first sends the message to the AP in its network. From there, the AP sends the message to its associated router, and this router forwards the message to the router at the Ethernet. There the message is given to a machine on the bus, and that machine then forwards the message to its final destination in the Ethernet.

The reason that routers are so named is that their purpose is to forward messages in their proper directions. This forwarding process is based on an internet- wide addressing system in which all the devices in an internet (including the machines in the original networks and the routers) are assigned unique addresses. (Thus, each machine in one of the original networks has two addresses: its original “local” address within its own network and its internet address.) A machine wanting to send a message to a machine in a distant network attaches the internet address of the destination to the message and directs the message to its local router. From there it is forwarded in the proper direction. For this forwarding purpose, each router maintains **a forwarding table** that contains the router’s knowledge about the direction in which messages should be sent depending on their destination addresses.

The “point” at which one network is linked to an internet is **often called a gateway** because it serves as a passageway between the network and the outside world. Gateways can be found in a variety of forms, and thus the term is used rather loosely. In many cases a network’s gateway is merely the router through which it communicates with the rest of the internet. In other cases, the term *gateway* may be used to refer to more than just a router. For example, in most residential Wi-Fi networks that are connected to the Internet, the term **gateway** refers collectively to both the network’s AP and the router connected to the AP because these two devices are normally packaged in a single unit.

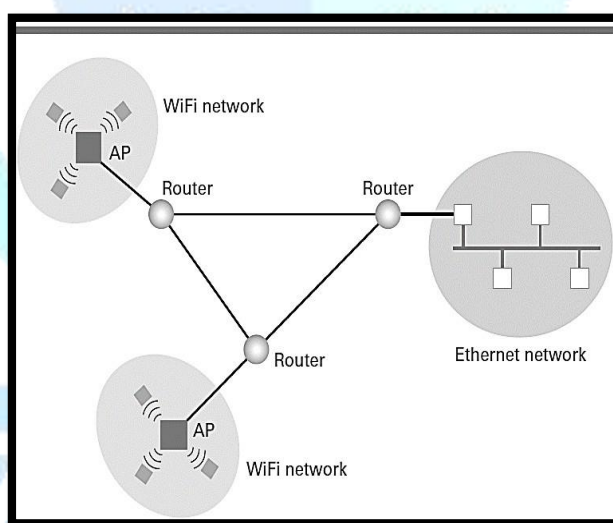


Figure 63: Routers connecting two WiFi networks and an Ethernet network to form an internet

Module 62

62. Networking and the Internet: Methods of Process Communication

The various activities (or processes) executing on the different computers within a network (or even executing on the same machine via time-sharing/multitasking) must often communicate with each other to coordinate their actions and to perform their designated tasks. Such communication between processes is **called inter-process communication**. A popular convention used for inter-process communication is the **client/**

server model. This model defines the basic roles played by the processes as either a **client**, which makes requests of other processes, or a **server**, which satisfies the requests made by clients.

An early application of the client/server model appeared in networks connecting all the computers in a cluster of offices. In this situation, a single, high-quality printer was attached to the network where it was available to all the machines in the network. In this case the printer played the role of a server (often called a **print server**), and the other machines were programmed to play the role of clients that sent print requests to the print server.

Another early application of the client/server model was used to reduce the cost of magnetic disk storage while also removing the need for duplicate copies of records. Here one machine in a network was equipped with a high-capacity mass storage system (usually a magnetic disk) that contained all of an organization's records. Other machines on the network then requested access to the records as they needed them. Thus the machine that actually contained the records played the role of a server (called a **file server**), and the other machines played the role of clients that requested access to the files that were stored at the file server.

Today the client/server model is used extensively in network applications, as we will see later in this chapter. However, the client/server model is not the only means of inter-process communication. Another model is the **peer-to-peer (often abbreviated P2P) model**. Whereas the client/server model involves one process (the server) providing a service to numerous others (clients), the peer-to-peer model involves processes that provide service to and receive service from each other (Figure 64). Moreover, whereas a server must execute continuously so that it is prepared to serve its clients at any time, the peer-to-peer model usually involves processes that execute on a temporary basis. For example, applications of the peer-to-peer model include instant messaging in which people carry on a written conversation over the Internet as well as situations in which people play competitive interactive games.

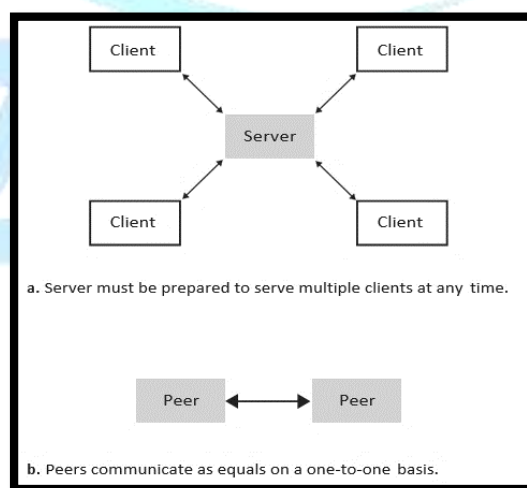


Figure 64: The client/server model compared to the peer-to-peer model

The peer-to-peer model is also a popular means of distributing files such as music recordings and motion pictures via the Internet. In this case, one peer may receive a file from another and then provide that file to other peers. The collection of peers participating in such a distribution is sometimes called a swarm. The swarm approach to file distribution is in contrast to earlier approaches that applied the client/server model by establishing a central distribution center (the server) from which clients downloaded files (or at least found sources for those files).

One reason that the P2P model is replacing the client/server model for file sharing is that it distributes the service task over many peers rather than concentrating it at one server. This lack of a centralized base of operation leads to a more efficient system. Unfortunately, another reason for the popularity of file distribution systems based on the P2P model is that, in cases of questionable legality, the lack of a central server makes legal efforts to enforce copyright laws more difficult. There are numerous cases, however, in which individuals have discovered that “difficult” does not mean “impossible” and have found themselves faced with significant liabilities due to copyright infringement violations. You might often read or hear the term *peer-to-peer network*, which is an exam-

ple of how misuse of terminology can evolve when technical terms are adopted by the nontechnical community. The term **peer-to-peer refers to** a system by which two processes communicate over a network (or internet). It is not a property of the network (or internet). A process might use the peer-to-peer model to communicate with another process and later use the client/server model to communicate with another process over the same network. Thus, it would be more accurate to speak of communicating by means of the peer-to-peer model rather than communicating over a peer-to-peer network.

ZAINEE
ACADEMY

Module 63

63. Networking and the Internet: Distributed Systems

With the success of networking technology, interaction between computers via networks has become common and multifaceted. Many modern software systems, such as global information retrieval systems, company-wide accounting and inventory systems, computer games, and even the software that controls a network's infrastructure itself are designed as **distributed systems**, meaning that they consist of software units that execute as processes on different computers. Early distributed systems were developed independently from scratch. But today, research is revealing a common infrastructure running throughout these systems, including such things as communication and security systems. In turn, efforts have been made to produce prefabricated systems that provide this basic infrastructure and therefore allow distributed applications to be constructed by merely developing the part of the system that is unique to the application.

Several types of distributed computing systems are now common. **Cluster computing** describes a distributed system in which many independent computers work closely together to provide computation or services comparable to a much larger machine. The cost of these individual machines, plus the high-speed network to connect them, can be less than a higher-priced supercomputer, but with higher reliability and lower maintenance costs. Such distributed systems are used to provide **high-availability**—because it is more likely that at least one member of the cluster will be able to answer a request, even if other cluster members break down or are unavailable—and **load-balancing**—because the workload can be shifted automatically from members of the cluster that have too much to do to those that may have too little. **Grid computing** refers to distributed systems that are more loosely coupled than clusters but that still work together to accomplish large tasks. Grid computing can involve specialized software to make it easier to distribute data and algorithms to the machines participating in a grid. Examples include University of Wisconsin's Condor system, or Berkeley's Open Infrastructure for Network Computing (BOINC). Both of these systems are often installed on computers that are used for other purposes, such as PCs at work or at home, that can then volunteer computing power to the grid when the machine is not otherwise being used. Enabled by the growing connectivity of the Internet, this type of voluntary, distributed grid computing has enabled millions of home PCs to work on enormously complex mathematical and scientific problems. **Cloud computing**, whereby huge pools of shared computers on the network can be allocated for use by clients as needed, is the latest trend in distributed systems. Much as the spread of metropolitan electrical grids in the early twentieth century eliminated the need for individual factories and businesses to maintain their own generators, the Internet is making it possible for entities to entrust their data and computations to "the Cloud," which in this case refers to the enormous computing resources already available on the network. Services such as Amazon's Elastic Compute Cloud allow clients to rent virtual computers by the hour, without concern for where the computer hardware is actually located. Google Drive and Google Apps allow users to collaborate on information or build Web services without needing to know how many computers are working on the problem or where the relevant data are stored. **Cloud computing** services provide reasonable guarantees of **reliability and scalability**, but also raise concerns about **privacy and security** in a world where we may no longer know who owns and operates the computers that we use.

Module 64

64. Networking and the Internet: Internet Architecture

As we have already mentioned, the Internet is a collection of connected networks. In general, these networks are constructed and maintained by organizations called **Internet Service Providers (ISPs)**. It is also customary to use the term ISP in reference to the networks themselves. Thus, we will speak of connecting to an ISP, when what we really mean is connecting to the network provided by an ISP.

The system of networks operated by the ISPs can be classified in a hierarchy according to the role they play in the overall Internet structure (Figure 65). At the top of this hierarchy are relatively few **tier-1 ISPs** that consist of very high-speed, high-capacity, international WANs. These networks are thought of as the backbone of the Internet. They are typically operated by large companies that are in the communications business. An example would be a company that originated as a traditional telephone company and has expanded its scope into providing other communication services.

Connecting to the tier-1 ISPs are the **tier-2 ISPs** that tend to be more regional in scope and less potent in their capabilities. (The distinction between the tier-1 and tier-2 ISPs is often a matter of opinion.) Again, these networks tend to be operated by companies in the communications business.

Tier-1 and tier-2 ISPs are essentially networks of routers that collectively provide the Internet's communication infrastructure. As such, they can be thought of as the core of the Internet. Access to this core is usually provided by an intermediary called an **access** or **tier-3 ISP**. An access ISP is essentially an independent internet, sometimes called an **intranet**, operated by a single authority that is in the business of supplying Internet access to individual homes and businesses. Examples include cable and telephone companies that charge for their service as well as organizations such as universities or corporations that take it upon themselves to provide Internet access to individuals within their organizations.

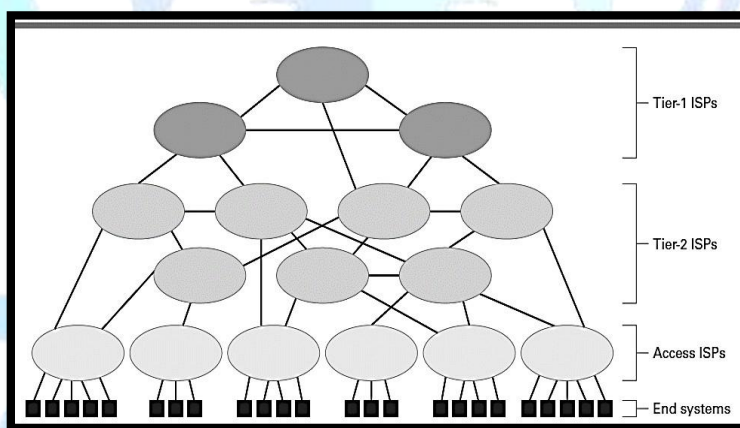


Figure 65: Internet composition

The devices that individual users connect to the access ISPs are known as **end systems** or **hosts**. These end systems may be laptops or PCs, but increasingly range over a multitude of other devices including telephones, video cameras, automobiles, and home appliances. After all, the Internet is essentially a communications system, and thus any device that would benefit from communicating with other devices is a potential end system.

The technology by which end systems connect to larger networks is also varied. Perhaps the fastest growing are wireless connections based on WiFi technology. The strategy is to connect the AP to an access ISP and thus provide Internet access through that ISP to end systems within the AP's broadcast range. The area within the AP or group of APs' range is often called a **hot spot**, particularly when the network access is publicly available or free. Hot spots can be found in individual residences, hotel and office buildings, small

businesses, parks, and in some cases span entire cities. A similar technology is used by the cellular telephone industry where hot spots are known as cells and the “routers” generating the cells are coordinated to provide continuous service as an end system moves from one cell to another.



Module 65

65. Networking and the Internet: Internet Addressing

Internet needs an internet-wide addressing system that assigns a unique identifying address to each computer in the system. In the Internet these addresses are known as **IP addresses**. Originally, each IP address was a pattern of 32 bits, but to provide a larger set of addresses, the process of converting to 128-bit addresses is currently underway. Blocks of consecutively numbered IP addresses are awarded to ISPs by the **Internet Corporation for Assigned Names and Numbers (ICANN)**, which is a nonprofit corporation established to coordinate the Internet's operation. The ISPs are then allowed to allocate the addresses within their awarded blocks to machines within their region of authority. Thus, machines throughout the Internet are assigned unique IP addresses.

IP addresses are traditionally written in **dotted decimal notation** in which the bytes of the address are separated by periods and each byte is expressed as an integer represented in traditional base 10 notation. For example, using dotted decimal notation, the pattern 5.2 would represent the two-byte bit pattern 0000010100000010, which consists of the byte 00000101 (represented by 5) followed by the byte 00000010 (represented by 2), and the pattern 17.12.25 would represent the three-byte bit pattern consisting of the byte 00010001 (which is 17 written in binary notation), followed by the byte 00001100 (12 written in binary), followed by the byte 00011001 (25 written in binary). In summary, a 32-bit IP address might appear as 192.207.177.133 when expressed in dotted decimal notation.

Addresses in bit-pattern form (even when compressed using dotted decimal notation) are rarely conducive to human consumption. For this reason, the Internet has an alternative addressing system in which machines are identified by mnemonic names. This addressing system is based on the concept of a **domain**, which can be thought of as a “region” of the Internet operated by a single authority such as a university, club, company, or government agency. (The word region is in quotations here because, as we will soon see, such a region may not correspond to a physical area of the Internet.) Each domain must be registered with ICANN—a process handled by companies, called **registrars**, that have been assigned this role by ICANN. As a part of this registration process, the domain is assigned a mnemonic **domain name**, which is unique among all the domain names throughout the Internet. Domain names are often descriptive of the organization registering the domain, which enhances their utility for humans.

As an example, the domain name of Marquette University is mu.edu. Note the suffix following the period. It is used to reflect the domain's classification, which in this case is “educational” as indicated by the edu suffix. These suffixes are called **top-level domains (TLDs)**. Other TLDs include com for commercial institutions, gov for U.S. government institutions, org for nonprofit organization, museum for museums, info for unrestricted use, and net, which was originally intended for ISPs but is now used on a much broader scale. In addition to these general TLDs, there are also two-letter TLDs for specific countries (called **country-code TLDs**) such as au for Australia and ca for Canada.

Once a domain's mnemonic name is registered, the organization that registered the name is free to extend the name to obtain mnemonic identifiers for individual items within the domain. For example, an individual host within Marquette University may be identified as eagle.mu.edu. Note that domain names are extended to the left and separated by a period. In some cases multiple extensions, called **subdomains**, are used as a means of organizing the names within a domain. These subdomains often represent different networks within the domain's jurisdiction. For example, if Yoyodyne Corporation was assigned the domain name yoyodyne.com, then an individual computer at Yoyodyne might have a name such as overthrust.propulsion.yoyodyne.com, meaning that the computer overthrust is in the subdomain propulsion within the domain yoyodyne within the TLD com. (We should emphasize that the dotted notation used in mnemonic addresses is not related to the dotted decimal notation used to represent addresses in bit pattern form.)

Although mnemonic addresses are convenient for humans, messages are always transferred over the Internet by means of IP addresses. Thus, if a human wants to send a message to a distant machine and

identifies the destination by means of a mnemonic address, the software being used must be able to convert that address into an IP address before transmitting the message. This conversion is performed with the aid of numerous servers, called **name servers**, that are essentially directories that provide address translation services to clients. Collectively, these name servers are used as an Internet-wide directory system known as the **domain name system (DNS)**. The process of using DNS to perform a translation is called a **DNS lookup**.

Thus, for a machine to be accessible by means of a mnemonic domain name, that name must be represented in a name server within the DNS. In those cases, in which the entity establishing the domain has the resources, it can establish and maintain its own name server containing all the names within that domain. Indeed, this is the model on which the domain system was originally based. Each registered domain represented a physical region of the Internet that was operated by a local authority such as a company, university, or government agency. This authority was essentially an access ISP that provided Internet access to its members by means of its own intranet that was linked to the Internet. As part of this system, the organization maintained its own name server that provided translation services for all the names used within its domain.

This model is still common today. However, many individuals or small organizations want to establish a domain presence on the Internet without committing the resources necessary to support it. For example, it might be beneficial for a local chess club to have a presence on the Internet as KingsandQueens.org, but the club would likely not have the resources to establish its own network, maintain a link from this network to the Internet, and implement its own name server. In this case, the club can contract with an access ISP to create the appearance of a registered domain using the resources already established by the ISP. Typically, the club, perhaps with the assistance of the ISP, registers the name chosen by the club and contracts with the ISP to have that name included in the ISP's name server. This means that all DNS lookups regarding the new domain name will be directed to the ISP's name server, from which the proper translation will be obtained. In this way, many registered domains can reside within a single ISP, each often occupying only a small portion of a single computer.

ZAINEE
ACADEMY

Module 66**66. Networking and the Internet: Internet Applications**

In the earlier days of the Internet, most applications were separate, simple programs that each followed a network protocol. A newsreader application contacted servers using the **Network News Transfer Protocol (NNTP)**, an application for listing **and copying files across** the network implemented the **File Transfer Protocol (FTP)**, or an application for accessing another computer from a great distance used the **Telnet** protocol, or later the **Secure Shell (SSH)** protocol. As web servers and browsers have become more sophisticated, more and more of these traditional network applications have come to be handled by web pages via the powerful **Hyper Text Transfer Protocol (HTTP)**.

MCQ

**What is the primary role of HTTP in the context of handling traditional network applications?
Select the correct option**

Making applications obsolete

Providing security features

Enabling sophisticated web servers and browsers.

Enhancing file transfer protocols

Electronic Mail A wide variety of systems now exist for exchanging messages between end users over the network; instant messaging (IM), browser-based online chatting, Twitter-based “tweets”, and the Facebook “wall” are but a few. One of the **oldest and most enduring** uses of the Internet is the electronic mail system, or email for short.

VoIP as an example of **a more recent** Internet application, consider **VoIP (Voice over Internet Protocol)** in which the Internet infrastructure is used to provide voice communication similar to that of traditional telephone systems. In its simplest form, VoIP consists of two processes on different machines transferring audio data via the **P2P model**—a process that in itself presents no significant problems. However, tasks such as initiating and receiving calls, linking VoIP with traditional telephone systems, and providing services such as emergency 911 communications are issues that extend beyond traditional Internet applications. Moreover, governments that own their country’s traditional telephone companies view VoIP as a threat and have either taxed it heavily or outlawed it completely.

Internet Multimedia Streaming an enormous portion of current Internet traffic is used for transporting audio and video across the Internet in real-time, known as streaming. **Netflix streamed more than 4 billion hours of programming to end users in the first three months of 2013 alone.** Combined with **YouTube**, these two services will consume more than half of the bandwidth of the Internet in 2014.

Module 67**67. Networking and the Internet: Internet Applications: Email****Messaging applications**

- ✓ Instant Messaging
- ✓ Browser based chatting
- ✓ Twitter based tweets
- ✓ Facebook wall.
- ✓ One of the oldest is Electronic mail (Email)

Protocol

- ✓ Simple Mail Transfer Protocol (SMTP)

Scenario

- ✓ mafzal from cust.edu.pk wants to send email to hmaurer from iicm.tugraz.at, the flow has been shown in the Figure 66.

```

1 220 mail.iicm.tugraz.at
SMTP Sendmail Gallifrey-1.0; 9 354 Enter mail, end
Sun, 12                      with "." on a line by
Aug 2018 14:34:10           itself
2 HELO mail.cust.edu.pk      10 Subject:
3 250 mail.iicm.tugraz.at    Extermination.
Hello mail.cust.edu.pk,      11
pleased to meet you          12 EXTERMINATE!
4 MAIL From:                 13 Regards, Dr. M.
mafzal@cust.edu.pk           Tanvir Afzal
5 250 2.1.0                  14 .
mafzal@cust.edu.pk... Sender 15 250 2.0.0
ok                             r7NJYAE1028071 Message
6 RCPT To:                   accepted for delivery
hmaurer@iicm.tugraz.at       16 QUIT
7 250 2.1.5                  17 221 2.0.0
hmaurer@iicm.tugraz.at...    mail.iicm.tugraz.at
Recipient ok                  closing connection
8 DATA

```

*Figure 66: Email procedure***Other Protocols**

- ✓ **SMTP for text messages.**
- ✓ MIME (Multipurpose Internet Mail Extensions) to convert non-ASCII to SMTP compatible form.
- ✓ Post Office Protocol Version 3 (POP3)
- ✓ **Internet Mail Access Protocol (IMAP)**
- ✓ User can download and maintain mail messages into folders etc, POP3 helps to store messages on local machine, **IMAP on mail server machine.**

Module 68**68. Networking and the Internet: VoIP**

As an example of a more recent Internet application, consider **VoIP (Voice over Internet Protocol)** in which the Internet infrastructure is used to provide voice communication similar to that of traditional telephone systems. In its simplest form, VoIP consists of two processes on different machines transferring audio data **via the P2P model**—a process that in itself presents no significant problems. However, tasks such as initiating and receiving calls, linking VoIP with traditional telephone systems, and providing services such as emergency 911 communications are issues that extend beyond traditional Internet applications. Moreover, governments that own **their country's traditional telephone companies** view VoIP as a threat and have either taxed it heavily or outlawed it completely. Existing VoIP systems come in four different forms that are competing for popularity. **VoIP soft phones** consist of P2P software that allows two or more PCs to share a call with no more special hardware than a speaker and a microphone. An example of a VoIP soft phone system is Skype, which also provides its clients with links to the traditional telephone communication system. One drawback to Skype is that it is a proprietary system, and thus much of its operational structure is not publicly known. This means that Skype users must trust the integrity of the Skype software without third-party verification. For instance, to receive calls, a Skype user must leave his or her PC connected to the Internet and available to the Skype system, which means that some of the PC's resources may be used to support other Skype communications without the PC owner's awareness—a feature that has generated some resistance.

MCQ

Which of the following is one of the major drawbacks of Skype?
Select the correct option

It uses computer resources without owner's awareness

Free version is not easily available

It takes a huge space on local drive

Call quality is not good

A second form of VoIP consists of **analog telephone adapters**, which are devices that allow a user to connect his or her traditional telephone to phone service provided by an access ISP. This choice is frequently bundled with traditional Internet service and/or digital television service.

The third type of VoIP comes in the form of embedded VoIP phones, which are devices that replace a traditional telephone with an equivalent handset connected directly to a TCP/IP network. Embedded VoIP phones are becoming increasingly common for large organizations, many of whom are replacing their traditional internal copper wire telephone systems with VoIP over Ethernet to reduce costs and enhance features.

Finally, the current generation of smartphones use wireless VoIP technology. That is, earlier generations of wireless phones only communicated with the telephone company's network using that company's protocols. Access to the Internet was obtained by gateways between the company's network and the Internet, at which point signals were converted to the TCP/IP system. However, the 4G phone network is **an IP-based network** throughout, which means a 4G telephone is essentially just another broadband-connected host computer on the global Internet.

Quiz

Which of the following protocol is not associated with the transfer of email messages?

Select the correct option

IMAP

POP3

SMTP

MIME



Module 69**69. Networking and the Internet: Internet Multimedia Streaming**

An enormous portion of current Internet traffic is used for transporting audio and video across the Internet in real-time, known **as streaming**. Netflix streamed more than 4 billion hours of programming to end users in the first three months of 2013 alone. Combined with YouTube, these two services will consume more than half of the bandwidth of the Internet in 2014.

On the surface, Internet streaming may not seem to require special consideration. For example, one might guess that an Internet radio station could merely establish a server that would send program messages to each of the clients who requested them. This technique is known **as N-unicast**. (More precisely, unicast refers to one sender sending messages to one receiver, **whereas N-unicast refers to a single sender involved with multiple unicasts**.) The N-unicast approach has been applied but has the drawback of placing a substantial burden on the station's server as well as on the server's immediate Internet neighbors. Indeed, N-unicast forces the server to send individual messages to each of its clients on a real-time basis, and all these messages must be forwarded by the server's neighbors.

Most alternatives to N-unicast represent attempts to alleviate this problem. One applies the P2P model in a manner reminiscent of file-sharing systems. That is, once a peer has received data, it begins to distribute that data to those peers that are still waiting, meaning that much of the distribution problem is transferred from the data's source to the peers.

Another alternative, called **multicast**, transfers the distribution problem to the internet routers. Using multicast, a server transmits a message to multiple clients by means of a single address and relies on the routers in the Internet to recognize the significance of that address and to produce and forward copies of the message to the appropriate destinations. Note then that applications relying on multicast require that the functionality of the Internet routers be expanded beyond their original duties. Multicast support has been implemented in small networks, but has yet to expand to the global Internet.

More importantly, most applications in this category are now **on-demand streaming**, in which the end user expects to view or listen to media at an arbitrary time of his or her choosing. This is quite a different problem from the Internet radio station example, because each end user expects to be able to start, pause, or rewind content at his or her own pace. In this case, N-unicast and multicast technologies are of little help. Each on-demand stream is effectively **unicast** from a media server that stores the content to the end user that wishes to retrieve it.

In order for this type of streaming to scale to thousands or even millions of simultaneous users, each with his or her own personal stream, replication of the content to many distinct servers is essential. Large-scale streaming services make use of **content delivery networks (CDNs)**, groups of servers distributed strategically around the Internet that specialize in streaming copies of content to nearby end users in their network "neighborhood." In many cases, CDN machines may reside in an access ISP network, allowing customers of that access ISP to stream copies of multimedia content at high speed from a nearby server that is much closer in the network than the streaming service's central server machines. A networking technology called **anycast**, which enables an end user to automatically connect to the **closest server** out of a defined group of servers, helps to make CDNs practical.

Internet streaming of high-definition, on-demand video has permeated far more than traditional PCs. A broad class of embedded devices such as televisions, DVD/Blu-ray players, smartphones, and game consoles connect directly to the TCP/IP network to select viewable content from a multitude of both free and subscription servers.

Module 70**70. Networking and the Internet: World Wide Web**

The World Wide Web had its origins in the work of Tim Berners-Lee who realized the potential of combining internet technology with the concept of linked- documents, **called hypertext**. His first software for implementing the Web was released in **December 1990**. While this early prototype did not yet support multimedia data, it included the key components of what we now recognize as the World Wide Web: a hypertext document format for embedding **hyperlinks to other documents**; a protocol for **transferring hypertext across the network**, and a **server process** that supplied hypertext pages upon request. From this humble beginning, the Web quickly grew to support images, audio and video, and by the mid-1990s had become the dominant application powering the growth of the Internet.

MCQ

Talent is used to establish a **remote connection** between two systems.

How does the multicast differ from N-unicast in handling data distribution?

Select the correct option

It involves direct communication between peers

It involves one sender and one receiver

It transfers the distribution problem to Internet routers

It relies on server-to-client real-time messages

<body>: The **<body>** tag contains the main content of the web page that is displayed to the user.

<html>: The **<html> tag** is the root element of the HTML document and does not contain any metadata.

<p>: **The <p> tag** is used to create a paragraph of text.

The **<head> tag** contains information about the web page that is not usually displayed to the user. This information is called metadata.

Module 71**71. Networking and the Internet: Web implementations**

Software packages that allow users to access hypertext on the Internet fall into one of two categories: browsers and web servers. **A browser resides on the user's computer and is charged with the tasks of obtaining materials**

requested by the user and presenting these materials to the user in an organized manner. Common Internet browsers include Firefox, Safari, and Internet Explorer. **The webserver** resides on a computer containing hypertext documents to be accessed. Its task is to provide access to the documents under its control as requested by clients (browsers). **Hypertext documents are normally transferred between browsers and web servers using a protocol known as the Hypertext Transfer Protocol (HTTP).** In order to locate and retrieve documents on the Web, each document is given a unique address called **a Uniform Resource Locator (URL).** **Each URL contains the information needed by a browser to contact the proper server** and request the desired document. Thus, to view a webpage, a person first provides his or her browser with the URL of the desired document and then instructs the browser to retrieve and display the document.

A typical URL is presented in Figure 67. It consists of four segments: the protocol to use to communicate with the server controlling access to the document, the mnemonic address of the machine containing the server, the **directory path** needed for the server to find the directory containing the document, and the name of the document itself. In short, the URL in Figure 67 tells a browser to contact the webserver on the computer known as eagle.mu.edu using the protocol HTTP and to retrieve the document named Julius_Caesar.html found within the subdirectory Shakespeare within the directory called authors.

Sometimes a URL might not explicitly contain all the segments shown in Figure 67. For example, if the server does not need to follow a directory path to reach the document, no directory path will appear in the URL. Moreover, sometimes a URL will consist of only a protocol and the mnemonic address of a computer. In these cases, the webserver at that computer will return a predetermined document, typically called a home page, that usually describes the information available at that website. Such shortened URLs provide a simple means of contacting organizations. For example, the URL <http://www.google.com> will lead to the home page of Google, which contains hyperlinks to the services, products, and documents relating to the company.

To further simplify locating websites, many browsers assume that the HTTP protocol should be used if no protocol is identified. These browsers correctly retrieve the Google home page when given the "URL" consisting merely of www.google.com.

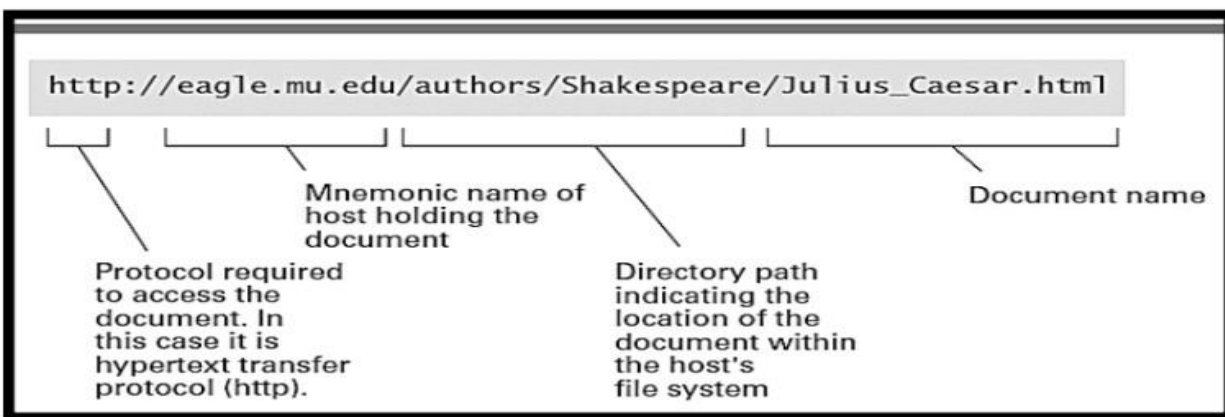


Figure 67: A typical URL

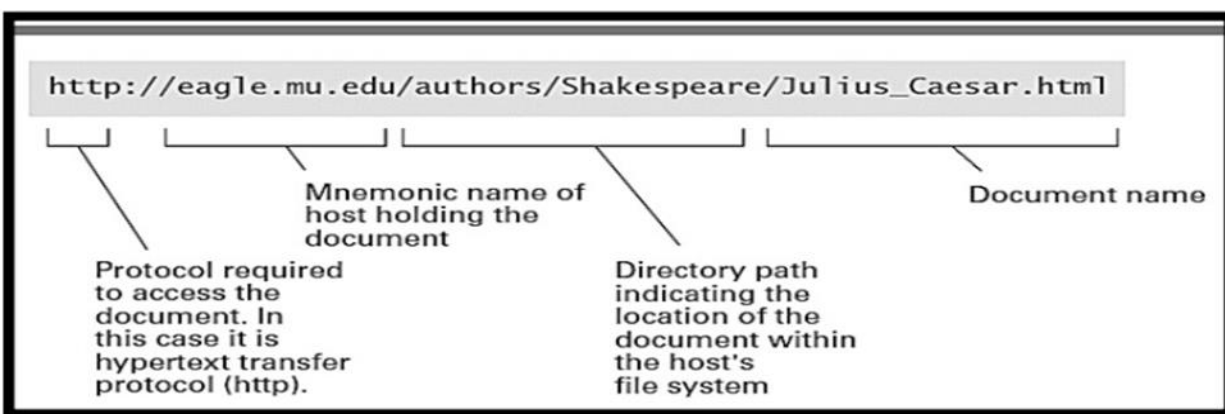


Figure 67: A typical URL

ZAINEE
ACADEMY

Module 72**. Networking and the Internet: HTML**

A traditional hypertext document is similar to a text file because its text is encoded character by character using a system such as ASCII or Unicode. The distinction is that a hypertext document also contains special symbols, called **tags**, that describe how the document should appear on a display screen, what multimedia resources (such as images) should accompany the document, and which items within the document are linked to other documents. This system of tags is known **as Hypertext Markup Language (HTML).**

Thus, it is in terms of HTML that an author of a webpage describes the information that a browser needs in order to present the page on the user's screen and to find any related documents referenced by the current page. The process is analogous to adding typesetting directions to a plain typed text (perhaps using a red pen) so that a typesetter will know how the material should appear in its final form. In the case of hypertext, the red markings are replaced by HTML tags, and a browser ultimately plays the role of the typesetter, reading the HTML tags to learn how the text is to be presented on the computer screen.

MCQ

What is the key distinction between a traditional hypertext document and a normal text file written in a text editor?

Select the correct option

Hypertext documents are longer than text files

Text files use ASCII or Unicode, while hypertext documents use tags

Text files contain multimedia resources, while hypertext documents do not

Hypertext documents are written in XML language.

Module 73**73. Networking and the Internet: HTML**

A traditional hypertext document is similar to a text file because its text is the HTML-encoded version (called the source version) of an extremely simple webpage is shown in Figure 68a. Note that the tags are delineated by the symbols < and >. The HTML source document consists **of two sections a head (surrounded by the <head> and </head> tags) and a body (surrounded by the <body> and </body> tags)**. The distinction between the head and body of a web- page is similar to that of the head and body of an interoffice memo. In both cases, the head contains preliminary information about the document (date, subject, etc. in the case of a memo). The body contains the meat of the document, which in the case of a webpage is the material to be presented on the computer screen when the page is displayed.

MCQ

tag contains the information related to metadata that is not usually displayed on the webpage

Select the correct option

body

html

head

p

- The <body> tag is the container for the content that is displayed on the webpage.
- The <html> tag is the root element of an HTML document.
- The <p> tag is used to define a paragraph of text.

The head of the webpage displayed in Figure 68 contains only the title of the document (surrounded by "title" tags). This title is only for documentation purposes; it is not part of the page that is to be displayed on the computer screen. The material that is displayed **on the screen** is contained in the **body** of the document. The first entry in the body of the document in Figure 68a is a level-one heading (surrounded by the <h1> and </h1> tags) containing the text "My Web Page." Being a level-one heading means that the browser should display this text prominently on the screen. The next entry in the body is a paragraph of text (surrounded by the <p> and </p> tags) containing the text "Click here for another page." Figure 68b shows the page as it would be presented on a computer screen by a browser.

In its present form, the page in Figure 68 is not fully functional in the sense that nothing will happen when the viewer clicks on the word here,

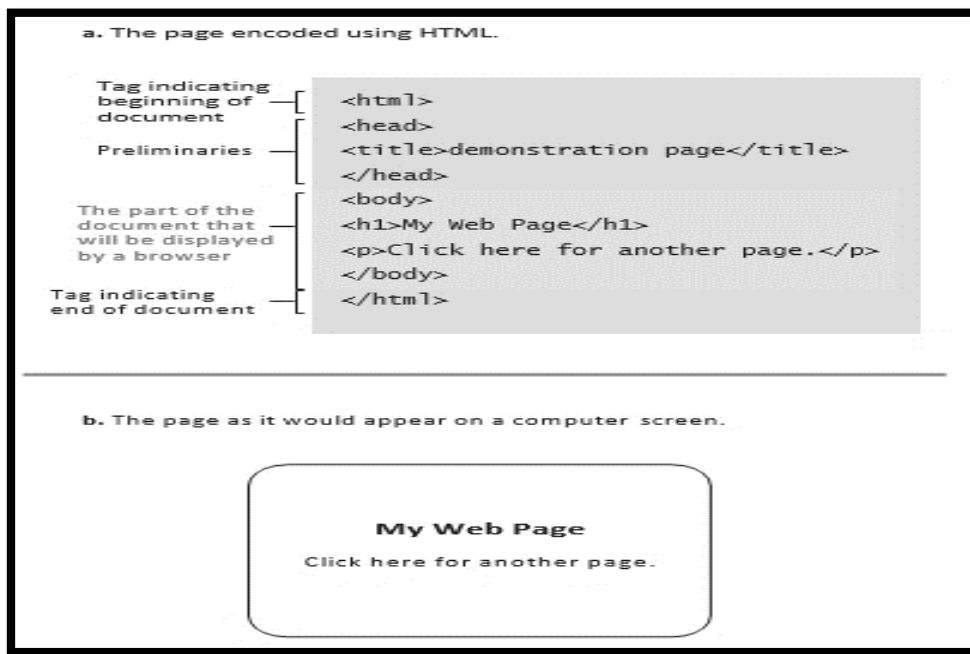


Figure 68: A simple webpage

even though the page implies that doing so will cause the browser to display another page. To cause the appropriate action, we must link the word *here* to another document.

Let us suppose that, when the word *here* is clicked, we want the browser to retrieve and display the page at the URL <http://crafty.com/demo.html>. To do so, we must first surround the word *here* in the source version of the page with the tags `<a>` and `</a>`, which are **called anchor tags**. Inside the opening anchor tag we insert the parameter

`href = http://crafty.com/demo.html`

(as shown in Figure 69a) indicating that the **hypertext reference (href)** associated with the tag is the URL following the equal sign (<http://crafty.com/demo.html>). Having added the anchor tags, the webpage will now appear on a computer screen as shown in Figure 4.10b. Note that this is identical to Figure 68b except that the word *here* is highlighted by color indicating that it is a link to another webpage. Clicking on such highlighted terms will cause the browser to retrieve and display the associated webpage. Thus, it is by means of anchor tags that webpages are linked to each other.

Finally, we should indicate how an image could be included in our simple webpage. For this purpose, let us suppose that a JPEG encoding of the image we want to include is stored as the file named `OurPic.jpg` in the directory `Images` at `Images.com` and is available via the webserver at that location. Under these conditions, we can tell a browser to display the image at the top of the webpage by inserting the image tag `<img src = "http://Images.com/Images/OurPic.jpg">` immediately after the `<body>` tag in the HTML source document. This tells the browser that the image named `OurPic.jpg` should be displayed at the beginning of the document. (**The term `src` is short for “source,”** meaning that the information following the equal sign indicates the source of the image to be displayed.) When the browser finds this tag, it will send a message to the HTTP server at `Images.com` requesting the image called `OurPic.jpg` and then display the image appropriately.

If we moved the image tag to the end of the document just before the `</body>` tag, then the browser would display the image at the bottom of the webpage. There are, of course, more sophisticated techniques for positioning an image on a webpage, but these need not concern us now.

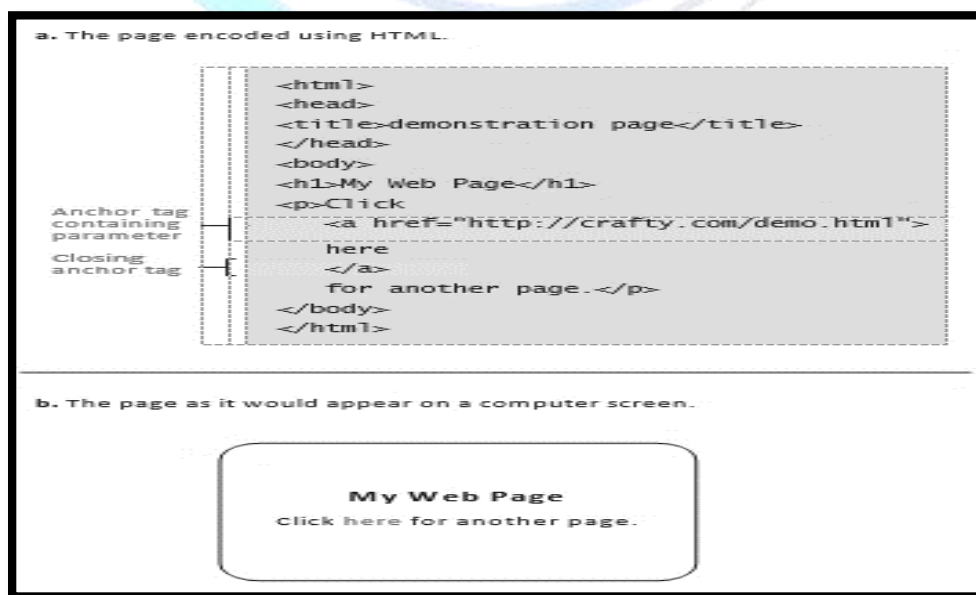


Figure 69: An enhanced simple webpage

Module 74

74. Networking and the Internet: More on HTML

If you are further interested to develop HTML pages, here are detailed content available for you:

W3schools

<https://www.w3schools.com/tags/default.asp>

Here you can get information about all HTML tags and how they can be used. For example, if you click on the tag “” available on the left panel on the above URL, you will be shown the screen like shown in the Figure 70.

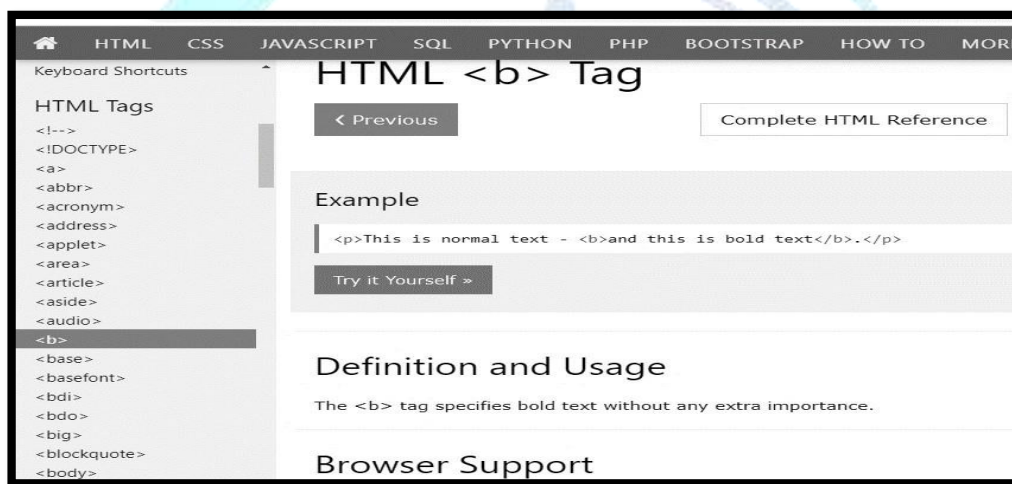


Figure 70: tag on w3school

If you further click on the green button “Try it Yourself”. You will see the details as shown in the Figure 71. Here you can notice, on the left side the code is demonstrated and on the right side, the output of the code is shown.

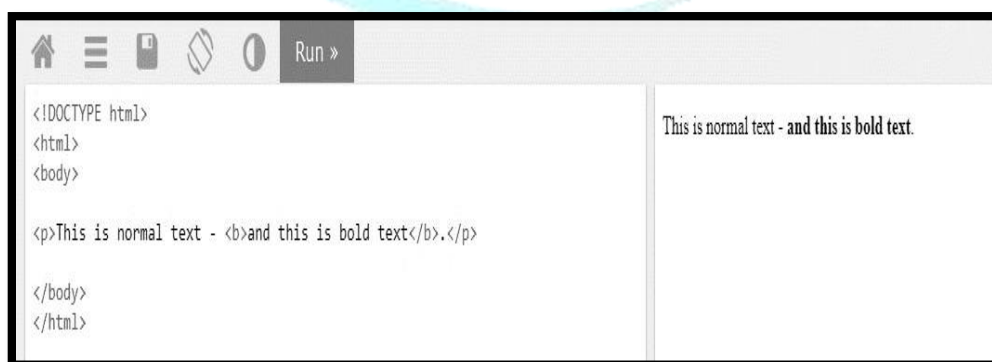


Figure 71: Code and output for tag

Module 75

75. Networking and the Internet: XML

HTML is essentially a notational system by which a text document along with the document's appearance can be encoded as a simple text file. In a similar manner we can also encode non-textual material as text files an example being sheet music. At first glance the pattern of staves, measure bars, and notes in which music is traditionally represented does not conform to the character-by-character format dictated by text files. However, we can overcome this problem by developing an alternative notation system. More precisely, we could agree to represent the start of a staff by `<staff clef = "treble">`, the end of the staff by `</staff>`, a time signature with the form `<time> 2/4 </time>`, the beginning and ending of a measure by `<measure>` and `</measure>`, respectively, a note such as an eighth note on C as `<notes> egth C </notes>`, and so on. Then the text

```
<staff clef = "treble"> <key>C minor</key>
<time> 2/4 </time>
<measure> <rest> egth </rest> <notes> egth G, egth G, egth G </notes></measure>
<measure> <notes> hlf E </notes></measure>
</staff>
```

could be used to encode the music shown in Figure 72 using such notation, sheet music could be encoded, modified, stored, and transferred over the Internet as text files. Moreover, software could be written to present the contents of such files in the form of traditional sheet music or even to play the music on a synthesizer.



Figure 72: The first two bars of Beethoven's Fifth Symphony

Note that our sheet music encoding system encompasses the same style used by HTML. We chose to delineate the tags that identify components by the symbols `<` and `>`. We chose to indicate the beginning and end of structures (such as a staff, string of notes, or measure) by tags of the same name—the ending tag being designated by a slash (a `<measure>` was terminated with the tag `</measure>`). And we chose to indicate special attributes within tags by expressions such as `clef = "treble"`. This same style could also be used to develop systems for representing other formats such as mathematical expressions and graphics.

The eXtensible Markup **Language (XML)** is a standardized style (similar to that of our music example) for designing notational systems for representing data as text files. (Actually, XML is a simplified derivative of an older set of standards called **the Standard Generalized Markup Language**, better known as SGML.) Following the XML standard, notational systems called **markup languages** have been developed for representing mathematics, multimedia presentations, and music. In fact, HTML is the markup language based on the XML standard that was developed for representing webpages. (Actually, the original version of HTML was developed before the XML standard was solidified, and therefore some features of HTML do not strictly conform to XML. That is why you might see references to XHTML, which is the version of

HTML that rigorously adheres to XML.) XML provides a good example of how standards are designed to have wide-ranging applications. Rather than designing individual, unrelated markup languages for encoding various types of documents, the approach represented by XML is to develop a standard for markup languages in general. With this standard, markup languages can be developed for various applications. Markup languages developed in this manner possess a uniformity that allows them to be combined to obtain markup languages for complex applications such as text documents that contain segments of sheet music and mathematical expressions.

Finally, we should note that XML allows the development of new markup languages that differ from HTML in that they emphasize semantics rather than appearance. For example, with HTML the ingredients in a recipe can be marked so that they appear as a list in which each ingredient is positioned on a separate line. But if we used semantic-oriented tags, ingredients in a recipe could be marked as ingredients (perhaps using the tags <ingredient> and </ingredient>) rather than merely items in a list. The difference is subtle but important. The **semantic approach** would allow **search engines** (websites that assist users in locating Web material pertaining to a subject of interest) to identify recipes that contain or do not contain certain ingredients, which would be a substantial improvement over the current state of the art in which only recipes that do or do not contain certain words can be isolated. More precisely, if semantic tags are used, a search engine can identify recipes for lasagna that do not contain spinach, whereas a similar search based merely on word content would skip over a recipe that started with the statement “This lasagna does not contain spinach.” In turn, by using an Internet-wide standard for marking documents according to semantics rather than appearance, a World Wide *Semantic* Web, rather than the World Wide *Syntactic* Web we have today, would be created.

MCQ

Which of the following application may not be considered a network-based application?

Select the correct option

Text editor application

Social media application

News Reader application

Remote access application

Module 76**76. Networking and the Internet: Client Side and Server Side**

Consider now the steps that would be required for a browser to retrieve the simple webpage shown in Figure 69 and display it on the browser's computer screen. First, playing the role of a client, the browser would use the information in a URL (perhaps obtained from the person using the browser) to contact the webserver controlling access to the page and ask that a copy of the page be transferred to it. The server would respond by sending the text document displayed in Figure 69a to the browser. The browser would then interpret the HTML tags in the document to determine how the page should be displayed and present the document on its computer screen accordingly. The user of the browser would see an image like that depicted in Figure 69b. If the user then clicked the mouse over the word here, the browser would use the URL in the associated anchor tag to contact the appropriate server to obtain and display another webpage. In summary, the process consists of the browser merely fetching and displaying webpages as directed by the user.

But what if we wanted a webpage involving animation or one that allows a customer to fill out an order form and submit the order? These needs would require additional activity by either the browser or the webserver. Such activities are called **client-side** activities if they are performed by a client (such as a browser) **or server-side** activities if they are performed by a server (such as a webserver). As an example, suppose a travel agent wanted customers to be able to identify desired destinations and dates of travel, at which time the agent would present the customer with a customized webpage containing only the information pertinent to that customer's needs. In this case the travel agent's website would first provide a webpage that presents a customer with the available destinations. On the basis of this information, the customer would specify the destinations of interest and desired dates of travel (a client-side activity). This information would then be transferred back to the agent's server where it would be used to construct the appropriate customized webpage (a server-side activity), which would then be sent to the customer's browser.

Another example occurs when using the services of a search engine. In this case a user at the client specifies a topic of interest (a client-side activity), which is then transferred to the search engine where a customized webpage identifying documents of possible interest is constructed (a server-side activity) and sent back to the client. Another example occurs in the case of Web mail- an increasingly popular means by which computer users are able to access their email by means of Web browsers. In this case, the webserver is an intermediary between the client and the client's mail server. Essentially, the webserver builds webpages that contain information from the mail server (a server-side activity) and sends those pages to the client where the client's browser displays them (a client-side activity). Conversely, the browser allows the user to create messages (a client-side activity) and sends that information to the webserver, which then forwards the messages to the mail server (a server-side activity) for mailing.

There are numerous systems for performing client-side and server-side activities, each competing with the others for prominence. An early and still popular means of controlling **client-side** activities is to include **programs written in the language JavaScript** (developed by Netscape Communications, Inc.) within the HTML source document for the webpage. From there a browser can extract the programs and follow them as needed. Another approach (developed by Sun Microsystems) is to first transfer a webpage to a browser and then transfer additional program units called applets (written in the language Java) to the browser as requested within the HTML source document. Still another approach is the system Flash (developed by Macromedia) by which extensive multimedia client-side presentations can be implemented.

An early means of controlling **server-side activities was to use a set of standards called CGI (Common Gateway Interface)** by which clients could request the execution of programs stored at a server. A variation of this approach (developed by Sun Microsystems) is to allow clients to cause program units called servlets to be executed at the server side. A simplified version of the servlet approach is applicable when the requested server-side activity is the construction of a customized webpage, as in our travel agent example. In this case webpage templates called **Java Server Pages (JSP)** are stored at the webserver and completed using information received from a client. A similar approach is used by Microsoft, where the templates from which customized webpages are constructed are called **Active Server Pages (ASP)**. In contrast to these proprietary systems, PHP (originally **standing for Personal Home Page** but now considered to mean **PHP Hypertext Preprocessor**) is an open source system for implementing server-side functionality.

Finally, we would be remiss if we did not recognize the security and ethical problems that arise from allowing clients and servers to execute programs on the other's machine. The fact that web servers routinely transfer programs to clients where they are executed leads to ethical questions on the server side and security questions on the client side. If the client blindly executes any program sent to it by a webserver, it opens itself to malicious activities by the server. Likewise, the fact that clients can cause programs to be executed at the server leads to ethical questions on the client side and security questions on the server side. If the server blindly executes any program sent to it by a client, security breaches and potential damage at the server could result.

MCQ

the additional activities which are performed by the server are called **server side**

More Info

While **HTML, CSS, and JavaScript are used for front-end development (client-side)**, **PHP is a server-side** scripting language specifically designed for web development to generate dynamic content on web pages.

body: This tag contains the main content of the webpage, such as text, images, and other elements displayed to the user. It doesn't hold metadata for hidden information.

html: This tag defines the entire HTML document, including both the head and body sections. While it can hold some metadata like the document type, it's not the primary location for non-displayed metadata.

p: This tag defines a paragraph element within the body, specifically for text content. It wouldn't hold hidden metadata.

head: This tag holds metadata about the webpage that is not directly displayed on the user's screen. This includes information like the title, character encoding, author, keywords, descriptions, and other details relevant to the document but not visible to the user.

Module 77

77. Networking and the Internet: Layered Approach to Internet Software (I)

A principal task of networking software is to provide the infrastructure required for transferring messages from one machine to another. In the Internet, this message-passing activity is accomplished by means of a hierarchy of software units, which perform tasks analogous to those that would be performed if you were to send a gift in a package from the West Coast of the United States to a friend on the East Coast (Figure 73). You would first wrap the gift as a package and write the appropriate address on the outside of the package. Then, you would take the package to a shipping company such as the U.S. Postal Service. The shipping company might place the package along with others in a large container and deliver the container to an airline, whose services it has contracted. The airline would place the container in an aircraft and transfer it to the destination city, perhaps with intermediate stops along the way. At the final destination, the airline would remove the container from the aircraft and give it to the shipping company's office at the destination. In turn, the shipping company would take your package out of the container and deliver it to the addressee.

In short, the transportation of the gift would be carried out by a **three-level hierarchy**: (1) the user level (**consisting of you and your friend**), (2) the shipping company, and (3) the airline. Each level uses the next lower level as an abstract tool. (You are not concerned with the details of the shipping company, and the shipping company is not concerned with the internal operations of the airline.) Each level in the hierarchy has representatives at both the origin and the destination, with the representatives at the destination tending to do the reverse of their counterparts at the origin.

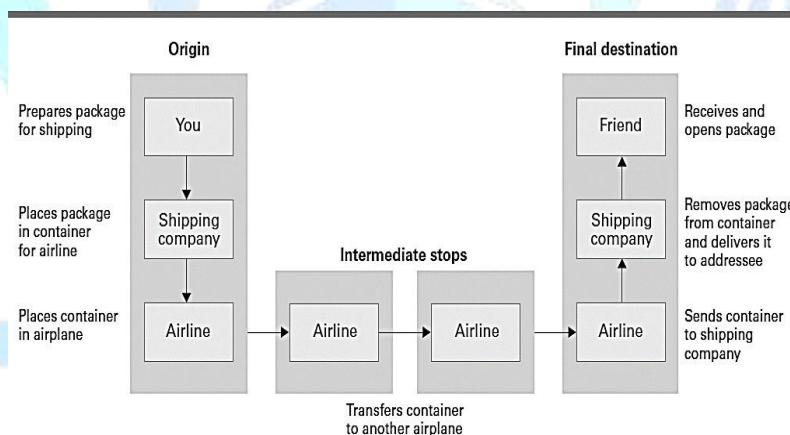


Figure 73: Package-shipping example

Such is the case with software for controlling communication over the Internet, except that the Internet software **has four layers** rather than three, each consisting of a collection of software routines rather than people and businesses. The four layers are known as the **application layer**, the **transport layer**, the **network layer**, and the **link layer** (Figure 74). A message typically originates in the application layer. From there it is passed down through the transport and network layers as it is prepared for transmission, and finally it is transmitted by the link layer. The message is received by the link layer at the destination and passed back up the hierarchy until it is delivered to the application layer at the message's destination.

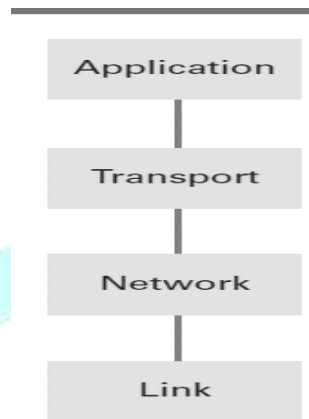


Figure 74: The Internet software layers

ZAINEE
ACADEMY

Module 78

78. Networking and the Internet: Layered Approach to Internet Software (II)

Let us investigate this process more thoroughly by tracing a message as it finds its way through the system (Figure 75). We begin our journey with the application layer.

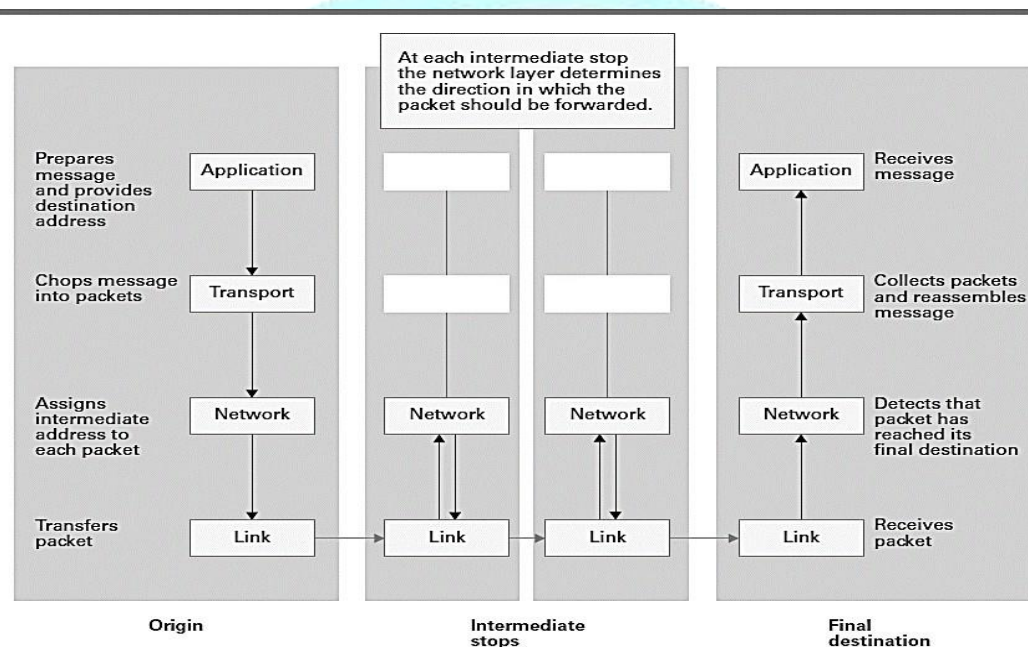


Figure 75: Following a message through the Internet

The application layer uses the **transport layer** to send and receive messages over the Internet in much the same way that you would use a shipping company to send and receive packages. Just as it is your responsibility to provide an address compatible with the specifications of the shipping company, it is the **application layer's** responsibility to provide an address that is compatible with the Internet infrastructure. To fulfill this need, the application layer may use the services of the name servers within the Internet to translate mnemonic addresses used by humans into Internet-compatible IP addresses. An important task of the transport layer is to **accept messages** from the application layer and to ensure that the messages are properly formatted for transmission over the Internet. Toward this latter goal, **the transport layer divides** long messages into small segments, which are transmitted over the Internet as individual units. This division is necessary because a single long message can obstruct the flow of other messages at the Internet routers where numerous messages cross paths. Indeed, small segments of messages can interweave at these points, whereas a long message forces other to wait while it passes (much like cars waiting for a long train to pass at a railroad crossing).

The transport layer adds sequence numbers to the small segments it produces so that the segments can be reassembled at the message's destination. Then it hands these segments, known as **packets**, to the network layer. From this point, the packets are treated as individual, unrelated messages until they reach the transport layer at their final destination. It is quite possible for the packets related to a common message to follow different paths through the Internet.

Module 79**79. Networking and the Internet: Layered Approach to Internet Software (III)**

It is the **network layer's job to decide in which direction a packet** should be sent at each step along the packet's path through the Internet. In fact, the combination of the network layer and the link layer below it constitutes the software residing on the Internet routers. The network layer is in charge of maintaining the router's forwarding table and using that table to determine the direction in which to forward packets. The link layer at the router is in charge of receiving and transmitting the packets.

Thus, when the network layer at a packet's origin receives the packet from the transport layer, it uses its forwarding table to determine where the packet should be sent to get it started on its journey. Having determined the proper direction, the network layer hands the packet to the link layer for actual transmission. The link layer has the responsibility of transferring the packet. Thus, the link layer must deal with the **communication details particular to the individual network in which the computer resides**. For instance, if that network is an Ethernet, the link layer applies CSMA/CD. If the network is a Wi-Fi network, the link layer applies CSMA/CA.

When a packet is transmitted, it is received by the link layer at the other end of the connection. There, the link layer hands the packet up to its network layer where the packet's final destination is compared to the **network layer's forwarding table to determine the direction of the packet's** next step. With this decision made, the network layer returns the packet to the link layer to be forwarded along its way. In this manner each packet hops from machine to machine on its way to its final destination.

Note that only the link and network layers are involved at the intermediate stops during this journey (see again Figure 4.14), and thus these are the only layers present on routers, as previously noted. Moreover, to minimize the delay at each of these intermediate "stops," the forwarding role of the network layer within a router is closely integrated with the link layer. In turn, the time required for a modern router to forward a packet is measured in millionths of a second.

At a packet's final destination, it is **the network layer that recognizes that the packet's journey** is complete. In that case, the network layer hands the packet to its transport layer rather than forwarding it. As the transport layer receives packets from the network layer, it extracts the underlying message segments and reconstructs the original message according to the sequence numbers that were provided by the transport layer at the message's origin. Once the message is assembled, the transport layer hands it to the appropriate unit within the application layer—thus completing the message transmission process.

Determining which unit within the application layer should receive an incoming message is an important task of the transport layer. This is handled by assigning unique **port numbers** (not related to the I/O ports discussed in Chapter 2) to the various units and requiring that the appropriate port number be appended to a message's address before starting the message on its journey. Then, once the message is received by the transport layer at the destination, the transport layer merely hands the message to the application layer software at the designated port number.

Users of the Internet rarely need to be concerned with port numbers because the common applications have universally accepted port numbers. For example, if a Web browser is asked to retrieve the document whose URL is: **<http://www.zoo.org/animals/frog.html>**

The browser assumes that it should contact the HTTP server at www.zoo.org via port **number 80**. Likewise, when sending email, an SMTP client assumes that it should communicate with the SMTP mail server through

Port **number 25**

In summary, communication over the Internet involves the interaction of four layers of software. The application layer deals with messages from the application's point of view. The transport layer converts these messages into segments that are compatible with the Internet and reassembles messages that are received before delivering them to the appropriate application. The network layer deals with directing the segments through the Internet. The link layer handles the actual transmission of segments from one machine to another. With all this activity, it is somewhat amazing that the response time of the Internet is measured in milliseconds, so that many transactions appear to take place instantaneously.



Module 80**80. Networking and the Internet: TCP /IP Protocol Suite**

The demand for open networks has been generated a need for published standards by which manufacturers can supply equipment and software that function properly with products from other vendors. One standard that has resulted is the **Open System Interconnection (OSI)** reference model, produced by the International Organization for Standardization. This standard is based on a seventh-level hierarchy as opposed to the fourth-level hierarchy we have just described. It is an often-quoted model because it carries the authority of an international organization, but it has been shown to replace the fourth-level point of view, mainly because it was established after the fourth-level hierarchy had already become the de facto standard for the Internet.

The TCP/IP protocol suite is a collection of protocol standards used by the Internet to implement the four-level communication hierarchy. Actually, the **Transmission Control Protocol (TCP)** and the Internet Protocol (IP) are the names of only two of the protocols in this vast collection- so the fact that the entire collection is referred to as the TCP/IP protocol suite is rather misleading. More precisely, TCP defines a version of the transport layer. TCP (transmission control protocol)" and **"UDP (user datagram protocol)"** both are the version of transport layer. We say a version because the TCP/IP protocol suite provides for more than one way of implementing the transport layer; one of the other options is defined by the **User Datagram Protocol (UDP)**. This diversity is analogous to the fact that when shipping a package, you have a choice of different shipping companies, each of which offers the same basic service but with its own unique characteristics. Thus, depending on the particular quality of service required, a unit within the application layer might choose to send data via a TCP or UDP version of the transport layer (Figure 76).

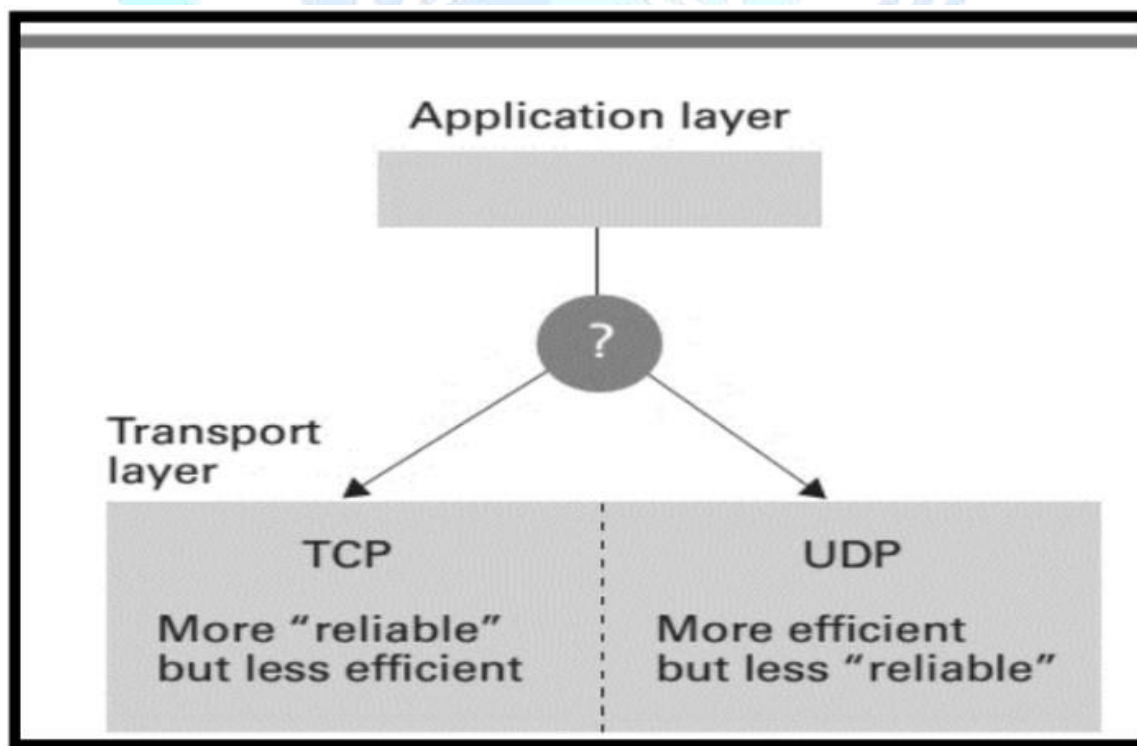


Figure 76: Choosing between TCP and UDP

There are several differences between TCP and UDP. One is that before sending a message as requested by the application layer, a transport layer based on TCP sends its own message to the transport layer at the destination telling it that a message is about to be sent. It then waits for this message to be acknowledged before starting to send the application layer's message. In this manner, a TCP transport layer is said to establish a connection before sending a message. A transport layer based on UDP does not establish such a connection prior to sending a message. It merely sends the message to the address it was given and forgets about it. For all it knows, the destination computer might not even be operational. For this reason, UDP is called a connectionless protocol.

Another difference between TCP and UDP is that TCP transport layers at the origin and destination work together by means of acknowledgments and packet retransmissions to assure that all segments of a message are successfully transferred to the destination. For this reason, **TCP is called a reliable protocol**, whereas UDP, which does not offer such retransmission services, is said to be **an unreliable protocol**.

MCQ

TCP provide both **Flow control** and **Congestion control** during transmission.

Another distinction between TCP and UDP is that TCP provides for both **flow control**, meaning that a TCP transport layer at a message's origin can reduce the rate at which it transmits segments to keep from overwhelming its counterpart at the destination, as well as **congestion control**, meaning that a TCP transport layer at a message's origin can adjust its transmission rate to alleviate congestion between it and the message's destination.

All this does not mean that UDP is a poor choice. After all, a transport layer based on UDP is more streamlined than a layer based on TCP, and thus if an application is prepared to handle the potential consequences of UDP, that option might be the better choice. For example, the **efficiency of UDP** makes it **the protocol of choice** for DNS lookups and VoIP. However, because email is less time sensitive, mail servers use TCP to transfer email.

IP is the Internet's standard for implementing the tasks assigned to the network layer. We have already observed that this task consists **of forwarding**, which involves relaying packets through the Internet, and **routing**, which involves updating the layer's forwarding table to reflect changing conditions. For instance, a router may malfunction, meaning that traffic should no longer be forwarded in its direction, or a section of the Internet may become congested, meaning that traffic should be routed around the blockage. Much of the IP standard associated with routing deals with the protocols used for communication among neighboring network layers as they interchange routing information.

An interesting feature associated with forwarding is that each time an IP network layer at a message's origin prepares a packet, it appends a value **called a hop count**, or time to live, to that packet. This value is a limit to the number of times the packet should be forwarded as it tries to find its way through the Internet. Each time an IP network layer forwards a packet, it decrements that packet's hop count by one. With this information, the network layer can protect the Internet from packets circling endlessly within the system. Although the Internet continues to grow on a daily basis, an initial hop count of 64 remains more than sufficient to allow a packet to find its way through the maze of routers within today's ISPs.

Module 81**81. Networking and the Internet: Security (Forms of Attacks)**

There are numerous ways that a computer system and its contents can be attacked via network connections. Many of these incorporate the use of malicious software (collectively **called malware**). Such software might be transferred to, and executed on, the computer itself, or it might attack the computer from a distance. Examples of software that is transferred to, and executed on, the computer under attack include viruses, worms, Trojan horses, and spyware, whose names reflect the primary characteristics of the software.

A virus is software that infects a computer by inserting itself into programs that already reside in the machine. Then, when the "host" program is executed, the virus is also executed. When executed, many viruses do little more than try to transfer themselves to other programs within the computer. Some viruses, however, perform devastating actions such as degrading portions of the operating system, erasing large blocks of mass storage, or otherwise corrupting data and other programs.

A worm is an autonomous program that transfers itself through a network, taking up residence in computers and forwarding copies of itself to other computers. As in the case of a virus, a worm can be designed merely to **replicate itself** or to perform more extreme vandalism. A characteristic consequence of a worm is an explosion of the worm's replicated copies that degrades the performance of legitimate applications and can ultimately overload an entire network or internet.

A Trojan horse is a program that enters a computer system disguised as a desirable program, such as a game or useful utility package, that is willingly imported by the victim. Once in the computer, however, the Trojan horse performs additional activities that might have harmful effects. Sometimes these additional activities start immediately. In other instances, the Trojan horse might lie dormant until triggered by a specific event such as the occurrence of a preselected date. Trojan horses often arrive in the form of attachments to enticing email messages. When the attachment is opened (that is, when the recipient asks to view the attachment), the misdeeds of the Trojan horse are activated. Thus, email attachments from unknown sources should never be opened.

Another form of malicious software is **spyware** (sometimes **called sniffing software**), which is software that collects information about activities at the computer on which it resides and reports that information back to the instigator of the attack. Some companies use spyware as a means of building customer profiles, and in this context, it has questionable ethical merit. In other cases, spyware is used for blatantly malicious purposes such as recording the symbol sequences typed at the computer's keyboard in search of passwords or credit card numbers. As opposed to obtaining information secretly by sniffing via spyware, **phishing is a technique of obtaining information explicitly by simply asking for it**. The term phishing is a play on the word fishing because the process involved is to cast numerous "lines" in hopes that someone will "take the bait" Phishing is often carried out via email, and in this form, it is little more than an old telephone con. The perpetrator sends email messages posing as a financial institution, a government bureau, or perhaps a law enforcement agency. The email asks the potential victim for information that is supposedly needed for legitimate purposes. However, the information obtained is used by the perpetrator for hostile purposes.

In contrast to suffering from such internal infections as viruses and spyware, a computer in a network can also be attacked by software being executed on other computers in the system. An example is a denial of service **(DoS) attack**, which is the process of overloading a computer with messages. Denial of service attacks have been launched against large commercial web servers on

the Internet to disrupt the company's business and in some cases have brought the company's commercial activity to a halt.

A denial of service attack requires the generation of a large number of messages over a brief period of time. To accomplish this, an attacker usually plants software on numerous unsuspecting computers that will generate messages when a signal is given. Then, when the signal is given, all of these computers (sometimes called a botnet) swamp the target with messages. Inherent, then, in denial of service attacks is the availability of unsuspecting computers to use as accomplices. This is why all PC users are discouraged from leaving their computers connected to the Internet when not in use. It has been estimated that once a PC is connected to the Internet, at least one intruder will attempt to exploit its existence within 20 minutes. In turn, an unprotected PC represents a significant threat to the integrity of the Internet.

Another problem associated with an abundance of unwanted messages is the proliferation of unwanted junk email, called spam. However, unlike a denial of service attack, the volume of spam is rarely sufficient to overwhelm the computer system. Instead, the effect of spam is to overwhelm the person receiving the spam. This problem is compounded by the fact that, as we have already seen, spam is a widely adopted medium for phishing and instigating Trojan horses that might spread viruses and other detrimental software.

MCQ

TCP more reliable and less efficient

_____ refers to the address of a file (resource) accessible on the internet.

Select the correct option

Protocol

Uniform Resource Locator (URL)

HTML

Web Browser

MCQ

Dos attack means

Select the correct option

Division of service

Denial of service

Disconnect to the service

Dental of service

